CS620

Notes for final-term

by "Muhammadians"

For Your kind information (read well):

Asslam-o-Alaikum!

Virtual university's students *"Junaid Waris and Farhan khan"* are here. As there are no official handouts given by the virtual university, the university provides a reading section after every module. We've just copied that and pasted text as well as images. After completion of the midterm notes,(*CS620 midterm notes by muhammadians*) we are going to compiled final term notes which named

"CS620 final term notes by Muhammadians".

Now this is the pdf file of final-term . Try to find out where you got these notes the first time, and if you don't find them, then you can contact us in our official group as well as for current papers(the group link is given below).

We hope you will find it best because it's difficult to read every module's reading material from LMS, so we tried to save your time and headache as well.We compiled this for the welfare of virtual university students. All credits go to only Virtual University, who provide us the material.

Don't print out this pdf file. Only going through, just reading bold material, do not ensure the preparation for good marks, although they do ensure passing marks. Well, I wish you the best of luck for your future! Remember us in your prayers. Thank you so much!

Regards:

- ©"Junaid Waris"
- ©"Farhan Khan"

Whats App Group Link:

https://chat.whatsapp.com/GkX5GZnrdnWDDwWk4QB1ec

(We may respond slowly but respond to you effectively.)

Module No.126:

Collections of Agents
Types of Agents

Agents are typically divided into three main types:

mobile agents that can move about the landscape;

stationary agents that cannot move at all; and

connecting agents that link two or more other agents.

NetLogo

In NetLogo, turtles are mobile agents, patches are stationary agents, and links are connecting agents. From a geometric standpoint, turtles are generally treated as shapeless area-less points, although they may be visualized with various shapes and sizes). Thus, even if a turtle appears to be large enough to be on several patches at the same time, it is only contained by the patch at the turtle ' s center (given by XCOR and YCOR). Patches are sometimes used to represent a passive environment and are acted upon by the mobile agents; other times, they can take actions and perform operations.

Patches and Turtles

A primary difference between patches and turtles is that patches cannot move. Patches also take up a defined space/area in the world; thus, a single patch can contain multiple turtle agents on it.

Links

Links are also unable to move themselves. They connect two turtles that are the " end nodes " of the link, but the visual representations of the links do move when their end nodes move.

Links are often used to represent the relationship between turtles. They can also represent the environment — e.g., by defining transportation routes that agents can move along, or by representing friendship/communication channels. Despite these differences, all three share the ability to behave, that is, to take actions and perform operations on themselves.

Breeds of Agents

Besides these three predefined agent-types, modelers can also create
their own types of agent called breeds. The breed of an agent designates the category or class to which the agent belongs. In the Traffic Basic model, all of the agents are of the same type, so it is not necessary to distinguish between different agent breeds. Instead, we used agents of the " turtles " breed, the default breed in NetLogo.

Different breeds of agents are required if different agents have different properties or actions.

Wolf and sheep

we had two breeds of agents, the “ wolf ” the “ sheep ” breeds. Even though these two breeds had the same set of properties, it was useful to create two breeds because each had different characteristic actions — wolves eat sheep, while sheep eat grass. We defined the breeds at the beginning of the model:

```
breed [sheep a-sheep]
breed [wolves wolf]
```

At the same time, we can define the properties that breeds have:

```
turtles-own [energy]
```

In this case, both the wolves and the sheep have the same property of “ energy, ” but we could also give them separate properties. For instance, we could give the wolves a “ fangstrength ” property and the sheep a “ wooliness ” property. The fang-strength could be used to determine how successful a wolf is at killing sheep. Wooliness, by contrast, could be an indication of how well the sheep was able to fend off the wolves ’ attack, if we assume that the wolves sometimes get a mouthful of wool instead of flesh. If this were the case, we would add two additional lines:

```
sheep-own [ wooliness ]
wolves-own [ fang-strength ]
```

These properties are in addition to the properties that all agents have, so that the sheep would have “ wooliness ” and “ energy ” just as the wolves have “ fang-strength ” and “ energy. ”

Sets of Agents Breeds are particular collections of agents where the collections are defined by the kinds of properties and actions of their agents. We can also define collections of agents in other ways. NetLogo uses the term agentset to designate an unordered collection of agents, and we will also use this term/definition throughout this textbook. Usually we construct agentsets either by collecting agents that have something in common (e.g., agent location or other properties) or by randomly selecting a subset of another agentset.

In the Traffic Basic model, we could create a set of all the agents that have a speed over 0.5. In NetLogo, this is usually done with the WITH primitive. Here is an example of how we could create such an agentset that we could insert into the GO procedure in this model:

```
let fast-cars turtles with [speed > 0.5]
```

However, we usually want to ask these turtles to do something specific. For instance, we can ask all of the turtles that have a high speed to set their size bigger so they are easier to see.

```
let fast-cars turtles with [speed > 0.5]
ask fast-cars [
  set size 2.0
]
```

The “ let ” statement above collects the fast cars into an agentset. If we don’t plan to “ talk to ” this agentset again, we can do without the LET and instead construct the agentset “ on the fly ” and ask the turtles directly:

```
ask turtles with [speed > 0.5] [
    set size 2.0
]
```

If you insert this code into the model, you will soon see that all of the cars are big. This is because we never told the turtles to set their size to small again, once their speed has dropped below 0.5. To do that we would need to add some more code. One way to accomplish this would be to use another ask.

```
ask turtles with [speed > 0.5] [
    set size 2.0
]
ask turtles with [speed <= 0.5] [
    set size 1.0
]
```

Another way to accomplish the same goal, without creating agentsets, is to ask all the turtles to do something, but choose what actions they take based on their properties. In the previous example, all of the faster turtles took their actions before any of the slower turtles took their actions. In the following example, all of the agents are asked with the same ASK command, so the order in which the turtles take actions will be random. In this case the result of running the code will be the same, but depending on what actions the agents take (for instance, if they were to change their speed, instead of their size), the results could be different.

```
ask turtles [
    ifelse speed > 0.5 [
set size 2.0
]
[
    set size 1.0
    ]
]
```

Another method for creating agentsets is on the basis of their location. The Traffic Basic model does this in the GO procedure:

```
let car-ahead one-of turtles-on patch-ahead 1
        ifelse car-ahead != nobody
            [ slow-down-car ]
```

TURTLES-ON PATCH-AHEAD 1 creates an agentset of all of the cars in the patch in front of the current car. It there is at least one such car, it then selects one of these cars at random, using the ONE-OF primitive, and sets the speed of the current car to a little less than the speed of that car ahead. There are many other ways to access a collection of agents based on their location, such as using NEIGHBORS, TURTLES-AT, TURTLES-HERE, and IN-RADIUS.

Agentsets and Lists Throughout our examples, we have used both agentsets and lists, sometimes explicitly and sometimes implicitly. This may be a good time to stop and clarify what they are, how they work, how they differ, and under what circumstances we would use one over the other. Agentsets and lists are both variables that can contain one or many other variables. We can specify that a variable is an agentset or a list when we create them, by using their respective constructor reporters. If we want to create an empty list, we can write

```
let a-list []
```

If we want, we can then add numbers, strings, and even turtles to the list.

```
;; we put items at the start of the list
set a-list fput 1 a-list
set a-list fput “and” a-list
set alist fput turtle 0 a-list
show a-list
;; prints [(turtle 0) “and” 1
```

In contrast to lists, which can hold any type of item, an agentset can hold only agents. Moreover, it can hold only agents of the same type, such as turtles, patches, or links. NetLogo has special reporters for empty agentsets, no-turtles, no-patches, and no-links.

```
;; this creates an empty agentset of turtles
let an-agentset no-turtles
```

no-turtles is an empty turtle agentset (i.e., an agentset containing no turtles), so by setting an-variable to no-turtles, we specify that this variable is of the type agentset. Now that we have an empty agentset, we can then add turtles to it by using the turtle-set reporter:

```
;; this adds turtle 0 to the agentset
set an-agentset (turtle-set turtle 0)
;; this adds turtle 1 and turtle 2 to the agentset
set an-agentset (turtle-set an-agentset turtle 1 turtle 2)
show an-agentset
;; prints (agentset, 3 turtles)
```

We can only put agents (turtle-breeds, patches and links) in agentsets, but we can put anything we want, including agents, in lists. There are two important properties that set agentsets and lists apart from each other.

First, we can “ ask ” agentsets to do things, but we cannot ask lists to do things. So for instance, when we use

```
ask turtles [] ;; do stuff
```

what we are really doing is first creating an agentset of all turtles, and then asking all those turtles, in a random order, to do whatever we put in the brackets. If we try this with lists, we will simply get an error.

Second, agentsets are unordered. This means that whenever we invoke them, they will output a list of agents in a random order. Notice how showing a list shows both what is inside the list and the order in which it appears, but showing an agentset shows only what is inside the agentset.

So when we want to interact with turtles, when would we use one or the other? Unless we have a very good reason to, we always use agentsets. The primary reason is that we usually want our turtles to do things in a random order. That is because there could be threats to model validity if the agents always execute in the same order. For instance, in the Wolf Sheep Predation model, some sheep would have an unfair advantage if they are always first to check if there is grass on their patches.

But sometimes we do want to determine the order in which turtles do things. For instance, it could be that we want some turtles to have an advantage. In that case we

would need to create an ordered list, and order them by whatever parameter we think determines their advantage. If turtles have different information, we might, for example, want the turtles with more information to take their turns later.

```
turtles-own [information] ;; information determines how late they get their turn
;; (later is better because then they will know what everyone else did
;; before making their decision)

;; create an empty list
let a-list []
;; sort-on reports a list containing turtles sorted by the parameter specified
;; in the brackets
set a-list sort-on [information] turtles
```

a-list now contains all turtles sorted by information in ascending order. But, as we just discussed, we cannot ask lists of agents to do things. Rather, we must iterate through the lists, and ask each agent in the list to do what we ask. For this we can use the foreach command:

```
foreach a-list [
    ask ? [ ] ;; do stuff here
]
```

The " ? " is a special variable that takes on the value of each element of the list. So this code will iterate through each turtle in the list, and ask each in that particular order, to do what we specify in the brackets.

Similarly we can create lists of patches. Suppose we want to label the patches with numbers in left-to-right, top-to-bottom order. We can create a sorted list of patches and then label them using the code below:

```
;; patches are labeled with numbers in left-to-right,
;; top-to-bottom order
let n 0
foreach sort patches [
    ask ? [
       set plabel n
       set n n + 1
    ]
]
```

Agentsets and Computation Before we end our discussion of agentsets, it should be noted that once you ask an agentset to perform an action, all the agents collected at that moment (and only those agents) will perform the action. If one agent ' s (agent A)

action causes another agent (agent B) in the agentset to no longer satisfy the collection criteria, B will still perform the action. Likewise, if A ' s action causes agent C, which is not in the agentset, to meet the collection criteria, C will still not perform the action.

Agent-set Ordering model

As an illustration, let ' s consider this code snippet from the Agentset Ordering model in the chapter 5 subfolder of the IABM Textbook folder in the models library:

```
to setup
    clear-all
    create-turtles 100 [
       set size random-float 2.0
       forward10
    ]
end

to go
    ask turtles [
       set color blue
       ]
    ask turtles with [ size < 1.0 ] [
       ask one-of turtles with [size > 1.0] [
           set size size - 0.5
       ]
       set color red
    ]
    print count turtles with [ color = red ]
    print count turtles with [ size < 1.0 ]
end
```

The bolded code has the potential to change the set of agents with size < 1.0 by making some agents smaller, thereby changing the agentset defined in the first ASK.

Agent-set Efficiency model

```
;; GO-1 sets red patch labels to a small random number and
;; green patch labels to a larger random number
to go-1
    if any? patches with [ pcolor = red] [
       ask patches with [ pcolor = red] [
              set plabel random 5 ;; red patches are labeled 0-4
       ]
    ]
    if any? patches with [ pcolor = green] [
       ask patches with [ pcolor = green] [
              set plabel 5 + random 5 ;; green patches are labeled 5-9
       ]
    ]
    tick
end
```

This code computes each of the two agentsets PATCHES WITH [PCOLOR = RED] and PATCHES WITH [PCOLOR = GREEN] twice.

```
;; GO-2 has the same behavior as go-1 above. But it is more efficient as it
computes each of the patch agentsets only once.
to go-2
 let red-patches patches with [ pcolor = red ]
 let green-patches patches with [ pcolor = green ]
 if any? red-patches [
     ask red-patches [
    set plabel random 5
    ]
]
if any? green-patches [
    ask green-patches [
    set plabel 5 + random 5
    ]
]
 tick
end
```

The procedure GO-2 below has the same behavior as GO-1, but it is more efficient in that it computes each of those agentsets only once.

A more pernicious issue arises if we change just one line of GO-1 and GO-2:

```
;; GO-3 shows what happens if patch colors are changed "on the fly"
to go-3
    if any? patches with [ pcolor = red] [
       ask patches with [ pcolor = red] [
              set pcolor green
       ]
    ]
    if any? patches with [ pcolor = green] [
       ask patches with [ pcolor = green] [
              set pcolor red
       ]
    ]
    tick
end
```

If you run this code, you might expect to get a picture that looks like figure 5.4; that is, you expect red patches to turn green and vice versa.

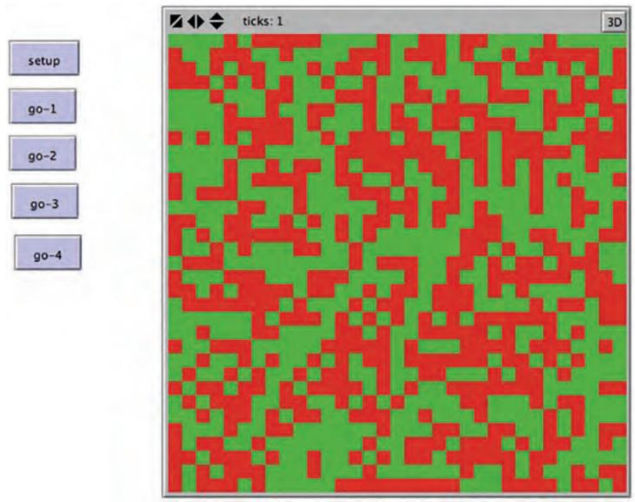


Figure 5.4
Constructing agentsets at the outset results in expected behavior.

In fact, this is not what happens. Instead, this code will result in a picture like figure 5.5, where all the patches are red. Why does this unexpected behavior happen? This is another example of the ordering issue we just discussed above. The first " if

statement ” turns the patches green, so by the time the second “ if statement ” is executed, all the patches are green and therefore then turn red.

By computing the agentsets ahead of time, as in GO-4, you are not only using more efficient code but are also ensuring that the green-patches agentsets you ask to execute the instructions are the same green-patches agentsets as at the start of the procedure and not the set of green patches that result from the first “ if-statement, ” resulting in the expected picture of figure 5.4.

```
;; GO-4 shows what happens if you keep track, at the outset, of which patches
;; are red and which are green
to go-4
         let red-patches patches with [pcolor = red]
         let green-patches patches with [pcolor = green]
         if any? red-patches [
             ask red-patches [
                 set pcolor green
             ]
         ]
         if any? green-patches [
             ask green-patches [
                 set pcolor red
             ]
         ]
    tick
end
```

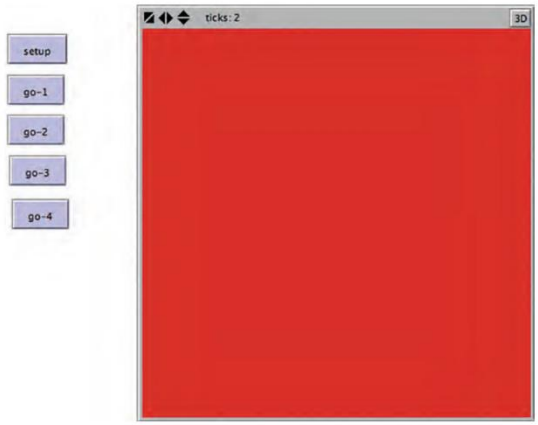


Figure 5.5
Constructing agentsets after the model state has changed, results in unintended outcome.

Module No.127:

The Granularity of an Agent

One of the first considerations when designing an agent-based model is at what granularity should you create your agent — at what level of complexity is the agent you are modeling.

Level of Complexity for the Agents

So how do you choose the level of complexity for the agents in your model? The guideline is to choose agents such that they represent the fundamental level of interaction that pertains to your question about the phenomenon.

Tumor Model

For instance, if you look at the Tumor model (shown in figure 5.6) in the Biology section of the NetLogo models library, the agents are cells within the human body, since the research question the model examines is how tumor cells spread.

NetLogo Tumor model

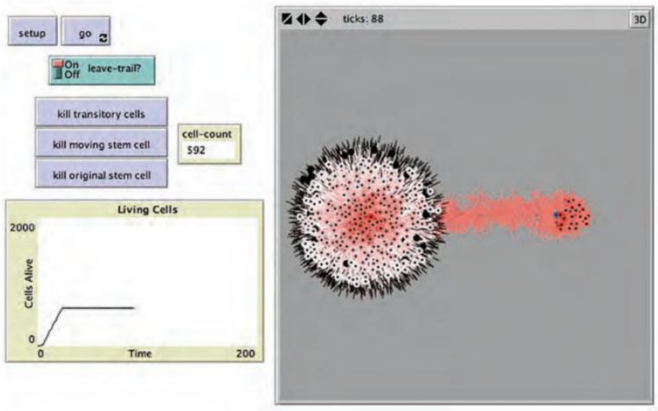


Figure 5.6
NetLogo Tumor model. http://ccl.northwestern.edu/netlogo/models/Tumor (*Wilensky, 1998b*).

However, even though the AIDS model in the Biology section of the NetLogo models library (see figure 5.7) is also concerned with disease, the basic agents are humans instead of cells.

NetLogo AIDS model

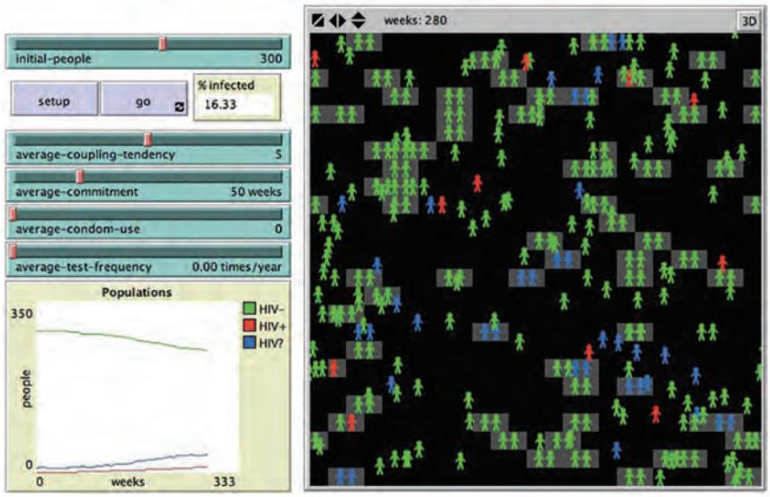


Figure 5.7
NetLogo AIDS model. http://ccl.northwestern.edu/netlogo/models/AIDS (*Wilensky, 1997a*).

Question Of Concern

This is because the AIDS model is more concerned with how the disease spreads between humans instead of how it spreads within the human body. In the Tumor model, the question of concern is how cells interact to create cancer.

Thus, the most important interactions happen at the cell level, and this makes cells a good choice for agents In the AIDS model, the question of concern is how humans interact to spread disease, with humans as the agents being the most appropriate choice.

Choices Of Granularity

These two models highlight the relationship between the question being investigated, the interactions being modeled, and the resulting choices of granularity for the agents.

Level Of Complexity

Both individual cells and groups of cells can be valid choices for the level of complexity of these agents, and deciding between them will depend on details of the question you are asking as well as considerations of computational complexity.

Module No.128:

Agent Cognition

As we have described, agents have different properties and behaviors. Still, how do agents examine their properties and the world around them to decide what actions to take? This question is resolved by a decision-making process called agent cognition.

Types Of Agent Cognition

We will discuss several types of agent cognition:

- reflexive agents,
- utility-based agents,
- goal-based agents, and
- adaptive agents (Russell & Norvig, 1995).

Often, these types of cognition are thought of in increasing order of complexity, with reflexive agents being the simplest, and adaptive agents the most complex. In reality, though this is not a strict hierarchy of complexity.

Instead, these are descriptive terms that help us talk about agent cognition and can be mixed and matched; for instance, it is possible to have a utility-based adaptive agent.

Reflexive
Reflexive agents are built around very simple rules. They use if-then rules to react to inputs and take actions (Russell & Norvig, 1995). For instance, the cars in the Traffic Basic model are reflexive agents. If you look at the code that controls their actions, it looks like this:

```
let car-ahead one-of turtles-on patch-ahead 1  ;;choose a car on the patch ahead
ifelse car-ahead != nobody [  ;; if there is a car ahead
    slow-down-car car-ahead ;; set car speed to be slower than car ahead
]
[   ;; otherwise, speed up
    speed-up-car ;; increase speed variable by the acceleration
]
;; ….
fd speed
```

Put in words, this code says that if there are cars ahead, then slow down below the speed of the car in front; if there are not any cars ahead, then speed up. This is a reflexive action based on the state of the program, hence the name reflexive agent. This is the most basic form of agent cognition (and often a good starting point), but it is possible to make the agent cognition more sophisticated.

Utility-based Form of Agent Cognition

For instance, we could elaborate this model by giving the cars gas tanks and fuel efficiencies based on the speed they are going. We could then have the cars change their speeds in order to improve their fuel efficiencies. As a result, the agents might have to speed up and slow down at different times than they currently do to minimize their gas usage while still not causing accidents. Giving the agents this type of decision-making process would give them a utility-based form of agent cognition in which they attempt to maximize a utility function — namely, their fuel efficiency (Russell & Norvig, 1995).

Implementation Of Model Of Agent Cognition

To implement this model of agent cognition, we need to start by replacing our SPEED-UP-CAR procedure with a new procedure that accounts for the car ' s fuel efficiency.

```
;; choose a car on the patch ahead
let car-ahead one-of turtles-on patch-ahead 1
ifelse car-ahead != nobody [ ;; if there is a car ahead
    slow-down-car car-ahead  ;; set car speed to be slower than car ahead
]
[ ;; otherwise, adjust speed to find ideal fuel efficiency
    adjust-speed-for-efficiency ]
```

We want the ADJUST-SPEED-FOR-EFFICIENCY procedure to maintain the same speed if the car is at the maximally fuel efficient speed. If it is not at its most efficient speed, the logic in this procedure should have the car speed up if it is moving too slow and slow down if the car is moving too fast. Additionally, the car would still need to slow down if it was about to crash into the car in front of it. As such, the true utility function includes an exception that gives a utility of 0 to any action that results in a crash. As a result, we can leave the first part of the code that slows the car before it crashes and just add ADJUST-SPEED-FOR_EFFICIENCY when there is no car directly ahead, as illustrated in the code below from the Traffic Basic Utility model, of the NetLogo models library.

```
;; car procedure
to adjust-speed-for-efficiency
 if (speed != efficient-speed) [ ;; if car is at efficient speed, do nothing
         if (speed + acceleration < efficient-speed) [
;; if accelerating will still put you below the efficient speed then accelerate
             set speed speed + acceleration
         ]                                          ;;
;; if decelerating will still put you above the efficient speed then decelerate
         if (speed - deceleration > efficient-speed) [
             set speed speed - deceleration
         ]
    ]
end
```

Utility Functions

In the language of utility functions, each car agent is minimizing a function f,

defined by:

$$f(v) = |v - v^*|$$

where v is the current velocity of the car and v* is the most efficient velocity. The constraint that is described in the code is that the v cannot be adjusted arbitrarily, but instead, can only be changed by a limited increment every time step. By trying different values for EFFICIENT-SPEED, you can obtain quite different traffic patterns from the original model. For low values of EFFICIENT-SPEED, the system can achieve a free-flow state (with no jams) on a consistent basis.

Goal-based Agents

The third type of agent cognition we will discuss is goal-based cognition. Imagine that each car in the Traffic Grid model (see figure 5.8) from the social sciences section of the NetLogo models library has a home and a place of work, with the goal of moving from home to work in a reasonable amount of time. Now, not only do the agents have to be able to speed up and slow down, but they also have to be able to turn left and right. In this version of the model, the cars are goal-based agents, since they have a goal (getting to and from work) that they are using to dictate their actions.

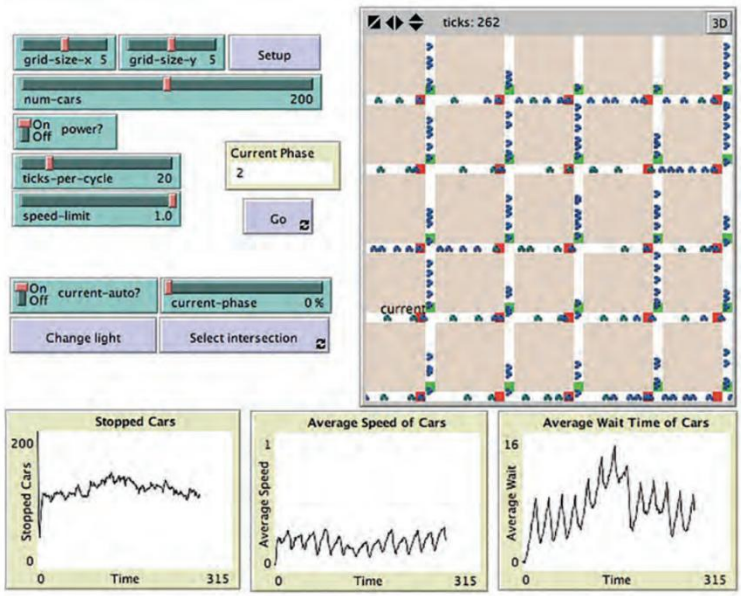


Figure 5.8
Traffic Grid model. http://ccl.northwestern.edu/netlogo/models/TrafficGrid (*Wilensky, 2002b*).

Implementing The Goal-based Version of the Model

To implement the goal-based version of the model, we start by defining the procedure that the cars will use to navigate between their two desired destinations. Instead of just moving straight all the time, as with the cars in the original model, the new model should have the car at each step deciding which of its neighboring road patches is the closest to its destination, subsequently proceeding in that direction. We do this in the procedure called NEXT-PATCH. First, each car checks if it has arrived at its goal, and if so, switches its goal:

```
;; if I am going home and I am on the patch that is my home
;; I turn around and head towards work (my goal is set to "work")
    if goal = house and patch-here = house [
       set goal work
    ]
;; if I am going to work and I am on the patch that is my work
;; I turn around and head towards home (my goal is set to "home")
    if goal = work and patch-here = work [
       set goal house
    ]
```

The code above doesn ' t quite work. That ' s because the house and work are off-road and the cars remain on the road, so the condition " patch-here = house " or " patch-here = work " will never be satisfied. As we placed the house and work adjacent to the road, we can fix this by having the car check if it is right next to its goal.

```
;; if I am going home and I am next to the patch that is my home
;; I turn around and head towards work (my goal is set to "work")
if goal = house and (member? patch-here [neighbors4] of house) [
      stay set goal work
    ]
;; if I am going to work and I am next to the patch that is my work
;; I turn around and head towards home (my goal is set to "home")
    if goal = work and (member? patch-here [neighbors4] of work) [
       stay set goal house
    ]
```

Having established a goal, each car then chooses an adjacent patch to move to that is the closest to its goal. It does this by choosing candidate patches to move to (adjacent patches on the road) and then selecting the candidate closest to its goal.

The resultant NEXT-PATCH procedure is:

```
;; establish goal of driver
to-report next-patch
;; if I am going home and I am next to the patch that is my home
;; I turn around and head towards work (my goal is set to "work")
if goal = house and (member? patch-here [neighbors4] of house) [
      stay set goal work
    ]
;; if I am going to work and I am next to the patch that is my work
;; I turn around and head towards home (my goal is set to "home")
    if goal = work and (member? patch-here [neighbors4] of work) [
       stay set goal house
    ]
;; CHOICES is an agentset of the candidate patches which the car can
;; move to (white patches are roads, green and red patches are lights)
let choices neighbors with [pcolor = white or pcolor = red or pcolor = green]
         ;; choose the patch closest to the goal, this is the patch the car will move to
    let choice min-one-of choices [distance [goal] of myself]
;; report the chosen patch
    report choice
end
```

Adaptive Agent

One of the powerful advantages of agent-based modeling is that agents can change not only their decisions but also their strategies (Holland, 1996).

Adaptive Agent: An agent that can change its strategy based on prior experience is an adaptive agent. Unlike conventional agents, which will also do the same thing when presented with the same circumstance, adaptive agents can make different decisions if given the same set of inputs.

Traffic Basic model

In the Traffic Basic model, cars do not always take the same action; they operate differently based on the cars around them by either slowing down or speeding up. However, regardless of what has happened to the cars in the past (i.e., whether they got stuck in traffic jams or not) they will continue
to take the same actions in the same conditions in the future. To be truly adaptive, agents need to be able to change not only their actions in time, but also their strategies.

They must be able to change how they act because they have encountered a similar situation in the past and can react differently this time based on their past experience.

In other words, the agents learn from their past experience and change their behavior in the future to account for this learning.

Example Of Adaptive Cognition

For instance, the agent could have observed that in the past that, when there was a car five patches ahead of it, it braked too quickly, even though it could have waited longer to brake without hitting the other car. In the future, it could change its rule for braking to brake only when a car is four patches ahead. This agent is now an adaptive **agent** because it not only modifies its actions, but also, its strategies.

Module No.129:

Other Kinds of Agents

We have already discussed breeds and various types of agents, but there are two other special kinds of agents that deserve at least a brief mention.

- The first type are meta-agents, agents made up of other agents;
- The second is proto-agents, placeholder agents that allow you to define interactions for your fully defined agents with other entities that have not been fully developed.

Meta-Agents Many things that we would consider agents in reality are actually composed of other agents. In reality, all " agents " are composed of other agents; to cite the oft-told parable, " It ' s turtles all the way down. " In other words, there is always a lower level of detail that you can use to describe an agent in your model, at least until you reach the most basic level so far described by physics. For instance, if we have selected a human to be the agent, he or she is actually composed of many subagents, with these subagents in turn being different depending on how you view the human. You could view the subagents of a human as the systems of the body — e.g., the immune and the respiratory systems — or you could view the subagents of a human as psychological aspects like the intellect and emotion. Moreover, these subagents are not the most basic level; in our example, the systems of the body are made up of organs, tissues, and cells.

Meta-agents and Sub-agents

At each of these levels, we are describing the relationship between meta-agents (agents composed of other agents) and subagents (agents which compose other agents). Still, agents can also be both meta-agents and subagents at the same time. For instance, organs are composed of cells and are meta-agents. However, they also play a compositional role in the systems of the human body and are thus subagents.

We can use meta-agents in our ABMs by defining the subagents that make up our agents and providing them with their own actions and properties.

Meta-agent Appears to be a Single Agent

From the perspective of another meta-agent, a meta-agent appears to be a single agent. If a person meets another person, they do not (usually) directly interface with the heart or the lungs (unless the meeting takes place at the surgical table). Instead, they interface with the person as a whole.

Modeling Agents and Their Interactions

When we think of modeling agents and their interactions, we have to determine what level of granularity we want to describe the agent behaviors. However, we always have the option of refining our model by converting agents into meta-agents that describe the agents that constitute other agents. Sometimes, it can be useful to represent the agents in our models not as autonomous individuals, but instead, as meta-agents composed of other agents. NetLogo doesn ' t include explicit language support for these meta-agents, though there are commands for locking together the movement of several agents (e.g., the TIE command) that may be useful in some circumstances. However, there is nothing to prevent you from designing models that have groups of agents representing single agents.

Proto-Agents

To truly be an agent within the agent-based modeling framework, an entity must have its own properties or actions. Sometimes, though, it can be desirable to create agents that, rather than having their own properties or behaviors, are instead placeholders for future agents. We call these agents proto-agents, and their primary purpose is to enable us to specify how other agents would interact with them if they

were fleshed out into full agents.

Example of Proto-agent

For instance, if you were creating a model of residential location decision making, you would have residents as agents; however, since where a resident lives is greatly influenced by places of employment and services (e.g., grocery stores and restaurants) you might also want to include these " service centers " as agents as well. The same level of detail necessary for the residents, who are the focal agents of concern, may not be necessary for the service centers. Instead, they might be rendered as placeholders to represent where residents might potentially find jobs and transact business. However, as you continue to refine the model, you might give the service centers additional decision-making abilities. For instance, they might have a more elaborate model of market demand and decide where to locate using their own properties and beliefs about the future growth of the world. Keeping these agents as proto-agents early on means that when you add in the more richly detailed versions to the model, you do not have to go back and revise your resident agents. Instead, you can use the interaction events that they had already been using to interface with the service center proto-agents.

Module No.130:

Environments

Another early and critical decision is how to design the environment of the agent-based model. The environment consists of the conditions and habitats surrounding the agents as they act and interact within the model. The environment can affect agent decisions, and, in turn, can be affected by agent decisions.

Environment Can Affect Agent

For instance, in the Ants model from chapter 1, the ants leave pheromone in the environment that changes the environment, and in turn changes the behavior of the ants. There are many different kinds of environments that are common in ABMs. In this section, we will discuss a few of the most common types of environments.

Implementation of an Environment

Before we discuss the types of environments, it is important to mention that the environment itself can be implemented in a variety of ways. First, the environment can be composed of agents such that each individual piece of the environment can have a full set of properties and actions.

NetLogo

In NetLogo, this is the default view of the environment — the environment is represented by the agentset of patches. This allows different parts of the environment to have different properties and act differently based on their local interactions.

Module No.131:

Spatial Environments

Spatial environments in agent-based models generally have two variants:

- discrete spaces and
- Continuous spaces.

Continuous Spaces: In a mathematical representation, in continuous spaces between any pair of points, there exists another point,

Discrete Spaces: While in discrete spaces, though each point has a neighboring point, there do exist pairs of points without other points between them, so that each point is separated from every other point.

ABM Implementation

However, when implemented in an ABM, all continuous spaces must be implemented as approximations, so that continuous spaces are represented as discrete spaces where the spaces between the points are very small. It should be noted that both discrete and continuous spaces can be either finite or infinite.

Discrete Spaces: The most common discrete spaces used in ABM are lattice graphs (also sometimes referred to as mesh graphs or grid graphs), which are environments where every location in the environment is connected to other locations in a regular grid.

Toroidal Square Lattice

For instance, every location in a toroidal square lattice has a neighboring location up, down, to the left, and to the right. As mentioned, the most common representation of the environment in NetLogo is patches, which are located on a 2D lattice underlying the world of the ABM (see figure 5.9 for a colorful pattern of patches whose code is simply ASK PATCHES [SET PCOLOR PXCOR * PYCOR]). This uniform connectivity makes them different from network-based environments.

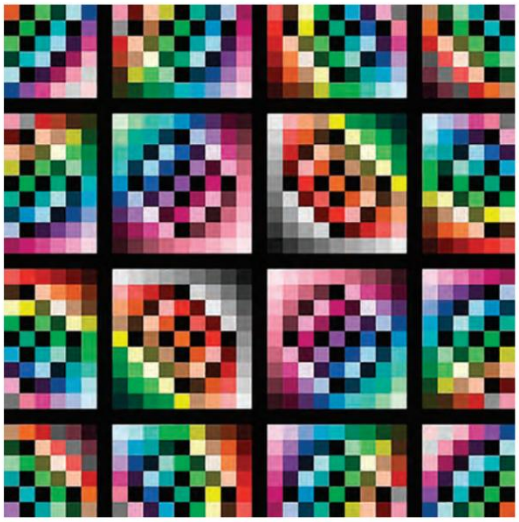


Figure 5.9
Patches displaying a range of colors.

The two most common types of lattices are:

- Square Lattices
- Hexagonal Lattices

Square Lattices The square lattice is the most common type of ABM environment. A square lattice is one composed of many little squares, akin to the grid paper used in mathematics classrooms.

There are two classical types of
neighborhoods on a square lattice:

- The von Neumann neighborhood, consisting of four
  neighbors located in the cardinal directions (see figure 5.10a); and
- The Moore neighborhood,
  comprising the 8 adjacent cells (See figure 5.10b).

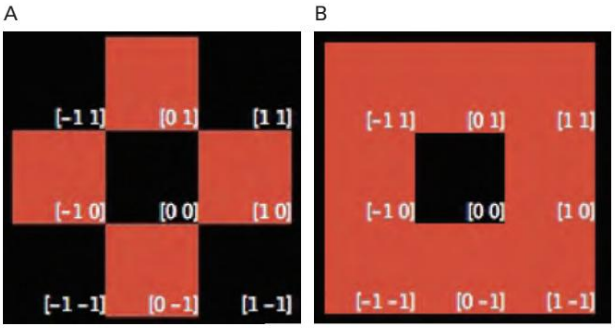


Figure 5.10
(a) Von Neumann Neighborhood. (b) Moore Neighborhood. The black cell in the center is the focal cell, and the red cells comprise its neighborhood.

Von Neumann Neighborhood

A von Neumann neighborhood (named after John von Neumann, a pioneer of cellular automata theory among other things) of radius 1 is a lattice where each cell has four neighbors: up, down, left, and right.

Moore Neighborhood

A Moore neighborhood (named after Edward F. Moore, another pioneer of cellular automata theory) of radius 1 is a lattice in which each cell has eight neighbors in the eight directions that touch either a side or a corner: up, down, left, right, up-left, up-right, down-left, and downright. In general, a Moore neighborhood gives you a better approximation to movement in a plane, and since many ABMs model phenomenon where planar movement is common, it is often the preferred modeling choice for discrete motion.

Hex Lattices: A hex lattice has some advantages over square lattices. The center of a cell in a square grid is farther from the centers of some of the adjacent cells (the diagonally adjacent ones) than other such cells. However, in a hex lattice, the distance between the center of a cell and all adjacent cells is the same. Moreover, hexagons are the polygons with the most edges that tile the plane, and for some applications, this makes them the best polygons to use. Both of these differences (equidistance between centers and the number of edges) mean that hex lattices more closely approximate a continuous plane than square lattices. But because square lattices match more closely a Cartesian coordinate system, a square lattice is a simpler structure to work with; even when a hexagonal lattice would be superior, many ABMs and ABM toolkits nevertheless employ square lattices.
However, with a little effort, any modern ABM environment can simulate a hexagonal lattice in a square lattice environment. For instance, you can see a hex lattice environment in the NetLogo Code examples, Hex Cells example, and Hex Turtles example (see figures 5.11 and 5.12).

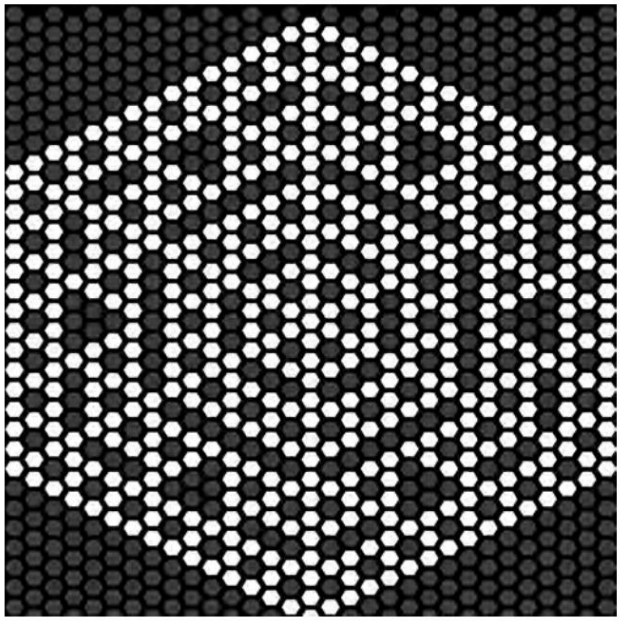


Figure 5.11
Hex Cells example from the NetLogo models library.

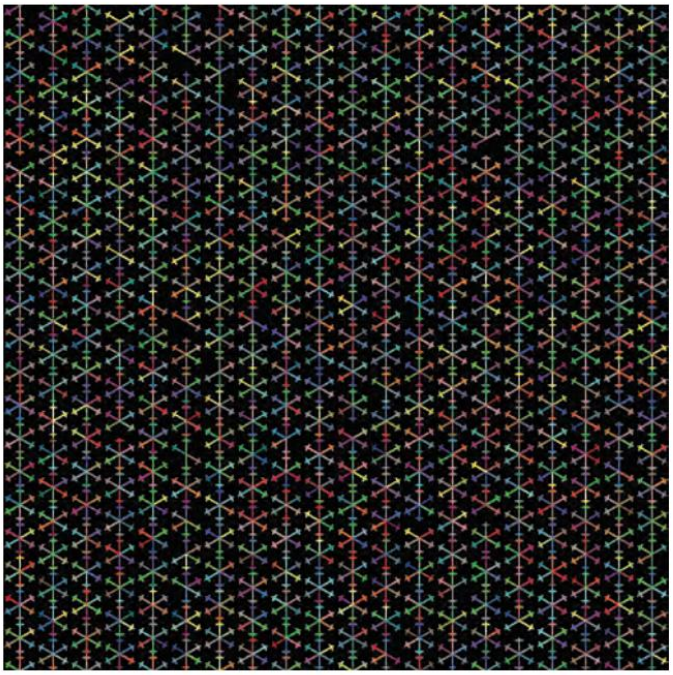


Figure 5.12
Hex Turtles example from the NetLogo models library.

In the Hex Cells example, each patch in the world maintains a set of six neighbors, with the agents located on each patch having a hexagonal shape. In other words, the world is still rectangular, but we have defined a new set of neighbors for each patch. In the Hex Turtles example, the turtles have an arrow shape and start along headings that are evenly divisible by 60 degrees; when they move they only move along these same 60-degree angles.

Continuous Spaces

- In a continuous space, there is no notion of a cell or a discrete region within the space. Instead, agents within the space are located at points in the space. These points can be as small as the resolution of the number representation allows.
- In a continuous space, agents can move smoothly through the space, passing over any points in between their origin and destination, whereas in a discrete space, the agent moves directly from the center of one cell to another.
- Because computers are discrete machines, it is impossible to exactly represent a continuous space. We can, however, represent it at a level of resolution that is very fine.
- In other words, all ABMs that use continuous spaces are actually using a very detailed discrete space, while the resolution is usually high enough that to suffice for most purposes.

Boundary Conditions One other factor that comes into play when working with spatial environments is how to deal with boundaries, an issue for Hex and Square lattices as much as for continuous spaces. If an agent reaches a border on the far left side of the world and wants to go farther left, what happens?

Three Standard Approaches

There are three standard approaches to this question, referred to as topologies of the environment:

(1) it reappears on the far right side of the lattice (toroidal topology);

(2) it cannot go any farther left (bounded topology); or

(3) it can keep going left forever (infinite plane topology).

Toroidal Topology

A toroidal topology is one in which all of the edges are connected to another edge in a regular manner. In a rectangular lattice, the left side of the world is connected to the right side, while the top of the world is connected to the bottom. Thus, when agents move off the world in one direction, they reappear at the opposite side. This is sometimes called wrapping, because the agents wrap off one side of the world and onto another.

In general, using a toroidal topology means that the modeler can ignore boundary conditions, which usually makes model development easier. If the world is nontoroidal, then modelers have to develop special rules to handle what to do when an agent encounters a boundary in the world. In a spatial model, this conflict is one of whether the agent should turn around simply take a step backward. Indeed, the most commonly used environment for an ABM is a square toroidal lattice.

Bounded Topology

A bounded topology is one in which agents are not allowed to move beyond the edges of the world. This topology is a more realistic representation of some environments. For example, if you are modeling agricultural practices, it is unrealistic for a farmer to be able to keep driving the tractor east and then end up back on the west side of the field. Using a torus environment may affect the amount of fuel required to plow the fields, so a bounded topology might be a better choice depending on the questions the model is trying to address. It is also possible to have some of the limits of the world be bounded while others are wrapping. For instance, in the Traffic Basic model, where the cars only drive from left to right, the top and bottom of the world are bounded.

Infinite-plane Topology

Finally, an infinite-plane topology is one where there are no bounds. In other words, agents can keep moving and moving in any direction forever. In practice, this is done by starting with a smaller world such that whenever agents move beyond the edges of the world, the world is expanded. At times, infinite planes can be useful if the agents truly need to move in a much larger world. While some ABM toolkits provide built-in support for infinite plane topologies, NetLogo does not. However, it is possible to work around this limitation by giving each turtle a separate pair of x and y coordinates from the built-in ones. Then, when an agent moves off the side of the world, we can hide the turtle and keep updating this additional set of coordinates until the agent moves back onto the view. In most cases, however, a toroidal or bounded topology will be the more appropriate (and simpler) choice to implement.

Module No.132:

Network-Based Environments

Link: A link is defined by the two ends it connects, which are frequently referred to as nodes. However, we will use the network/node/link vocabulary throughout this module.

In NetLogo, links are their own agent-type.

Lattice networks

In fact, lattice graphs can also be called lattice networks, with the property that each position in the network looks exactly like every other position in the network. However, ABM environments usually do not implement lattice environments as networks for both conceptual and efficiency reasons. Additionally, using patches as the default topology allows for either discrete or continuous representations of the space, whereas a network is always discrete.

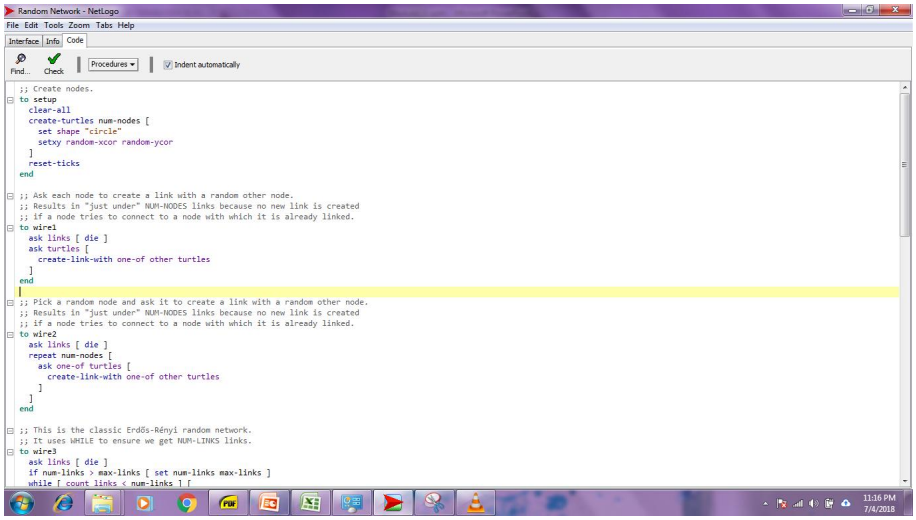
Random Networks

Network-based environments have been found useful in studying a wide variety of phenomena, such as the spread of disease or rumors, the formation of social groups, the structure of organizations, or even the structure of proteins. There are several

network topologies that are commonly used in ABM. Besides the regular networks described earlier, the three most common network topologies are random, scale-free, and small-world.

In random networks, each individual is randomly connected to other individuals. These networks are created by randomly adding links between agents in the system. For example, if you had a model of agents moving around a large room and connected every agent in the room to another agent based on which agent had the next largest last two digits of their social security number, you would probably create a random network. We show one simple methods for creating a random network. This code is also in the Random Network model in the chapter 5 subfolder of the IABM Textbook folder of the NetLogo models library

Code for creating a random network



Scale-free networks have the property that any subnetwork of the global network has the same properties as the global network. A common way to create this type of network is by adding new nodes and links to the system so that extant nodes with a large number of links are more likely to receive new connections (Barab á si, 2002). This technique is sometimes called preferential attachment, since nodes with more connections are attached to preferentially. This method for network creation tends to produce networks with central nodes that have many radiating links; because of the resemblance to a bicycle wheel, this network structure is sometimes also called hub-and-spoke. Many real-world networks such as the Internet, electricity grids, and airline routes have similar properties to a scale-free network.

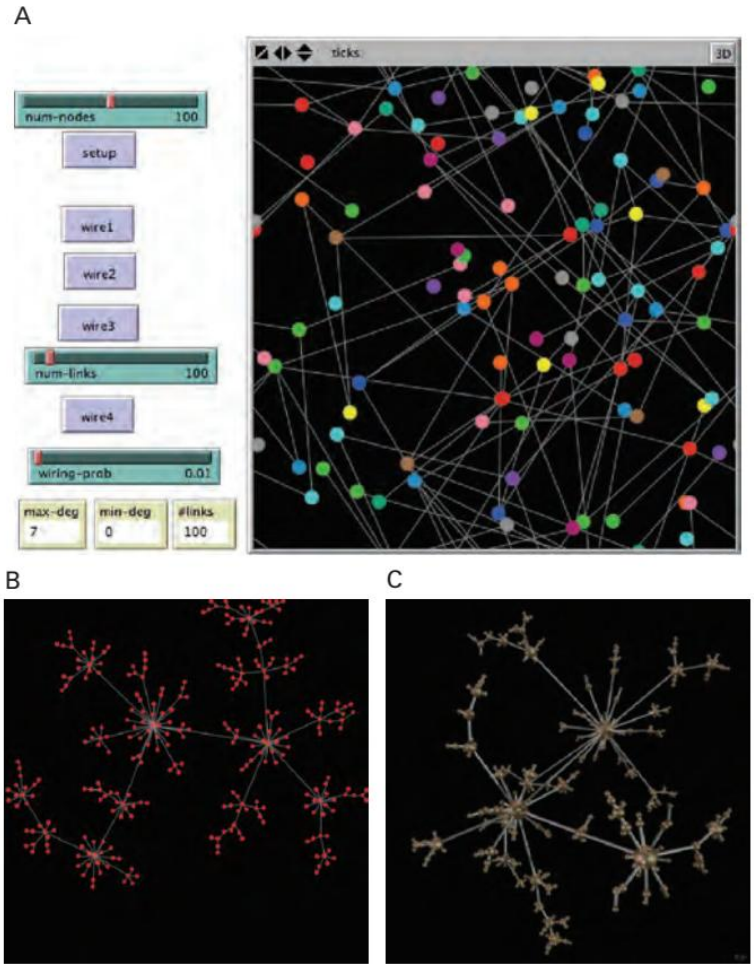


Figure 5.15
(A) Random Network model showing classic Erdös-Rényi random network. (B, C) Scale-Free Network from Preferential Attachment model. http://ccl.northwestern.edu/netlogo/models/PreferentialAttachment (Wilensky, 2001).

Small-world Network

The final standard network topology is known as a small-world network. Small-world networks are made up of dense clusters of highly interconnected nodes joined by a few long distance links between them. Due to these long distance links, it does not take many links for information to travel between any two random nodes in the network. Small-world networks are sometimes created by starting with regular networks, like the 2D lattices described before, then randomly rewiring some of the connections to create large jumps between occasional agents.

Ways to Characterize Networks

There are many ways to characterize networks. Two commonly used ways are

- Average Path Length
- Clustering Coefficient

Average Path Length

The average path length is the average of all the pairwise distances between nodes in a network. In other words, we measure the distance between every pair of nodes in the network and then average the results. Average path length characterizes how far the nodes are from each other in a network.

Clustering Coefficient of a Network

The clustering coefficient of a network is the average fraction of a node ' s immediate neighbors who are also neighbors of the node ' s other neighbors. In other words, it is a measure of the fraction of my friends who are also friends with each other. In networks with a high average clustering coefficient, any two neighboring nodes tend to share many of their neighbors in common, while in networks with a low average clustering coefficient there is generally little overlap between groups of surrounding neighbors.

NetLogo also includes a special extension, the network extension, for creating, analyzing, and working with networks.

Module No.133:

Special Environments

The two methods we have demonstrated for defining the environment, two-dimensional (2D) grid-based (i.e., lattice) and network-based, are both instances of " interaction topologies. " Interaction topologies describe the paths along which agents can communicate and interact in a model. Besides the two interaction topologies we have looked at so far, there are several other standard topologies to consider. Two of the most interesting topologies involve the use of 3D worlds and Geographic Information Systems (GIS). 3D worlds allow agents to move in a third dimension as well as the two dimensions in traditional ABMs. GIS formats enable the importation of layers of real-world geographical data into ABMs. We will discuss each of these in turn.

3D Worlds

3D environments enable model developers to explore complex systems which are irreducibly bound up with a third dimension as well as sometimes increasing the apparent physical realism to their models.

There is a version of NetLogo called NetLogo 3D (it is a separate application in the NetLogo folder) that allows modelers to explore ABMs in three dimensions. There are many implementations of classic ABMs that have been developed for this environment in the NetLogo 3D models library (Wilensky, 2000). For instance, there is a three-dimensional version of the Percolation model (see figure 5.16).

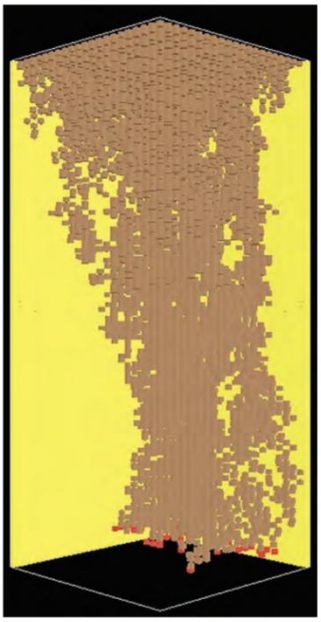


Figure 5.16
Percolation 3D. http://ccl.northwestern.edu/netlogo/models/Percolation3D (*Wilensky & Rand, 2006*).

Working With 3D Worlds

Working with 3D worlds is not much different from working with 2D worlds. Although, using a 3D world does require additional data and commands. For instance, agents now have a Z-coordinate, and new commands are needed to manipulate this new degree of freedom.

In 3D, the orientation of an agent can no longer be described by its heading alone, we must also use pitch and roll to describe the orientation of the agent. If you think about an agent as an airplane, then the pitch of the agent is how far from horizontal the nose of the airplane is pointing. For instance, if the nose of the airplane is pointing straight up then the pitch is 90 degrees (see figure 5.17).

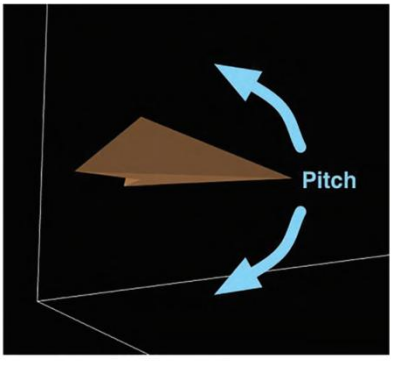


Figure 5.17
Pitch.

Using the airplane metaphor the roll of an agent is how far the wings are from horizontal. For instance, if the wings are pointing up and down then the roll of the agent is 90 degrees (see figure 5.18). In many 3D systems, heading has a new name, which is yaw, but in NetLogo we still use HEADING regardless of whether you are in 2D or 3D NetLogo to keep things consistent.

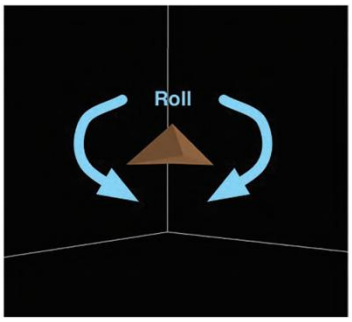


Figure 5.18
Roll.

To see the difference between writing 3D models and 2D models, let us examine the code in each model. The code for the PERCOLATE procedure in 2D Percolation looks

like this:

```
to percolate
    ask current-row with [pcolor = red] [
       ;; oil percolates to the two patches southwest and southeast
       ask patches at-points [[-1 -1] [1 -1]]
           [ if (pcolor = brown) and (random-float 100 < porosity)
              [ set pcolor red ] ]
       set pcolor black
       set total-oil total-oil + 1
    ]
    ;; advance to the next row
    set current-row patch-set [patch-at 0 -1] of current-row
end
```

Notice that in the 2D version, the code has each patch look at patches that are below,

left, and right of the patch. This will change in the 3D version. In 3D Percolation (Wilensky & Rand, 2006), PERCOLATE looks like this:

```
to percolate
    ask current-row with [pcolor = red] [
       ;; oil percolates to the four patches one row down in the z-coordinate and
    ;; southwest, southeast, northeast, and northwest
       ask patches at-points [[-1 -1 -1] [1 -1 -1] [1 1 -1] [-1 1 -1]]
           [ if (pcolor = black) and (random-float 100 < porosity)
              [ set pcolor red ] ]
       set pcolor brown
       set total-oil total-oil + 1
    ]
    ;; advance to the next row
    set current-row patch-set [patch-at 0 0 -1] current-row
end
```

The difference between 3D percolate and 2D

The only difference between the 3D percolate and its 2D counterpart is that instead of

percolating a line of patches at a time, we percolate a square of patches. As a result, each patch must ask four patches below it (in a cross shape) to percolate, not just two as it does in the 2D version.

GIS-Based

Geographic Information Systems (GIS) are environments that record large amounts of data that are related to physical locations in the world.

GISs are widely used by environmental scientists, urban planners, park managers, transportation engineers, and many others, and help to organize data and make decisions about any large area of land.

Using GIS, we can index all the information about a particular subject or phenomenon by its location in the physical world. Moreover, GIS researchers have developed analysis tools that enable them to quickly examine the patterns of this data and its spatial distribution.

GIS Tools And Techniques

As a result, GIS tools and techniques allow for a more in-depth exploration of the pattern of a complex system.

Agents moving on a GIS terrain may be constrained to interact on that terrain. Thus, GIS can serve as an interaction topology.

The 3D model, Flocking 3D

Where does ABM fit in to this picture? GIS can provide an environment for an ABM to operate in. Since ABM encompasses a rich model of process of a complex system, it is a natural match for GIS, which has a rich model of pattern. By allowing ABM to examine and manipulate GIS data, we can build models that have a richer description of the complex system we are examining. GIS thus enables modelers to construct more realistic and elaborate models of complex phenomena.

Module No.134:

Interactions

Now that we have discussed both agents and the environments in which they exist, we will look at how agents and environments interact.

There are five basic classes of interactions that exist in ABMs:

- agent-self
- environment-self
- agent-agent
- environment-environment
- agent-environment

We will discuss each in turn along with some examples of these common interactions.

Agent-Self Interactions Agents do not always need to interact with other agents or the environment. In fact, a lot of agent interaction is done within the agent. For instance, most of the examples of advanced cognition that we discussed in the Agent section involve the agent interacting with itself.

The agent considers its current state and decides what to do. One classic type of self-agent interaction is birth.

Birth **events are a typical event in ABMs**

In these, one agent creates another agent. Below is the birth routine:

```
;; check to see if this agent has enough energy to reproduce
to reproduce
    if energy > 200 [
       set energy energy - 100  ;; reproduction costs energy to the parent
       hatch 1 [ set energy 100 ] ;; which is transferred to the offspring
   ]
end
```

As you can see, the agent considers its own state and, based on this state, decides whether or not to give birth to a new agent. It then manipulates its state, lowering its energy and creating the new agent. This is the typical way of having agents reproduce.

Environment-Self Interactions Environment-self interactions are when areas of the environment alter or change themselves. For instance, they could change their internal state variables as a result of calculations. The classic example of an environment-self interaction is when the grass regrows:

```
;; regrow the grass
to regrow-grass
    ask patches [
       set grass-amount grass-amount + grass-regrowth-rate
       if grass > 10 [
           set grass 10
                          ]
recolor-grass
    ]
end
```

Each patch is asked to examine its own state and increment the amount of grass it has, but if it has too much grass then it is set back to the maximum value it can contain.

Agent-Agent Interactions Interactions between two or more agents are usually the most important type of action within agent-based models. We saw a canonical example of agent-agent interactions in the Wolf Sheep Predation model when the wolves consume the sheep:

```
;; wolves eat sheep
to eat-sheep
    if any? sheep-here [ ;; if there are sheep here then eat one
       let target one-of sheep-here
       ask target [
           die
       ]
       ;; increase the energy by the parameter setting
       set energy energy + energy-gain-from-sheep
    ]
end
```

In this case, one agent is consuming another agent and taking its resources, whereby the wolf always eats the sheep. However, it is also possible to add competition or flight

to this model, where the wolf gets a chance of eating the sheep and the sheep gets a chance to flee. Competition is another example of agent-agent interaction.

Communication

A final example of agent-agent interaction is communication. Agents can share information about their own state as well as that of the world around them. This type of interaction allows agents to gain information to which they might not have direct access.

Environment-Environment Interactions: Interactions between different parts of the environment are probably the least commonly used type of interaction in agent-based models. However, there are some common uses of environment-environment interactions: one of these is diffusion. In the Ants model discussed in chapter 1, the ants place a pheromone in the environment, which is then diffused throughout the world via an environment-environment interaction. This interaction is contained in the following piece of code in the main GO procedure:

```
diffuse chemical (diffusion-rate / 100)
ask patches
    [ set chemical chemical * (100 - evaporation-rate) / 100
;; slowly evaporate chemical
       recolor-patch ]
```

Agent-Environment Interactions: Agent-environment interactions occur when the agent manipulates or examines part of the world in which it exists, or when the environment in some way alters or observes the agent. A common type of agent-environment interaction involves agents observing the environment. The Ants model demonstrates this kind of interaction when the ants examine the environment to look for food and sense pheromone:

```
to look-for-food ;; turtle procedure
    if food > 0
    [ set color orange + 1      ;; pick up food
        set food food - 1               ;; and reduce the food source
        rt 180                                          ;; and turn around
    stop ]
    ;; face in the direction where the chemical smell is strongest
    if (chemical >= 0.05) and (chemical < 2)
    [ uphill-chemical ]
end
```

In the Ants model, the patches contain food and chemical, so the first part of this code checks to see if there is any food in the current patch. If there is food, the ant then picks up the food and turns around back to the nest, and the procedure stops.

Otherwise, the ant checks to see if there is chemical, wherein it follows the chemical in that direction.

Agent Movement

Another common type of agent-environment interaction is agent movement. In some ways, movement is simply an agent-self interaction, since it only alters the current agent's state.

In the Ants model the ants move around by " wiggling ".

Module No.135:

Observer/User Interface

Now that we have talked about the agents, the environments, and the interactions that occur between agents and environmental attributes, we may discuss who controls the running of the model.

The observer is a high-level agent that is responsible for ensuring that the model runs and proceeds according to the steps developed by the model author.

The observer issues commands to agents and the environment, telling them to manipulate their data or to take certain actions. Most of the control that model developers have with an ABM is mediated through the observer. However, the observer is a special agent. It does not have many properties, though it can access global properties like any agent or patch can.

Properties Specific to the observer

The only properties that one could consider to be specific to the observer are those relating to the perspective from which the modeled world is viewed. For instance, in NetLogo the view may be centered on a specific agent, or focusing a highlight on a certain agent, using the FOLLOW, WATCH, or RIDE commands (See figure 5.21).

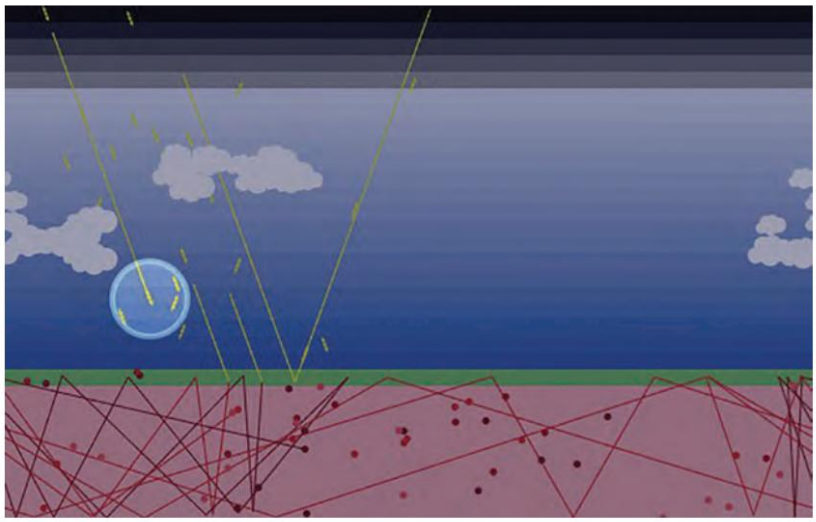


Figure 5.21
Climate Change model (with a sun ray turtle being watched, which is shown by the transparent "halo" around it). http://ccl.northwestern.edu/netlogo/models/ClimateChange (*Tinker & Wilensky, 2007*).

User Input and Model Output

We have already made use of many of the standard ways
to interface with an ABM when we extended models in chapter 3 and when we built our first model in chapter 4, but it is worth recapping some of them herein. ABMs require a control interface or a parameter group that allows the user to setup different parameters and settings for the ABM. The most common control mechanism is a button, which executes one or more commands within the model; if it is a forever button it will continue to execute those commands until the user presses the button again.

Command Center

A second way that the user can request that actions be performed within the ABM is via the command center (along with the mini – command centers within agent monitors). The command center is a very useful feature of NetLogo, as it allows the user to interactively test out commands, and manipulate agents and the environment.

Sliders, Switches & Input Boxes

Sliders enable the model user to select a particular value from a range of numerical values. For instance, a slider could range from 0 to 50 (by increments of 0,1) or 1 to 1,000 by increments of 1. In the Code tab the value of a slider is accessed as if it were a global variable. Switches enable the user to turn various elements of a model off or on. In the Code tab, they are also accessed as global variables, but they are Boolean variables. Choosers enable a model user to select a choice from a predefined drop-down menu that the modeler has created. Again these are accessed as global variables in the Code tab, but they are variables that have strings as their values, these strings being the various choices in the chooser. Finally, input boxes are more free-form, allowing the user to input text that the model can use.

Monitors , Plots & Notes

As for output controls, monitors display the value of a global variable or calculation updated several times a second. They have no history but show the user the current state of the system. Plots provide traditional 2D graphs enabling the user to observe the change of an output variable over time. Output boxes enable the modeler to create freeform text-based output to send to the user. Finally, notes enable the modeler to place text information on the Interface tab (for example, to give a model user directions on how to use the model). Unlike in monitors, the text in notes is unchanging (unless you manually edit them).

Visualization

Visualization is the part of model design concerned with how to present the data contained in the model in a visual way. Creating cognitively efficient and aesthetic visualizations can make it much easier for model authors and users to understand the

model. Though there is a long history of work on how to present data in static images

(Bertin, 1967; Tufte, 1983, 1996), there is much less work on how to represent data in

real-time dynamic situations. However, attempts have been made to take current static

guidelines and apply them to dynamic visualizations (Kornhauser, Rand & Wilensky,

2007). In general, there are three guidelines that should be kept in mind whenever designing the visualization of an ABM: simplify, explain, and emphasize.

Guidelines for Designing Visualization

Simplify the visualization Make the visualization as simple as possible so that anything that does not present additional usable information (or that is irrelevant to the current point being explained) has been eliminated from the visualization. This prevents the model user from being distracted by unnecessary " graph clutter " (Tufte, 1983).

Explain the components If there is an aspect of the visualization that is not immediately obvious then there should be some quick way to determine what that visualization is illustrating, such as a legend or description. Without clear and direct descriptions of what is going on the model user may misinterpret what the model author is attempting to portray. If a model is to be useful it is necessary that anyone viewing the model can easily understand what it is saying.

Emphasize the main point Model visualizations are themselves simplifications of all the possible data that a model could present to the model user. Therefore, a model visualization should emphasize the main points and interactions that the model author wants to explore, and, in turn, to communicate to end users. By exaggerating certain aspects of the visualization they can draw attention to these key results.

NetLogo Ethnocentrism model

Every NetLogo agent has a shape and color, and by selecting appropriate shapes and colors we can highlight some agents while backgrounding others. For instance, to simplify visualization if agents in our model are truly homogenous, like the ants, then we might make all the agents the same color, as in the Ants model. To explain the visualization we might change their shape to indicate something about their properties. For instance, in the Ethnocentrism model based on Hammond and Axelrod ' s model (2003) agents employing the same strategies have the same shape, even if they are different colors.

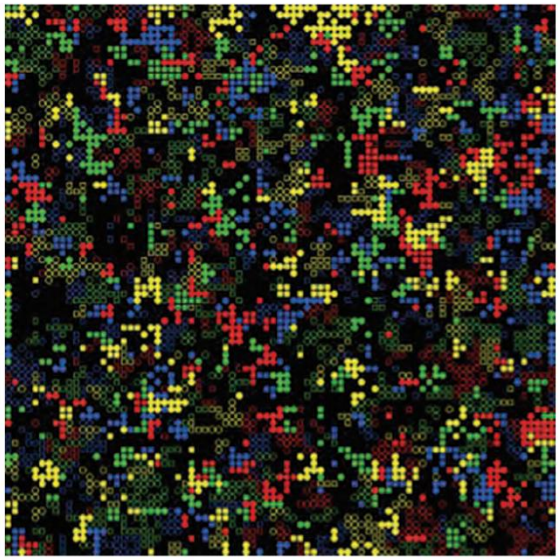


Figure 5.23
NetLogo Ethnocentrism model. This model uses the shape of agents to visualize agent strategy. http://ccl .northwestern.edu/netlogo/models/ethnocentrism (*Wilensky, 2003*).

Batch vs. Interactive

When you open a blank NetLogo model, it starts with a command center that says " Observer. " This blank window allows you to manipulate NetLogo in an interactive manner. If you open a model, you still have to press SETUP and GO to make the model work. Moreover, with most models, even when they are running, you can manipulate sliders and settings to see how these new parameters affect the performance of a model even during the middle of a run. This kind of spur-of-the-moment control is called interactive running, because the user can interact as the model is running.

In contrast to interactive running, another type of user running of a model is called batch running. With batch running, instead of controlling the model directly, the user writes a script to run the model many times, usually with different seeds for the pseudo-random number generator and with different parameter sets.

Module No.136:

Schedule
The schedule is a description of the order in which the model operates. Different ABM toolkits can have more or less explicit representations of the schedule. In NetLogo, there is no single identifiable object that can be identified as " the schedule. "

Rather, the schedule is the order of events that occur within the model, which depends on the sequence of buttons that the user pushes, and the code/procedures that those buttons run. We will first discuss the common SETUP/GO idiom which is employed in almost all agent-based models, and then move on to discuss some of the subtler issues concerning scheduling in ABMs.

SETUP and GO First of all, there is usually an initialization procedure that creates the agents, initializes the environment, and readies the user interface. In NetLogo this procedure is usually called SETUP and it executes whenever a user presses the SETUP button on a NetLogo model. The SETUP routine usually starts by clearing away all the agents and data related to the pervious run of the model. Then it examines how the user has manipulated the various variables controlled by the user interface creating new agents and data to reflect the new run of the model. For instance, in Traffic Basic the SETUP procedure looks like this:

```
to setup
    clear-all
    ask patches [ setup-road ]
    setup-cars
    watch sample-car
end
```

The other main part of the schedule is what is often called the main loop, or in NetLogo, the GO procedure. The GO procedure describes what happens in one time unit (or tick) of the model. Usually that involves the agents being told what to do, the environment changing if necessary, and the user interface updating to reflect what has happened. In Traffic Basic the GO procedure looks like this:

```
to go
;; if there is a car right ahead of you, slow down to a speed below its speed
    ask turtles [
        let car-ahead one-of turtles-on patch-ahead 1
        ifelse car-ahead != nobody
            [   slow-down-car car-ahead]
            ;; otherwise, speed up
            [ speed-up-car ]
    ;; don't slow down below speed minimum or speed up beyond speed limit
        if speed < speed-min [ set speed speed-min ]
        if speed > speed-limit [ set speed speed-limit ]
        fd speed ]
    tick
end
```

Asynchronous vs. Synchronous Updates

If a model uses an Asynchronous update schedule, this means that when agents change their state, that state is immediately seen by other agents.

In a Synchronous update schedule changes made to an agent are not seen by other agents until the next clock tick — that is, all agents update simultaneously.

Sequential vs. Parallel Actions

Within the realm of asynchronous updating, agents can
act either sequentially or in parallel.

Sequential actions involve only one agent acting at a time while Parallel actions are those in which all agents act independently. In NetLogo (versions 4.0 and later), sequential action is the standard behavior for agents.

Moreover, for the agents to act truly in parallel, you would need parallel hardware so that the actions of each agent would be carried out by a separate processor.

However, there is an intermediate solution. Simulated concurrency uses one processor to simulate many agents acting in parallel.

Module No.137:

Schedule
The schedule is a description of the order in which the model operates. Different ABM toolkits can have more or less explicit representations of the schedule. In NetLogo, there is no single identifiable object that can be identified as " the schedule. " Rather, the schedule is the order of events that occur within the model, which depends on the sequence of buttons that the user pushes, and the code/procedures that those buttons run. We will first discuss the common SETUP/GO idiom which is employed in almost all agent-based models, and then move on to discuss some of the subtler issues concerning scheduling in ABMs.

SETUP and GO First of all, there is usually an initialization procedure that creates the agents, initializes the environment, and readies the user interface. In NetLogo this procedure is usually called SETUP and it executes whenever a user presses the SETUP button on a NetLogo model. The SETUP routine usually starts by clearing away all the agents and data related to the pervious run of the model. Then it examines how the user has manipulated the various variables controlled by the user interface creating new agents and data to reflect the new run of the model. For instance, in Traffic Basic the SETUP procedure looks like this:

```
to setup
    clear-all
    ask patches [ setup-road ]
    setup-cars
    watch sample-car
end
```

The other main part of the schedule is what is often called the main loop, or in NetLogo, the GO procedure. The GO procedure describes what happens in one time

unit (or tick) of the model. Usually that involves the agents being told what to do, the environment changing if necessary, and the user interface updating to reflect what has happened. In Traffic Basic the GO procedure looks like this:

```
to go
;; if there is a car right ahead of you, slow down to a speed below its speed
    ask turtles [
         let car-ahead one-of turtles-on patch-ahead 1
         ifelse car-ahead != nobody
             [   slow-down-car car-ahead]
             ;; otherwise, speed up
             [ speed-up-car ]
    ;; don't slow down below speed minimum or speed up beyond speed limit
         if speed < speed-min [ set speed speed-min ]
         if speed > speed-limit [ set speed speed-limit ]
         fd speed ]
    tick
end
```

Asynchronous vs. Synchronous Updates

If a model uses an Asynchronous update schedule, this means that when agents change their state, that state is immediately seen by other agents.

In a Synchronous update schedule changes made to an agent are not seen by other agents until the next clock tick — that is, all agents update simultaneously.

Sequential vs. Parallel Actions

Within the realm of asynchronous updating, agents can
act either sequentially or in parallel.

Sequential actions involve only one agent acting at a time while Parallel actions are those in which all agents act independently. In NetLogo (versions 4.0 and later), sequential action is the standard behavior for agents.

Moreover, for the agents to act truly in parallel, you would need parallel hardware so that the actions of each agent would be carried out by a separate processor.

However, there is an intermediate solution. Simulated concurrency uses one processor to simulate many agents acting in parallel.

Module No.138:

Here, we will understand about the different types of measurements in an ABM. Statistical Analysis of ABM:

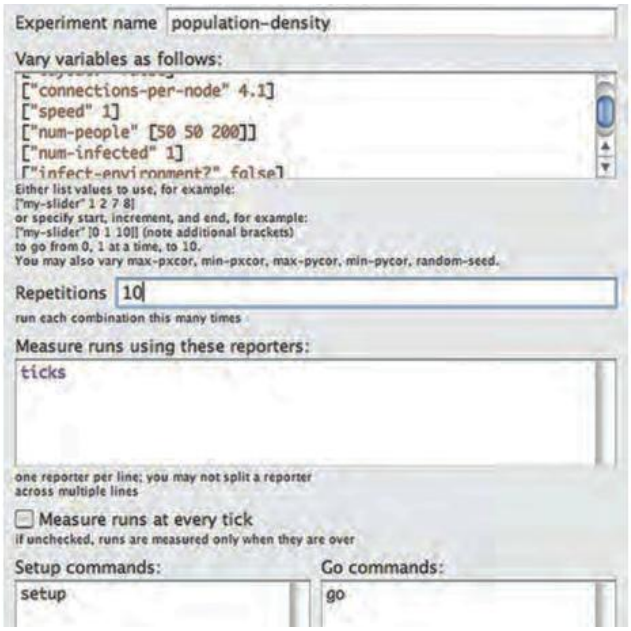
- Moving beyond Raw Data
- Statistical results are the most common way of looking at any kind of scientific data
- The general methodology behind descriptive statistics is to provide numerical measures These measures summarize a large data set

- They also describe the data set in such a way that it is not necessary to examine every single value.

Summary Statistics

Population	Mean	Std. Dev.
50	366.8	47.39385802
100	213.8	27.40154091
150	144.4	17.65219533
200	118.7	12.12939497

-



-

BehaviorSpace Data Imported into a Spreadsheet

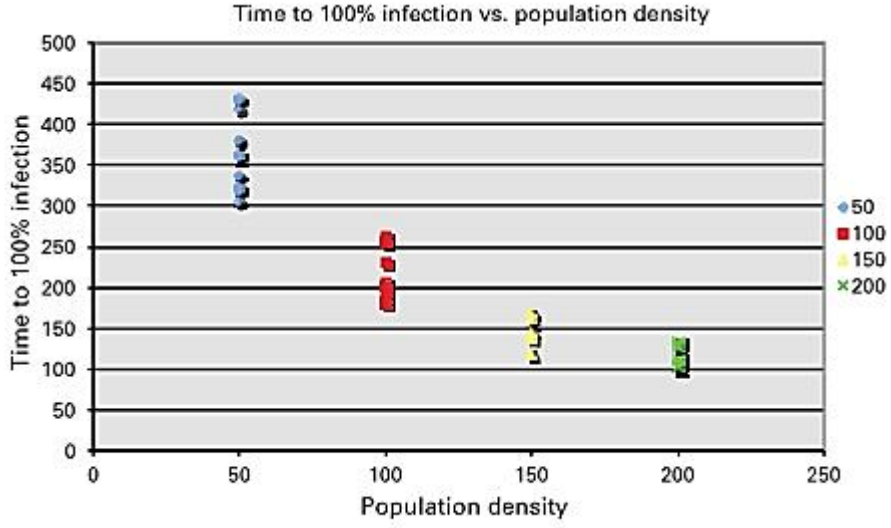
BehaviorSpace Table data

population-density

DATE
TIME

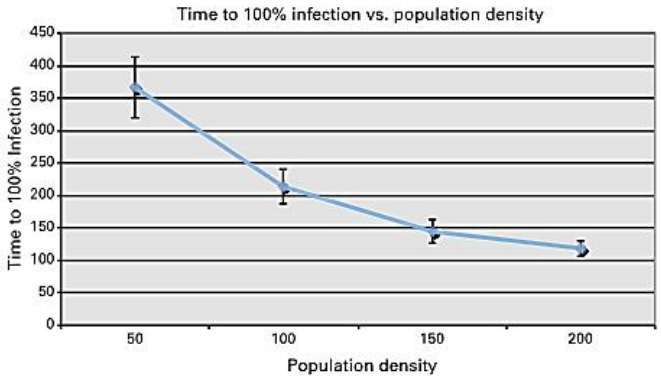
[run number]	network?	layout?	connections-per-node	speed	num-people	num-infected	infect-environment?	[tick]	ticks
1	FALSE	FALSE	4.1	1	50	1	FALSE	299	299
2	FALSE	FALSE	4.1	1	50	1	FALSE	432	432
3	FALSE	FALSE	4.1	1	50	1	FALSE	444	444
4	FALSE	FALSE	4.1	1	50	1	FALSE	400	400
5	FALSE	FALSE	4.1	1	50	1	FALSE	467	467
6	FALSE	FALSE	4.1	1	50	1	FALSE	397	397
7	FALSE	FALSE	4.1	1	50	1	FALSE	337	337

-



-

•



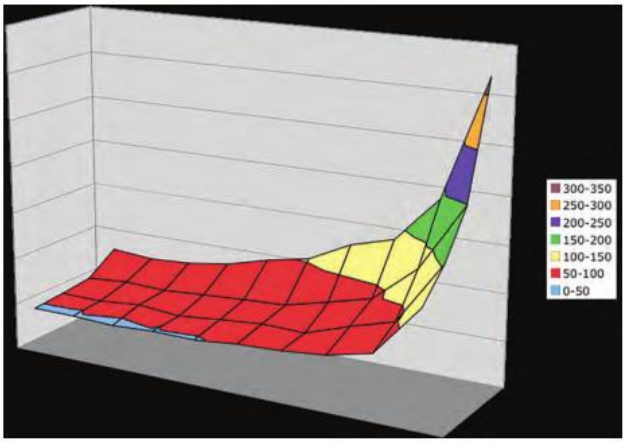


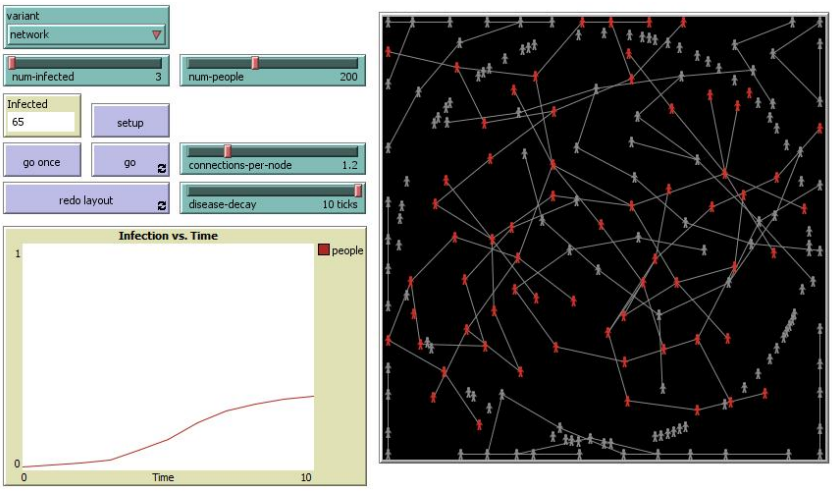
Figure 6.9
3D Chart of NUM-PEOPLE and DISEASE-DECAY versus time to 100 percent infection.

- We have learnt about the types of measurements in the analysis of an ABM.
- Credits: Uri Wilensky book.

Module No.139:

Here we will Model the spread of disease in detail.

- If a cold virus infects someone, that person might spread that disease to five other people (six now infected) before they recover.
- In fact, the rate of infection initially rises exponentially.
- Suppose that we are interested in understanding the spread of disease, and we want to build an ABM of such a spread. How should we go about doing it?
- First, we need some agents that keep track of whether they are infected with a cold or not
- We need the ability to initialize the model by infecting a group of individuals
- Individuals move around randomly on a landscape and infect other individuals whenever they come into contact with them.



- We conclude that as the population density increases, the time to full infection dramatically decreases
- In the beginning, when the first person becomes infected, if there are not many other people around, the person has no one to infect, and thus the infection rate increases slowly.
- However, if there are many people around then there will be plenty of infection opportunities
- Moreover, at the end of the run, when there are only one or two uninfected agents, they will be more likely to run into someone with an infection if the population count is high.
- This is true despite the fact that the total number of people that need to be infected increases.
- To describe these patterns of behavior it makes sense to turn to some statistics
- Credits : Uri Wilensky

Module No.140:

Here we will learn about the general methodology behind descriptive statistics that is to provide numerical measures that summarize a large data set and describe the data set in such a way that it is not necessary to examine every single value.

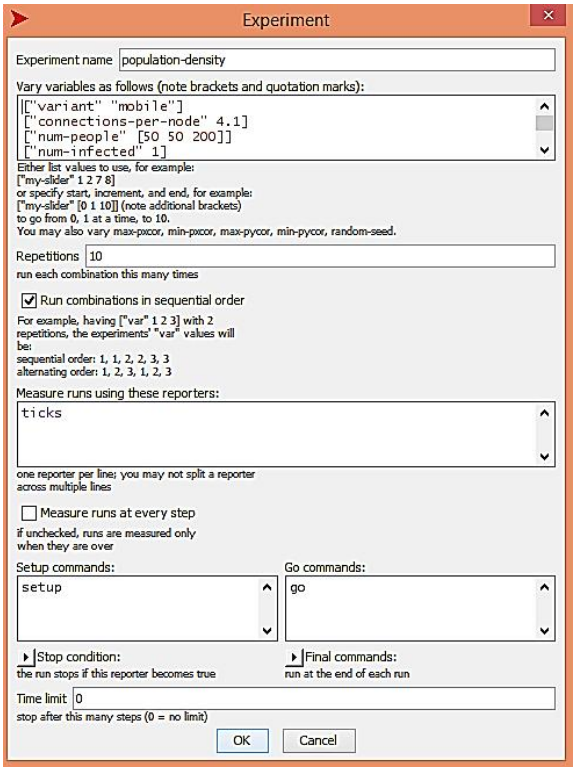
- Suppose we are interested in determining whether a coin is fair. (i.e., it is as likely to turn up heads when flipped as it is to turn up tails) then we can conduct a series of experiments where we flip the coin and observe the results.
- It is much easier to look at means and standard deviations than it is to examine large series of data
- (e.g., for HHHHTTHTTT, the observed probability is 0.5, and the expected outcome for ten trials is to observe five heads with a standard deviation of 1.58).
- To apply this technique to our Spread of Disease model in more depth, we can create summary statistics

Summary Statistics		
Population	Mean	Std. Dev.
50	366.8	47.39385802
100	213.8	27.40154091
150	144.4	17.65219533
200	118.7	12.12939497

- We see that the mean time to 100 percent infection declines as the population density increases.
- Another interesting result is that as the population density goes up, the standard deviation goes down.
- This means that the data is less varied.
- These results seem to confirm our original hypothesis that as population density increases the mean time to infection declines.
- Within ABM, statistical analysis is a common method of confirming or rejecting hypotheses
- ABMs create large amounts of data (the Spread of Disease model is just a small example), and if we can summarize that data we can examine large amounts of output in an efficient manner.
- Most ABM toolkits give you the basic ability to carry out simple statistical analysis within the package itself
- (e.g., in NetLogo there are MEAN and STANDARD-DEVIATION primitives). Thus, while the model is running, the ABM itself can generate summary statistics.
- The NetLogo R extension can be used to conduct analyses with the R statistical package
- Credits : Uri Wilensky

Module No.141:

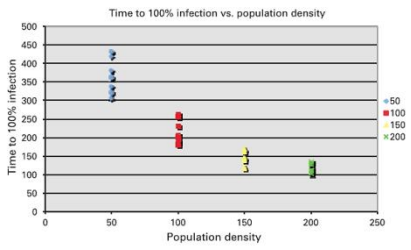
- When you are trying to collect statistical results from an ABM you should run the model multiple times and collect different results at different points.
- Here we will understand how ABM toolkits will provide you with a way to collect the data from these runs automatically.
- In NetLogo there is a tool called BehaviorSpace. ABM toolkits are often full-featured programming languages, allowing you to write your own tools for creating experiments to produce the data sets you want to analyze.
- These tools will automatically run the model multiple times with multiple different settings and collect the results in some easy to use format like the CSV files mentioned earlier.
- Let us call this experiment “ population density”



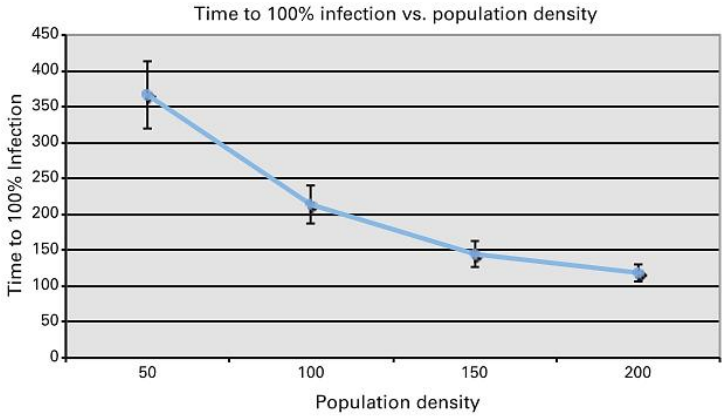
- The SETUP and GO commands allow you to specify any additional NetLogo code that you need to make the model start and go.
- “ Stop condition ” allows you to specify special stop conditions for each run, and
- “ Final commands ” allows you to insert any commands that you want executed between runs of the model
- In general in ABM, it is important to carry out multiple runs of your experiments so that you can determine if some result is truly a pattern or just a one-time occurrence.
- One common way is to start by manually running your model multiple times, but to get a better sense of the results it is usually much easier to use a batch experiment tool.
- We have illustrated the BehaviorSpace tool, which is the batch experiment tool for NetLogo.
- Credits : Uri Wilensky

Module No.142:

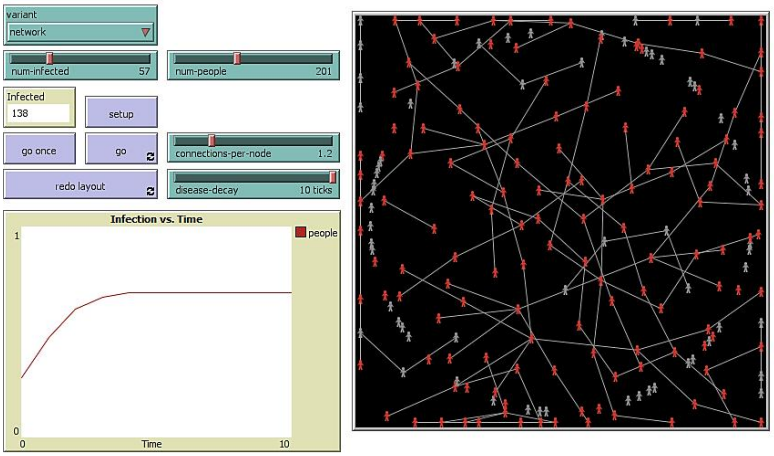
In NetLogo creating simple graphs is easy to do and will often suffice for simple data analysis. But with complex data sets, designing a useful and immediately informative graph can be challenging and is the subject of an extensive body of literature.



- We can quickly see how the data is distributed and how the data changes with population density



-
- This new figure might not be easier to understand than previous figure, but if there were one hundred data points in previous figure rather than ten data points, then a figure like this one might be very helpful.
- Many ABM toolkits include capabilities for continually updating graphs and charts during the running of a model and thus enable you to see the progress of the model temporally
- For instance, in the Spread of Disease model there is a graph that illustrates the change in the fraction of infected agents as time proceeds

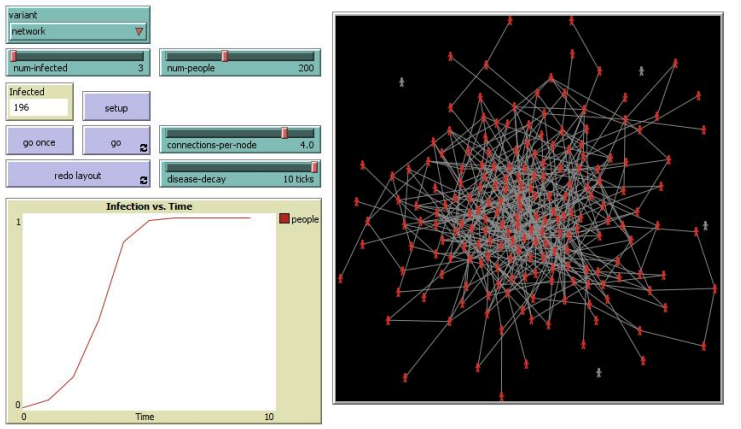


-
- This is one example of how we can use time series to help understand the behavior of a model.
- Summary: We have seen the use of graphs (plots) in simulation software
- Credits: Uri Wilensky

Module No.143:

Here, we understand the analysis of “networks within ABM.”

- Interactions do not occur in physical space. But rather they can also occur across social networks.
- E.g. some diseases spread only through certain kinds of social networks. If we set the chooser to “network” we can explore this further.
- The property which determines how many connections.
- Each individual has to other individual in the network.

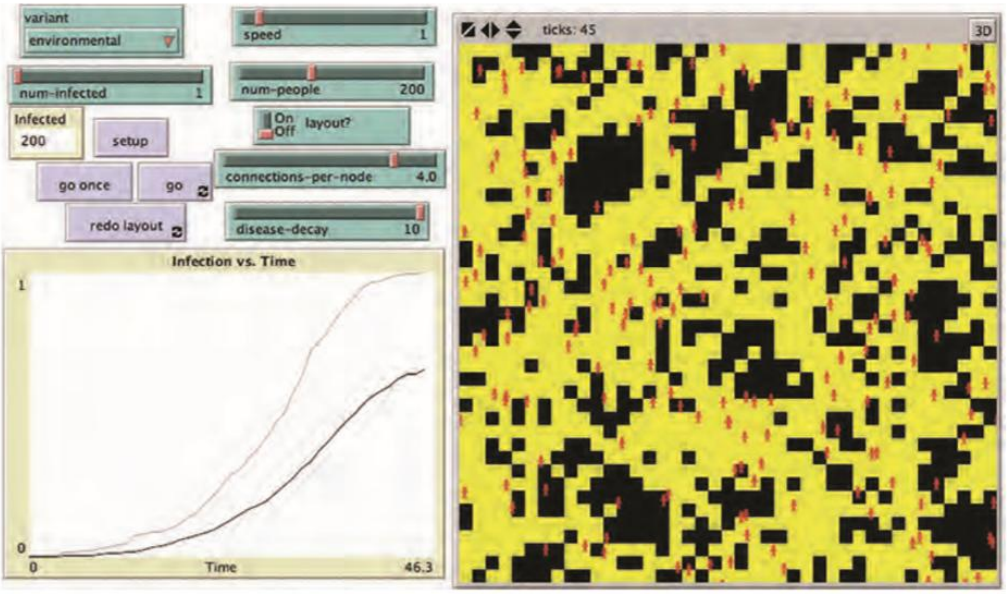
- Known as “connections-per-node”.
- 

- A well known property of random graph is the average number of individuals infected.
- Grows substantially
- The connections-per-node exceeds 1.0
- Forms a giant component in the network.
- The property of average path length, which measure the distance between any two nodes in the network.
- Affect the spread of disease in the network.
- Another property is clustering co-efficient.
- Different properties and tools are there to measure and analyze a wide variety of metrices
- Associated with social network analysis (SNA).
- To summarize, each of these network properties can be analyzed as to their effect on the spread of disease.
- Reports and toolkits like UCInet can be used for further examination.
- Credits: Uri Wilensky book.

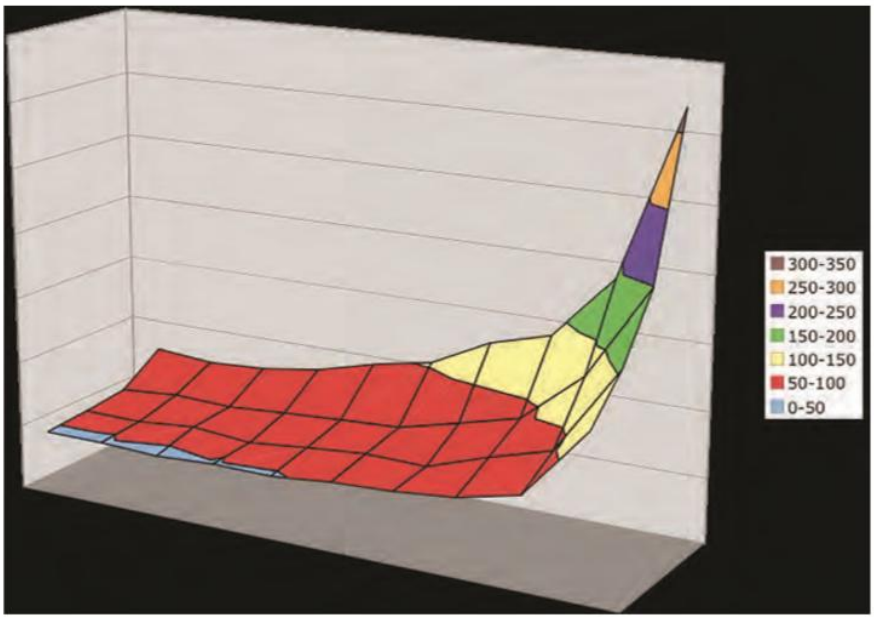
Module No.144:

In this section we are going to see agent-to-environment and environment-to-agent transmission.

- The key areas to understand are:
    - Spread of disease model for examining.
    - Environmental interaction effect.



- The path below any agent will become yellow.
- Change the rate if DISEASE-DECAY 0 to 10 at a single time intervals
- A long DIESEASE-DECAY might have negligible over having a small DISEASE-DECAY.
- Investigating using behaviour space, we notice that a powerful aspect of ABMs is that it also shows us the pattern of infection.



- Leaving long stringy patterns of environmental infection.
- To summarize, we have seen the working of environmental data and ABMs.
- Credits: Uri Wilensky book

Module No.145:

- Here, we understand the about the “correctness of a model”. Output relating the concerned issue must be accurate.
- Model accuracy is evaluated through validation, verification and replication.

- Implemented model corresponds to, and explains, some phenomenon in the real world.
- Model verification: determining whether an implemented model corresponds to the target conceptual model.
- Make sure that the model has been implemented correctly.
- Model replication is the implementation by one researcher or group of researchers of a conceptual model previously implemented by someone else.
- Set of results from a model that corresponds to the real world is not sufficient.
- Multiple runs are often needed to confirm that a model is accurate.
- Verification, validation, and replication collectively underpin the correctness, and thus utility, of a model.
- Credits: Uri Wilensky book.

Module No.146:

- Here, we understand the role of "Verification" in Correctness of a Model.
- In larger models, the code can be difficult to understand as it evolves over time.
- Verification ensures the elimination of "bugs" from the code.
- The process of debugging becomes more difficult for complex models.
- Our goal, however, is to keep the process easy and simple.
- Build the model simply to begin with – it will be easier to verify. Expand the complexity of the model as necessary. This incremental approach makes it easy to verify the additional components.
- Even if all the components are verified, it is still possible that the system is not. Complications may arise from the interaction between model components.
- In this section we have examined the issue of verification of a model in the context of a simple ABM.
- Credits: Uri Wilensky book

Module No.147:

Here, we understand need of "communication" for the correctness of a model.

- Sometimes a team of people builds a model. Other team members actually implement the model. Therefore, verification becomes critical.
- Communication is critical to ensure that the implemented model is correct.
- It is essential.
- E.g. In the voting model.
- Political scientists differentiate between Moore and Van Neumann neighborhoods.
- Small world network and hexagonal versus a rectangular grid.
- In an ideal situation, the model author and the implementer is the same person which averts the sort of communication errors.
- But when it is not, then there is often room for human error and misunderstanding.
- In the past it was difficult to be expert model implementer and model author.
- However, low-threshold ABM languages, such as NetLogo is narrowing the gap between author and implementer.

- We have seen how communication plays an important role in the correctness of a model.
- How communication can fill the gap between the implementer and the author of the model.
- Credits: Uri Wilensky book.

Module No.148:

Here, we will understand how “Describing Conceptual Models” plays a role in analyzing correctness of a model.

- Implementing the voting model, we may realize subsequently that we never understand the idea completely. This could happen if we talked with a political scientist.
- Describing how we plan to implement the model is a specific type of document.
- We and the political scientist should have the same “conceptual model” in our mind.
- Describe the model in more formal terms. This includes a pictorial description of the model using flowcharts.
- Subsequently, we can convert the flowcharts into pseudo-code
- The goal of pseudo-code is to serve as a midway point between natural language and programming language.
- Pseudocode:

```
Voters have votes = {0, 1}

For each voter:
    Set vote either 0 or 1, chosen with equal probability
Loop until election
    For each voter
       If majority of neighbors' votes = 1 and vote = 0 then set vote 1
       Else If majority of neighbors' votes = 0 and vote = 1 then set vote 0
       If vote = 1: set color blue
       Else: color = green
    Display count of voters with vote = 1
    Display count of voters with vote = 0
End loop
```

- Other methods are UML, choosing a language similar to pseudo-code and NetLogo.
- We have learnt about processes involved in describing conceptual model which include flowcharts, pseudo-code, UML, and NetLogo.
- Credits: Uri Wilensky book

Module No.149:

Here, We will discuss and understand how we can use “Verification Testing” for finding out about the correctness of a model.

- When implementing the design into code we need to follow ABM core design principles.

```
patches-own
[
    vote ;; my vote (0 or 1)
    total ;; sum of votes around me
]

to setup
    clear-all
    ask patches [
           if (random 2 = 0) ;; half a chance of this
           [ set vote 1 ]
       ]
    ask patches [
       if (random 2 = 0) ;; half a chance of this
           [ set vote 0 ]
    ]
    ask patches [
       recolor-patch
    ]
end

to recolor-patch ;; patch procedure
    ifelse vote = 0
       [ set pcolor green ]
       [ set pcolor blue ]
end
```

```
to setup
    clear-all
    ask patches [
          set vote random 2    ;; 0 or 1, with equal probability
    ]
    ask patches [
       recolor-patch
    ]
    check-setup
end
```

```
to check-setup
    let diff abs ( count patches with [ vote = 0 ] - count patches
with [ vote = 1 ] )
    if diff > .1 * count patches [
       print "Warning: Difference in initial voters is greater than 10^4%."
    ]
end
```

- This verification technique is a form of unit testing. We can modify the code without disrupting previous code.

```
to go
    ask patches [
       set total (sum [vote] of neighbors)
    ]
    ;; this is equivalent to count neighbors with [vote = 1]
    ;; use two ask patches blocks so all patches compute "total"
    ;; before any patches change their votes
    ask patches [
       ifelse vote = 0 and total >= 4 [
           set vote 1
       ]
           [if vote = 1 and total <= 4 ] [
               [set vote 0
           ]
           recolor-patch
       ]
       tick
       end
```

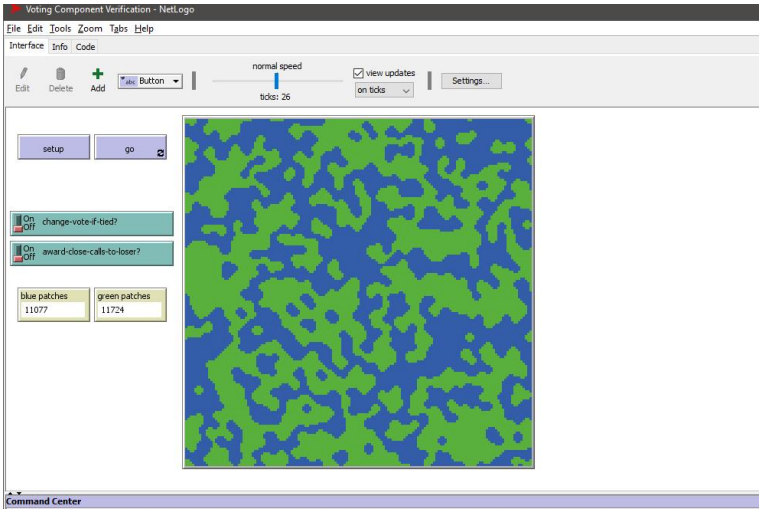
- We have understood how an incremental approach makes it easy to implement the model correctly.
- We are then able to write our code into NetLogo for verification purpose.
- Credits: Uri Wilensky book

Module No.150:

Here, we will learn about going beyond verification using agent-based modeling.

- Sometimes results are not produced to what the implementers and authors hypothesized.

- Results of modeling can therefore end up causing further confusions for the scientists.
- Jagged edges can occur due to ties in votes.
- Design the model that neighbors do not change their votes if tied.
- Switch called CHANGE VOTE IF TIED.



- Command Center

```
patches-own [
  vote   ;; my vote (0 or 1)
  total  ;; sum of votes around me
]

to setup
  clear-all
  ask patches [
    set vote random 2 ;; set vote to either 0 or 1
    recolor-patch
  ]
  reset-ticks
  check-setup
end

to go
  ;; keep track of whether any patch has changed their vote
  let any-votes-changed? false
  ask patches [
    set total (sum [ vote ] of neighbors)
  ]
  ;; use two ask patches blocks so all patches compute "total"
  ;; before any patches change their votes
  ask patches [
    let previous-vote vote
    if total < 3 [ set vote 0 ] ;; if majority of your neighbors vote 0, set your vote to 0
    if total = 3 [
      ifelse award-close-calls-to-loser?
        [ set vote 1 ]
        [ set vote 0 ]
    ]
    if total = 4 and change-vote-if-tied? [
      set vote (1 - vote) ;; invert the vote
    ]
    if total = 5 [
      ifelse award-close-calls-to-loser?
        [ set vote 0 ]
        [ set vote 1 ]
    ]
    if total > 5 [ set vote 1 ] ;; if majority of your neighbors vote 1, set your vote to 1
    if vote != previous-vote [ set any-votes-changed? true ]
    recolor-patch
  ]
  ;; if the votes have stabilized, we stop the simulation
  if not any-votes-changed? [ stop ]
  tick
end

to recolor-patch  ;; patch procedure
  ifelse vote = 0
    [ set pcolor green ]
    [ set pcolor blue ]
end

;; This procedure checks to see if the SETUP procedure sets up the model with roughly
;; equal numbers of blue and green patches
to check-setup
  ;; count the difference between the number of green and the number of blue patches
  let diff abs (count patches with [ vote = 0 ] - count patches with [ vote = 1 ])
  if diff > .1 * count patches [
    print "Warning: Difference in initial voters is greater than 10%."
  ]
end

; Copyright 2008 Uri Wilensky.
```

- Switch AWARD CLOSE CALLS TO LOSER?
- With both switches on, we have a different outcome.

```
to go
  ask patches
    [ set total (sum [vote] of neighbors) ]
  ;; use two ask patches blocks so all patches compute "total"
  ;; before any patches change their votes
  ask patches
    [ if total > 5 [ set vote 1 ]
      if total < 3 [ set vote 0 ]
      if total = 4
        [ if change-vote-if-tied?
          [ set vote (1 - vote) ] ] ;; switch vote
      if total = 5
        [ ifelse award-close-calls-to-loser?
          [ set vote 0 ]
          [ set vote 1 ] ]
      if total = 3
        [ ifelse award-close-calls-to-loser?
          [ set vote 1 ]
          [ set vote 0 ] ]
      recolor-patch ]
  tick
end
```

- We have seen the details of how to go beyond verification.
- Credits: Uri Wilensky book

Module No.151:

Here, we will discuss sensitivity analysis and robustness.

- Creating the parameter to test the hypothesis of initial balance in one direction.
- Using BehaviorSpace, we can run the experiment varying from 25 to 75 percent increment.
- Two conditions:
- If no voter switch vote in the last step the model will stop.
- The model will stop after one hundred times the steps have executed.

```
patches-own
[
  vote   ;; my vote (0 or 1)
  total  ;; sum of votes around me
]

to setup
  clear-all
  ask patches [
    ifelse random 100 < initial-green-pct
      [ set vote 0 ]
      [ set vote 1 ]
    recolor-patch
  ]
  reset-ticks
  check-setup
end

to go
  ;; keep track of whether any patch has changed their vote
  let any-votes-changed? false
  ask patches [
    set total (sum [ vote ] of neighbors)
  ]
  ;; use two ask patches blocks so all patches compute "total"
  ;; before any patches change their votes
```

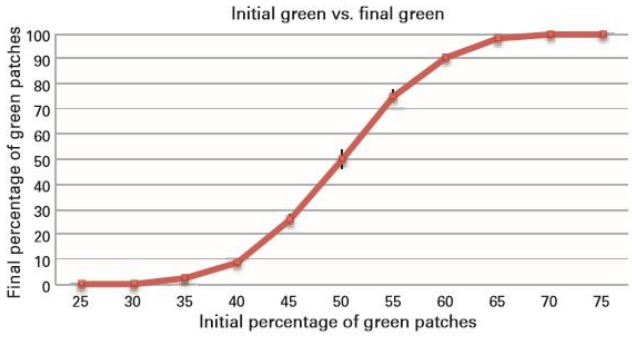
```
  ]
  ;; use two ask patches blocks so all patches compute "total"
  ;; before any patches change their votes
  ask patches [
    let previous-vote vote
    if total < 3 [ set vote 0 ] ;; if majority of your neighbors vote 0, set your vote to 0
    if total = 3 [
      ifelse award-close-calls-to-loser?
        [ set vote 1 ]
        [ set vote 0 ]
    ]
    if total = 4 and change-vote-if-tied? [
      set vote (1 - vote) ;; invert the vote
    ]
    if total = 5 [
      ifelse award-close-calls-to-loser?
        [ set vote 0 ]
        [ set vote 1 ]
    ]
    if total > 5 [ set vote 1 ] ;; if majority of your neighbors vote 1, set your vote to 1
    if vote != previous-vote [ set any-votes-changed? true ]
    recolor-patch
  ]
```

```
  ;; if the votes have stabilized, we stop the simulation
  if not any-votes-changed? [ stop ]
  tick
end

to recolor-patch  ;; patch procedure
  ifelse vote = 0
    [ set pcolor green ]
    [ set pcolor blue ]
end

;; This procedure checks to see if the SETUP procedure sets up the model with
;; roughly expected numbers, given the value of the initial-green-pct slider
to check-setup
  let expected-green (count patches * initial-green-pct / 100)
  let diff-green (count patches with [ vote = 0 ]) - expected-green
  if diff-green > (.1 * expected-green) [
    print "Initial number of green voters is more than expected."
  ]
  if diff-green < (- .1 * expected-green) [
    print "Initial number of green voters is less than expected."
  ]
end


; Copyright 2008 Uri Wilensky.
; See Info tab for full copyright and license.
```



- Past research methodologies for sensitivity analysis include Active Nonlinear Testing.
- We have seen the details of Sensitivity Analysis and Robustness
- Credits : Uri Wilensky book

Module No.152:

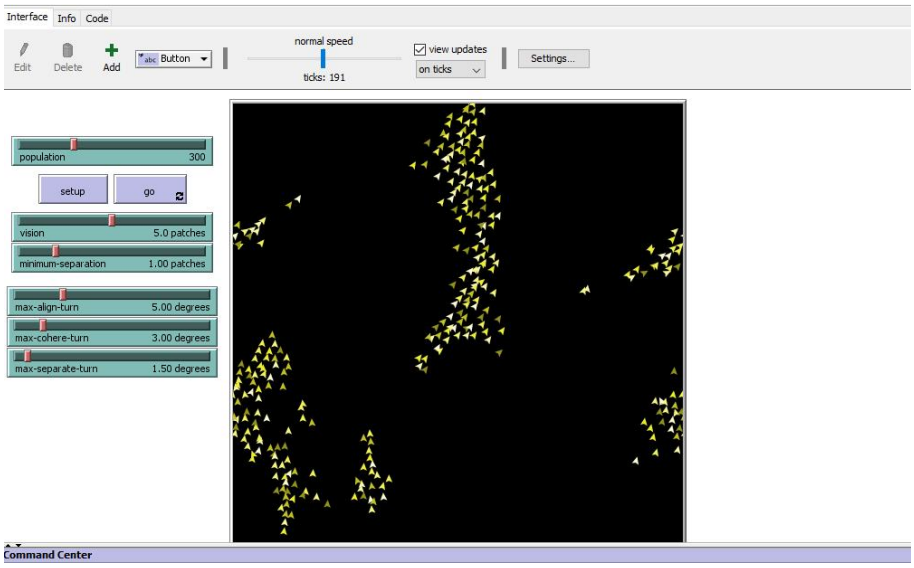
Here, we are going to examines benefits and issues related to verification.

- We need to develop an understanding the cause of unexpected outcomes and the impact of small changes.
- Implemented model needs to correspond well with the conceptual model.
- This all depends on the model's low level rules as well as an understanding of the mechanisms involved. A bug in the code can produce surprising results.
- Understanding the operation of the model is therefore quite essential. It is also important to note that the verification process is not binary.
- We have discussed verification benefits and issues.
- Credits: Uri Wilensky book

Module No.153:

We will discuss the validation of agent based modeling.

- Validation involves corresponding between implemented model and reality.
- The various topics related to validation include:
- Two axis.
- Macrovalidation.
- Face validation.
- Flocking model.
- A classical agent based model.




```
turtles-own [
  flockmates         ;; agentset of nearby turtles
  nearest-neighbor   ;; closest one of our flockmates
]

to setup
  clear-all
  create-turtles population
    [ set color yellow - 2 + random 7  ;; random shades look nice
      set size 1.5  ;; easier to see
      setxy random-xcor random-ycor
      set flockmates no-turtles ]
  reset-ticks
end

to go
  ask turtles [ flock ]
  ;; the following line is used to make the turtles
  ;; animate more smoothly.
  repeat 5 [ ask turtles [ fd 0.2 ] display ]
  ;; for greater efficiency, at the expense of smooth
  ;; animation, substitute the following line instead:
  ;;   ask turtles [ fd 1 ]
  tick
end
```

```
to flock  ;; turtle procedure
  find-flockmates
  if any? flockmates
    [ find-nearest-neighbor
      ifelse distance nearest-neighbor < minimum-separation
        [ separate ]
        [ align
          cohere ] ]
end

to find-flockmates  ;; turtle procedure
  set flockmates other turtles in-radius vision
end

to find-nearest-neighbor ;; turtle procedure
  set nearest-neighbor min-one-of flockmates [distance myself]
end

;;; SEPARATE

to separate  ;; turtle procedure
  turn-away ([heading] of nearest-neighbor) max-separate-turn
end

;;; ALIGN

to align  ;; turtle procedure
  turn-towards average-flockmate-heading max-align-turn
end
```

```
;;; ALIGN

to align  ;; turtle procedure
  turn-towards average-flockmate-heading max-align-turn
end

to-report average-flockmate-heading  ;; turtle procedure
  ;; We can't just average the heading variables here.
  ;; For example, the average of 1 and 359 should be 0,
  ;; not 180.  So we have to use trigonometry.
  let x-component sum [dx] of flockmates
  let y-component sum [dy] of flockmates
  ifelse x-component = 0 and y-component = 0
    [ report heading ]
    [ report atan x-component y-component ]
end

;;; COHERE

to cohere  ;; turtle procedure
  turn-towards average-heading-towards-flockmates max-cohere-turn
end

to-report average-heading-towards-flockmates  ;; turtle procedure
  ;; "towards myself" gives us the heading from the other turtle
  ;; to me, but we want the heading from me to the other turtle,
  ;; so we add 180
  let x-component mean [sin (towards myself + 180)] of flockmates
  let y-component mean [cos (towards myself + 180)] of flockmates
  ifelse x-component = 0 and y-component = 0
    [ report heading ]
    [ report atan x-component y-component ]
end
```

```
;;; HELPER PROCEDURES

to turn-towards [new-heading max-turn]  ;; turtle procedure
  turn-at-most (subtract-headings new-heading heading) max-turn
end

to turn-away [new-heading max-turn]  ;; turtle procedure
  turn-at-most (subtract-headings heading new-heading) max-turn
end

;; turn right by "turn" degrees (or left if "turn" is negative),
;; but never turn more than "max-turn" degrees
to turn-at-most [turn max-turn]  ;; turtle procedure
  ifelse abs turn > max-turn
    [ ifelse turn > 0
        [ rt max-turn ]
        [ lt max-turn ] ]
    [ rt turn ]
end


; Copyright 1998 Uri Wilensky.
; See Info tab for full copyright and license.
```

- Here, we have discussed the validation of agent based models.
- Credits: Uri Wilensky book.

Module No.154:

Here, we will understand the difference between “macrovalidation and microvalidation“ and examine their role in the validation of a model.

- ABMs are built from agents hence we can directly compare them with those that exist in the real world.
- For instance, we can ask whether the agents have properties similar to real birds in flocking model.
- There are some limitation e.g. real birds can fly in three dimensions but our bird agents move in two dimension only. However, these limitations does not make our model invalid.
- We can also build the flocking model in three dimensions and examine the resultant flocks to see if they are relevantly different from the 2D flocks.
- The other major avenue of validation is to investigate the relationship between the global properties of the model and the flocking patterns of real birds, a process called macrovalidation.
- By showing that our model corresponds to the macro-level phenomenon, we further validate that our model is descriptive of real world systems.
- Macrovalidation tells you if you have captured the important parts of the system, whereas microvalidation tells you if you ’ve captured the important parts of the agent’s individual behavior.

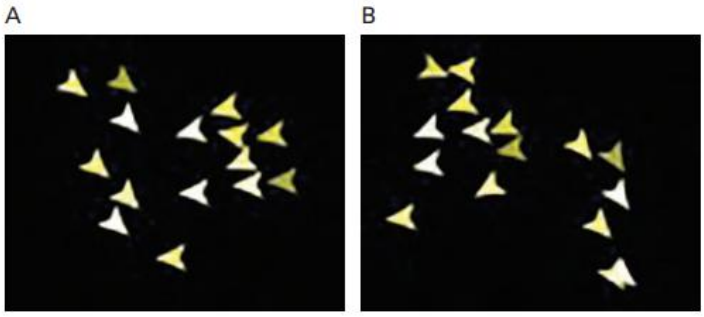


Figure 7.9
Two mini-flocks (A) before and (B) after a collision.

-
- We now understand how macrovalidation and microvalidation compares to each other.
- Credits: Uri Wilensky book.

Module No.155:

Here we will discuss how “face validation” is different from “empirical validation”.

- Face validation is the process of showing that the mechanisms and properties of the model look like mechanisms and properties of the real world.
- Empirical validation is making sure that the model generates data. It corresponds to similar patterns of data in the real world.
- Face validity can exist at both micro and macro levels.
- Determining whether the birds in the model correspond to real birds is face microvalidity.
- While determining if the flocks correspond to the appearance of real flocks is face macrovalidity.
- Empirical validation sets a higher bar. Data produced by the model must correspond to empirical data derived from the real-world.
- Measures and numerical data generated both by the model and the actual phenomenon.
- Inputs and outputs in “ the real world ” are often poorly defined. It is hard to isolate and measure parameters from a real-world phenomenon.
- The process of finding the parameters and initial conditions that cause the model to match with real-world data is called calibration.
- We are now able to understand how these four types of validation (micro-face, macro-face, micro-empirical, and macro-empirical) characterize the majority of validation efforts carried out.
- Credits: Uri Wilensky book

Module No.156:

Here we will understand why it is important to validate a model.

- A valid model can be useful for extracting general principles about the world.
- Changing the mechanisms and parameters can often help predict what might occur in the real world.
- A model is not either valid or invalid:

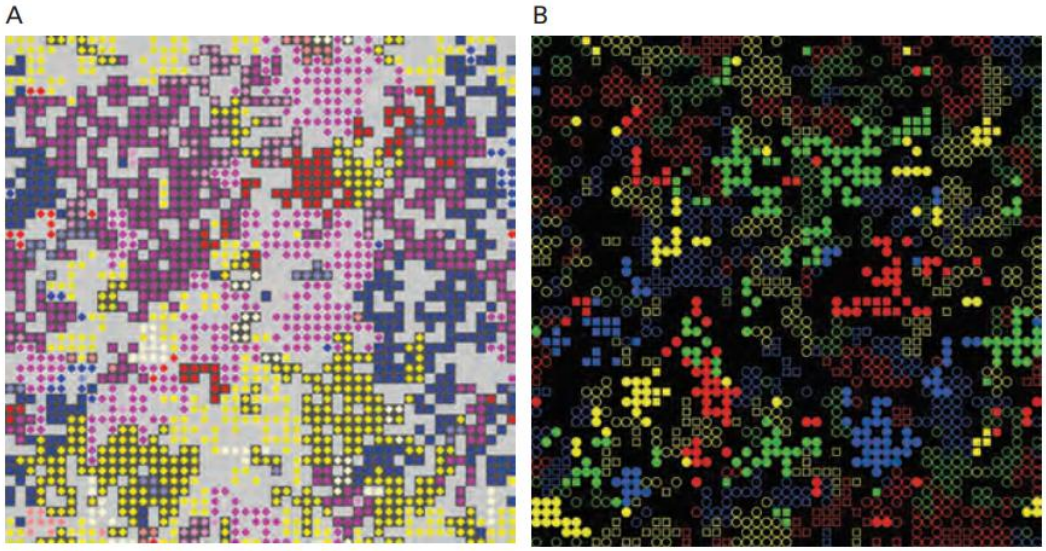
- A model can be said to be more or less valid based upon how closely it has been compared to the real process it is modeling.
- A model is never inherently valid.
- Its validity comes from the context of what it is being used for.
- Something in the model corresponds to something in reality.
- The user compare his observation's with real world and use it as the basis of his validation.
- Here we have understood the benefits of validation and question it arises.
- Credits: Uri Wilensky book.

Module No.157:

Here, we will understand about the "Replication".

Replication

- Model replication is the implementation by one researcher or group of researchers of a conceptual model previously implemented by someone else**.**
- Scientists must publish the details of how the experiment was conducted.
- Then subsequent teams of scientists carry out the experiment themselves to ascertain.
- Replicating a physical experiment strengthens original results.
- Comparing both the experimental setup and ensuing results.
- Replicating a computational model serves this same purpose.
- Replicating a computational model increases our confidence.
- New implementation of the model has yielded the same results as the original.



- This model includes a mechanism for inheritance of strategies.
- The model suggests that " ethnocentric " behavior can evolve under a wide variety of conditions.
- Even when there are no native " ethnocentrics ".
- Here we understand about the basic concepts of replication and how it is useful for the correctness of a model.

- Credits: Uri Wilensky book

Module No.158:

Here, we will understand about the “Replication of Computational Models: Dimensions and Standards”.

Replication

- Replication refers to the creation of a new implementation of a conceptual model based on the previous implementation.

Dimensions

- An original model and replicated model can differ across at least six dimensions.
- Time
- Hardware
- Languages
- Toolkits
- Algorithms
- Authors

Successful Replication

- A successful replication is one in which the replicators are able to establish that the replicated model creates outputs sufficiently similar to the outputs of the original.

Replication Standard

- The criterion by which the replication ’s success is judged is called the replication standard.
- Different replication standards exist for the level of similarity between model outputs.

Categories of Replication Standard

- There are three categories of replication standard.
- Numerical identity
- Distributional equivalence.
- Relational alignment.

Data

- ABMs usually produce large amounts of data.
- much of which is usually irrelevant.
- Data that are central to the conceptual model should be measured and tested during replication.

Summary

- Here, we understand about dimensions of conceptual model like time, hardware, toolkit, language, algorithms and author.
- Then we discussed about replication standard and its categories.
- Credits: Uri Wilensky book

Module No.159:

Overview

- We are going to see the benefits of replication in this section.

Benefits of Replication

- Advances scientific knowledge.
- Model verification.
- Model validation.
- Shared understanding.
- Shared understanding.
- Creation of sets of terms, idioms and best practices.
- Communicate about model.
- Model verification.
- Distinct implementations. producing same results.
- Conceptual model grows by capturing confidence.
- Correction made if there is any difference found in implemented model and replication.
- Model validation.
- Correspondence between the outputs.
- Validation of a model on the basis of output closer to real-world.
- Reevaluating the original mapping.
- Researchers investing and getting interested in it through replication
- Describing the modeling process through developing a language.
- Culture of replication fosters.
- "mean" and "standard deviation" applying them to tests, overtime, replication of ABM experiment to understand "time step" and "shuffled lists".

Summary

- Here, we understood the benefits of replication in modeling.
- Credits: Uri Wilensky book.

Module No.160:

Overview

- Here, we will learn about recommendations for model replicators.

Recommendations for model replicators

- RS is to produce the level of precision to establish hypothesis regularity.
- Examples : “numerical identity”, “distributional equivalence” and “relational alignment”.
- How detailed the description of the conceptual model is in the original paper or to communicate with authors or original model.
- Beneficial to delay the contact with original author until after first attempt to recreate the original model.
- Differences between public conceptual model and an implementation can be interesting and resulting in new discoveries.
- Becoming familiar with the toolkit in which original model was written.
- Better understanding of original model.
- Replicator understands the subtler working.
- Deliberately implementing a strategy.
- Obtain the source code of original model.
- Effective for illuminating discrepancies in the two model implementations.
- Groupthink
- Unconsciously adopting some of the practices of the original model developer.

Table 7.1
Details to be included in published replications. For each category, sample options to choose from are listed.

Categories of Replication Standards:
Numerical Identity, Distributional Equivalence, Relational Alignment
Focal Measures:
Identify particular measures used to meet goal
Level of Communication:
None, Brief email contact, Rich discussion and personal meetings
Familiarity with Language/Toolkit of Original Model:
None, Surface understanding, Have built other models in this language/toolkit
Examination of Source Code:
None, Referred to for particular questions, Studied in-depth
Exposure to Original Implemented Model:
None, Reran original experiments, Ran experiments other than original ones
Exploration of Parameter Space:
Examined results from original paper, Examined other areas of the parameter space

Summary

- We have understood the recommendation for model replicators in modeling.
- Credits: Uri Wilensky book.

Module No.161:

Overview

- Here, we will understand the recommendations for the model authors.

Recommendations for the model authors

- Research paper should be well specified.
- Complete source code for the model may not sufficient.
- Model authors must make their source code publicly available or at least the "pseudo code".
- To what extent the model developer presents a sensitivity analysis.
- Small modification to the original model can effect results drastically.
- These sensitive differences should be published by the authors.
- Original author may not be the original implementer.
- Implementation will not be veridical.
- Model authors implement their own models using " low-threshold " and toolkits.
- Authors to examine their conceptual models through the lens of a potential model replicator.

Summary

- We have understood the recommendations for model authors in modelling.
- Credits: Uri Wilensky boo

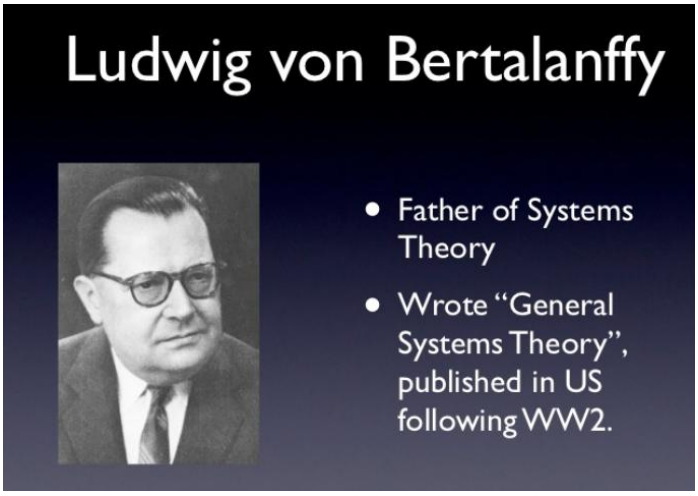
Module No.162:

Overview

- Here, we will understand the basics about complex adaptive systems (CAS).

Complex Adaptive Systems

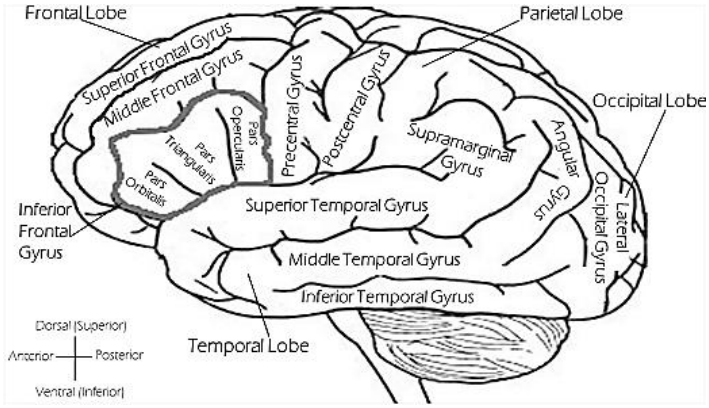
- Complexity
- Adaptation
- Systems



- The systems approach is an old concept.
- The approach stands on the assumption that breaking down of a complex concept into simple easy to understand units helps in better understanding of the complexity.
- Ludwig von Bertalanffy first proposed it as 'General System Theory'

- A system is a delineated part of the universe which is distinguished from the rest by an imaginary boundary. (NECSI)
- Properties of the system, the properties of the universe excluding the system which affect the system and the interactions / relationships.
- An adaptive system (or a complex adaptive system, CAS) is a system that changes its behavior in response to its environment.
- The adaptive change that occurs is often relevant to achieving a goal or objective. (NECSI)
- Complexity is:
- ...the (minimal) length of a description of the system.
- ...the (minimal) amount of time it takes to create the system.
- Here the length of a description is measured in units of information.

Examples



Summary

- We have learnt about the basic ideas of Complex Adaptive Systems.
- Credits: Uri Wilensky Book.

Module No.163:

Overview

- Here, we will understand about the historical perspective of CAS

History

- Started with the concepts of General Systems Theory
- Biologists focused primarily on individual cells
- Social Scientists focused on connections
- John Holland and cas/CAS

Summary

- We have learnt about the historical perspective of CAS.
- Credits: Niazi & Hussain book.

Module No.164:

Overview

- Here, we will understand about the basic ideas of complexity

Complexity

- Disambiguation from Computational Complexity

- Nonlinearity
- Large number of variables
- Chaos theory
- The term "chaos" is popularly used to refer to disorder or confusion.
- In science, chaos is an important conceptual paradox that has a precise mathematical meaning:
- A chaotic system is a deterministic system that is difficult to predict.
- A deterministic system is defined as one whose state at one time completely determines its state for all future times.
- So what does it mean for a chaotic system to be difficult to predict?
- Butterfly effect

Summary

- We have learnt about the basics of Complexity in CAS.
- Credits: Niazi and Hussain book.

Module No.165:

Overview

- Here, we will understand about adaptation in CAS.

Adaptation

- Adaptive changes in interactions
- Who changes
- What changes

- When changes
- How changes

Summary

- We have learnt about the adaptation in CAS.
- Credits: Niazi and Hussain book.

Module No.166:

Overview

- Here, we will understand about the Systems approach.

Systems Approach

- Concept of a System - boundaries
- Many systems can be correlated
- Example: Life/Humans/Society
- Whole vs. parts
- Reductionist vs. Wholistic approach

Summary

- We have learnt about the systems perspective of things.
- Credits: Niazi and Hussain book.

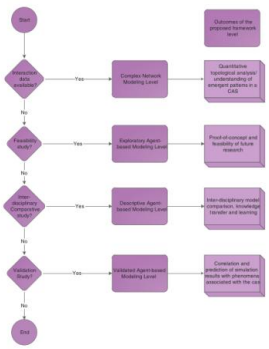
Module No.167:

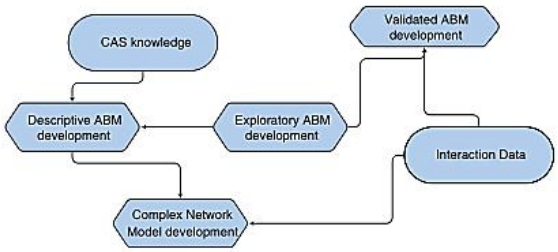
Overview

- Here, we will understand about the problems encountered in modeling of CAS.

Modeling of CAS

- Cognitive Agent-based (CABC) Framework





Summary

- We have learnt about the modeling of CAS using CABC framework
- Credits: Niazi and Hussain book.

Module No.168:

Overview

- Here, we will understand about the agent-based approach to modeling

Agent-based Approach

- Agent – something that acts
- There is a concept of agent in domains as distinct as:
    - AI/CS/Robotics
    - Sociology (Individual)
    - Biology
    - Ecology
    - Etc.
- CABC framework has 3 levels for Agent-based modeling
- Exploratory Agent-based modeling (EABM)
- Descriptive Agent-based Modeling (DREAM)
- Validated Agent-based Modeling (VOMAS-based Modeling or VABM)
- Exploratory ABM
- DREAM
- Virtual Overlay Multiagent System (VOMAS)
- Validated Agent-based Modeling

Summary

- We have learnt about various approaches for modeling using Agents and the CABC framework.
- Credits: Niazi and Hussain book.

Module No.169:

Overview

- Here, we will understand about complex networks/graphs and modeling using the CABC Level 1

Complex Network-based Modeling

- What is a graph?
- Graph/Network
- Same entity different terms in different domains
- Network = weighted graph
- Different aspects of modeling using networks

Labels

- arcs/lines
- Directed/Undirected
- Centrality-based
- Ego
- Sentiment graphs
- Community Detection

Key benefits

- Very well-defined
- Lot of free tools
- Lot of community support
- Well-defined mathematical models

Cons

- Difficult to figure out
- Difficulty in graph mining – lack of expertise in the community
- Missing data can be very difficult
- Requires complete data
- Complexity in deciding data boundaries

Summary

- We have learnt about complex-network modeling using the CABC framework
- Credits: Niazi and Hussain book.

Module No.170:

Overview

- Here, we will understand how to put modeling and simulation to use employing the CABC framework**.**

Cognitive Agent-based Computing Framework

- Data and Models

- How much data is needed
- Why choose Agent-based Modeling?
- Why choose Complex network modeling?
- When to choose EABM?
- When to choose DREAM?
- When to choose VOMAS?

Summary

- We have learnt about putting the CABC framework to work.
- Credits: Niazi and Hussain book.

Module No.171:

Overview

- Here, we will learn about the useful statistical models.

Useful Statistical Model

- The models that are useful in the case of limited data are:
- Queueing systems.
- Inventory and supply-chain systems.
- Reliability and maintainability.
- Limited data.
- Other distributions.

Queueing Systems

- Simulation solved the waiting line problems.
- Interval and service time patters were given in queuing examples.
- In these examples service and arrival time was probabilistic.
- But it is possible to have constant arrival time and constant service time.
- "Arrivals" can occur due to many reasons like machine break downs.
- Exponential distribution used to simulate the random exponential distribution of services.
- For special cases truncated normal distribution can be utilized.

Inventory and supply-chain systems

- Three random variables
- Number of units demanded per order or per time period.
- Time between demands.
- Lead time.
- Simple inventory systems have constant demand and lead time constant or zero.
- Mathematical tractability based demand and lead time in inventory theory text could be invalid.
- Variety of demand patterns are satisfied by geometric, Poisson and negative binomial distributions.

Reliability and Maintainability

- Failure time is modeled with many distributions.
- When failure occurs randomly model distribution is considered as exponential.
- Modeling standby redundancy required for gamma distribution.

Limited Data

- Three distributions.
- Uniform distribution
- Triangular distribution
- Beta distribution
- Uniform distribution is just a special case of beta distribution

Summary

- Here, we discussed about useful statistical model, queuing systems, Inventory and supply-chain systems, Limited data, Reliability, and maintainability.
- Credits: Jerry Banks book.

Module No.172:

Overview

- Here, we will learn about the Bernoulli distribution.

Bernoulli Distribution

- Bernoulli trail is one of the simplest experiment you can do in statistic and probability which has one out of two possible outcome.
- Bernoulli distribution is a discrete probability distribution for Bernoulli trail which has one out of two possible outcomes success or fail.
- Let us consider an example which consists of n trails and each of which can be a success or a failure.
- Xj = 1 if the jth experiment is success
- Xj = 0 if its failure.
- N trails and each trail has only two out comes success or failure.

$$p\left(X_1, X_2, \ldots\ldots, X_n\right) = p_1\left(X_1\right) \cdot p_2\left(X_2\right) \cdot \cdot \cdot p_n\left(X_n\right)$$

$$p_j(x_j) = p(x_j) = \begin{cases} p, & x_j = 1,\ j = 1, 2, \ldots, n \\ 1 - p = q, & x_j = 0,\ j = 1, 2, \ldots, n \\ 0, & \text{otherwise} \end{cases}$$

- Success here happens with probability (p) and failure at (p – 1).
- X is a discrete random variable. It takes value 1 in case of success and 0 in case of failure.

- Below is the expected value of random variable X is calculated

a $$E(X_j) = 0 \cdot q + 1 \cdot p = p$$

- Below is the variance of the random variable X is calculated as:

$$V(X_j) = \left[\left(0^2 \cdot q\right) + \left(1^2 \cdot p\right) \right] - p^2 = p(1-p)$$

Summary

- We have understand about the Bernoulli distribution.
- Credits: Jerry Banks book.

Module No.173:

Overview

- Here, we will learn about the binomial distribution.

Binomial Distribution

- Binomial distribution is the discrete probability distribution with parameters n and p of the number of successes in the sequence of n independent experiments.
- Each n and p with its own boolean-value.
- Random variable.
- Bernoulli trial's random variable X has binomial distribution:

$$p(x) = \begin{cases} \binom{n}{x} p^x q^{n-x}, & x = 0, 1, 2, \ldots, n \\ 0, & \text{otherwise} \end{cases}$$

- This equation is for calculating the probability of a specific outcome with all successes.

$$P(\overbrace{SSS\ldots\ldots SS}^{x \text{ of these}} \overbrace{FF\ldots\ldots FF}^{n-x \text{ of these}}) = p^x q^{n-x}$$

- S = success.
- In first X trials.
- n-x =f$^{n-x=f}$ (failures).

$$\binom{n}{x} = \frac{n!}{x!(n-x)!}$$

- q=1-p$q = 1 - p$
- This equation shows the required number of successes and failures.

$$p(x) = \begin{cases} \binom{n}{x} p^x q^{n-x}, & x = 0, 1, 2, \ldots, n \\ 0, & \text{otherwise} \end{cases}$$

- Again this equation can be used to calculate mean and variance of binomial distribution.
- X = sum of independent Bernoulli random variables. Then:

$$X = X_1 + X_2 + \cdots + X_n$$

- Expected value E(X)$E(X)$

$$E(X) = p + p + \cdots + p = np$$

- nn = total number of experiments.
- pp = probability of each experiment getting successful result.
- X = binomially random variable.
- Variance V(X)$V(X)$

$$V(X) = pq + pq + \cdots + pq = npq$$

- nn = total number of experiments.
- pp = probability of each experiment getting successful result.
- X = binomially random variable.
- q = probability of failure in each experiment.

Summary

- We have understand about the Binomial distribution.
- Credits: Jerry Banks book.

Module No.174:

Overview

- Here, we will learn about the Poisson distribution.

Poisson Distribution

- A discrete frequency distribution that gives the probability of independent events occurring in a fixed interval of time is known as the Poisson distribution.

$$p(x)=\begin{cases}\dfrac{e^{-\alpha}\alpha^{x}}{x!}, & x=0,1,\ldots\\ 0, & \text{otherwise}\end{cases}$$

- X = 0,1,2,3,….
- e= $e =$ 2.71828
- a= $a =$ mean number of successes in the given time interval or region of space.
- a>0 $a > 0$

$$E(X)=\alpha=V(X)$$

- This is the important property of Poisson distribution that is mean and variance is equal to a a .

$$F(x)=\sum_{i=0}^{x}\frac{e^{-\alpha}\alpha^{i}}{i!}$$

- This is the cumulative distribution function.

Summary

- We have understand the Poisson distribution.
- Credits: Jerry Banks book.

Module No.175:

Overview

- Here, We will understand “Uniform Distribution” also known as “rectangular distribution”.

Uniform Distribution

- The uniform distribution is a continuous distribution.
- It takes values within a specified range.
- Probability density function for a uniform distribution always take a random number on [0,1].

$$f(x) = \begin{cases} \frac{1}{b-a} & \text{if } a \le x \le b \\ 0 & \text{otherwise} \end{cases}$$

- Consider a random variable X on the interval [a, b] distributed uniformly. The, equation is given by:
- X=a+b-aR$\boldsymbol{X = a + (b - a)R}$
- For the derivative of function.
- First, compute the cdf of variable X^{X} .
- Second, set Fx=R $\boldsymbol{F(x) = R}$ on the range of X^{X} .
- Third, solve the equation **Fx**$^{\mathbf{F(x)}}$ in terms of R$^{\boldsymbol{R}}$ for **X**.
- **Step 1:**
    - The cdf for the equation is :

$$F(x) = \begin{cases} 0 & : x < 0 \\ x & : 0 \le x < 1 \\ 1 & : x \ge 1. \end{cases}$$

- If X^{X} takes only discrete values 0 and 1 then:

$$F(x) = \begin{cases} 0 & : x < 0 \\ 1/2 & : 0 \le x < 1 \\ 1 & : x \ge 1. \end{cases}$$

- **Step 2:**
- **Set**

$$F(x) = (X - a)(X - b)R$$

- **Step 3:**
- Solve for X^{X} in terms of R^{R} which proves.

X=a+b-aR$X = a + (b - a)R$

Summary

- Here, we understood about the fundamentals of Uniform Distribution.
- Credits, Discrete Event System Simulation, Jerry Banks.

Module No.176:

Overview

- Here, we will understand About “Exponential Distribution”.

Exponential Distribution

- The Exponential Distribution is a continuous valued probability distribution.
- Takes positive real values and λ which controls the shape of the distribution.
- The probability density function

$$f(x;\lambda) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0, \\ 0 & x < 0. \end{cases}$$

- This can also be defined as right-continuous Heaviside step function.

$$f(x;\lambda) = \lambda e^{-\lambda x} H(x)$$

- Here λ^{λ} is called the rate parameter.
- The cumulative distribution function

$$F(x;\lambda) = \begin{cases} 1 - e^{-\lambda x} & x \geq 0, \\ 0 & x < 0. \end{cases}$$

- This can also be defined as heaviside step function

$$F(x;\lambda) = (1 - e^{-\lambda x})H(x)$$

- Here λ^{λ} is the number of occurrences per time unit.

The inverse transform technique is used when the form of cdf is so simple that its f-$1^{f^{-1}}$ can be computed easily.

This technique for exponential distribution consist of four steps defined next.

- **Step 1:**
- Compute the cdf of random variable x^{x} given

$$F(x) = 1 - e^{-\lambda x}, \; x \geq 0.$$

- **Step 2**
- Set Fx=R$^{F(x) = R}$ on the range of x^{x} which becomes as.

$$1 - e^{-\lambda x} = R \text{ on the range } x \geq 0.$$

- **Step 3**
- Solve the equation FX=R$^{F(X) = R}$ for $X^{\text{for X}}$ in terms of R^{R} as follows:

$$\begin{aligned} 1 - e^{-\lambda x} &= R \\ e^{-\lambda x} &= 1 - R \\ -\lambda X &= \ln(1 - R) \\ X &= -\frac{1}{\lambda}\ln(1 - R) \end{aligned}$$

- Here, X is called random-variate generator.
- **Step 4**
- **Generate uniform random numbers to compute desired random variates.**

$$X_i = F^{-1}(R_i)$$

- For exponential case.

$$X_i = -\frac{1}{\lambda}\ln(1 - R_i)$$

- **If** i=1,2,3...$^{i = 1, 2, 3 \ldots}$ **replace** 1-Ri$^{1 - Ri}$ **with** R_i^{Ri} **as follows:**

$$X_i = -\frac{1}{\lambda} \ln R_i$$

- Which proves that both Ri^{R_i} and 1-Ri$^{1-R_i}$ are uniformly distributed on [0, 1].

Summary

- We understand about "Exponential Distribution"
- Credits, Discrete Event System Simulation, Jerry Banks.

Module No.177:

Overview

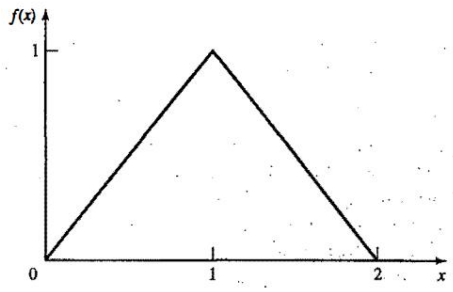
- Here, we will discuss about "Triangular Distribution".

Triangular Distribution

- A triangular distribution is a continuous probability distribution with a probability density function shaped like a triangle defined by
- The minimum value a^{a}
- The maximum value b^{b}
- The peak value **c**
- Where a<b$^{a < b}$ and a≤c≤b$^{a \leq c \leq b}$
- Probability density function with variable X^{X}

$$f(x) = \begin{cases} x, & 0 \leq x \leq 1 \\ 2-x, & 1 < x \leq 2 \\ 0, & \text{otherwise} \end{cases}$$

- Where endpoints are 0,2$^{(0,2)}$ and mode at 1^{1}.



- **Triangular Distribution**

- Its cumulative distribution function is given as :

$$F(x)=\begin{cases}0, & x\le 0\\ \frac{x^2}{2}, & 0<x\le 1\\ 1-\frac{(2-x)^2}{2}, & 1<x\le 2\\ 1, & x>2\end{cases}$$

- **For** $0\le X\le 1$: $0 \le X \le 1$:

$R=x22$ $R=\frac{x^2}{2}$

- Which implies that $\mathbf{x}=2R$ $x=\sqrt{2R}$
- **For** $1\le X\le 2$: $1 \le X \le 2$:

$R=1-2-X22$ $R=1-\frac{(2-X^2)}{2}$

- Which implies that $x=2-21-R$ $x=2-\sqrt{2(1-R)}$
- **Thus** X^X **is generated as:**

$$X=\begin{cases}\sqrt{2R}, & 0\le R\le \frac{1}{2}\\ 2-\sqrt{2(1-R)}, & \frac{1}{2}<R\le 1\end{cases}$$

Summary

- Here, we have understood the basics of Triangular Distribution, its pdf, and cdf.

Module No.178:

Overview

- Here, we will learn about characteristics of queueing systems.

CHARACTERISTICS OF QUEUEING SYSTEMS

- Customers and servers are the key elements of a queueing system.
- A Customer is anything that visits a facility and requires some service.

- Servers the service provider.
- Not every time the customer visits the service facility.

Table 6.1 Examples of Queueing Systems

System	*Customers*	*Server(s)*
Reception desk	People	Receptionist
Repair facility	Machines	Repairperson
Garage	Trucks	Mechanic
Tool crib	Mechanics	Tool-crib clerk
Hospital	Patients	Nurses
Warehouse	Pallets	Fork-lift Truck
Airport	Airplanes	Runway
Production line	Cases	Case-packer
Warehouse	Orders	Order-picker
Road network	Cars	Traffic light
Grocery	Shoppers	Checkout station
Laundry	Dirty linen	Washing machines/dryers
Job shop	Jobs	Machines/workers
Lumberyard	Trucks	Overhead crane
Sawmill	Logs	Saws
Computer	Jobs	CPU, disk, CDs
Telephone	Calls	Exchange
Ticket office	Football fans	Clerk
Mass transit	Riders	Buses, trains

Summary

- We have learnt about the characteristics of the Queueing systems.
- Credits: Jerry Banks books.

Module No.179:

Overview

- Here, we will learn about the calling population.

The Calling Population

- Calling population is the population of potential customers.
- Finite or infinite.
- Example of the bank having five machines and each machine is a customer and worker is its server.
- Infinite calling population when the system has a large population of potential.
- Examples are banks and restaurants.
- Arrival rate is the main difference between the finite and infinite population models.
- Infinite population system is not affected by the arrival rate.
- The Finite system depends on the no of customers waiting and being served.

Summary

- We have learnt about the calling population and what are effects of finite and infinite population.
- Credits: Jerry Banks book.

Module No.180:

Overview

- Here, we will learn about the system capacity.

System Capacity

- Every queueing system has some capacity.
- They have different number of customers entering the system.
- There is a limit for every queueing system to hold the number of customers.
- If the entering system finds the capacity full it will return to the calling population.
- Some systems have unlimited capacity
- Like tickets sold for a concert to students and many students can wait to purchase them.
- In limited systems, there is a distinction in arrival time and effective arrival time.

Summary

- We have learnt about the system capacity.
- Credits: Jerry Banks book.

Module No.181:

Overview

- Here, we will learn about the arrival process.

Arrival Process

- For infinite population models the arrival process characterized in terms of interarrival times of successive customers.
- Probability distribution is used to characterize the interarrival times when arrival is occurred in random time.
- Poisson arrival process is the most important for random arrivals.
- If A_n = inter-arrival
- Between n-1(customer) and n(customer) then
- A_n is exponentially distributed with mean $1/\lambda^{\lambda}$ time units.

- The arrival rate is λ^{λ} customers per time unit.
- So the number of arrivals in a time interval of length t has Poisson distribution with mean λt $^{\lambda t}$ customers.
- In restaurants and banks the Poisson arrival process has been employed successfully.
- Scheduled arrival is the second arrival class.
- Inter-arrival could be either constant or constant plus or minus a small random amount to represent early or late arrivals.

- Third situation occurs when one customer is always present in the queue.
- So we do not have an idle server.
- For finite population the arrival process is different.
- We define the customer as pending when he is out of the queue system and a member of potential calling population.
- **Arrival Process**

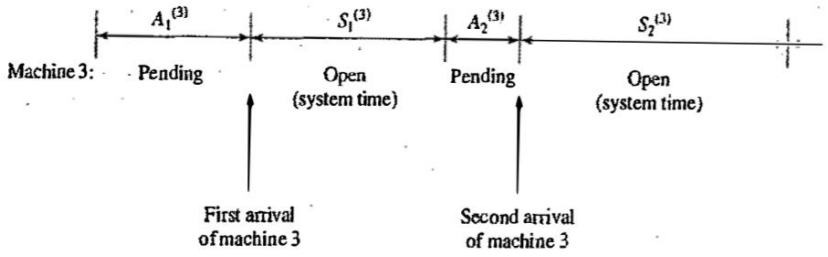


Figure 6.2 Arrival process for a finite-population model.

- Important application of finite population model is machine repair problem.

Summary

- We have learnt about the finite and infinite arrival processes.
- Credits: Jerry Banks book.

Module No.182:

Overview

- Here, we will learn about the Queue behavior and Queue discipline.

Queue Behavior

- The actions of the customer while waiting in the queue system for his service is known as the queue behaviour.
- The customers can baulk, renege or jockey.

Queue Discipline

- The logical ordering of the customers in a queue and determining that which customer should come for service when the server is free.
- FIFO, LIFO, SIRO, SPT and PR are some common queue disciplines.

Summary

- We have learnt about the Queue behavior and Queue discipline.
- Credits: Jerry Banks book.

Module No.183:

Overview

- Here, we will learn about the Service Times and the Service Mechanism.

Service Times and the Service Mechanism

- Service times of successive arrivals are denoted by S_1 S_2…..
- Their duration can be random or constant.
- This latter case is characterized as a sequence of random or identically distributed variables.
- The exponential, Weibull, gamma, lognormal and truncated normal distributions are used for service times.
- Sometimes the services are identically distributed for all same customers. Same type customers can have different service time.
- Number of service centers and interconnecting queues are present in a queue system.
- Parallel working servers for each service centre.
- Server = c
- Single server (c = 1)
- Multiple servers ($1<c<\infty^{\infty}$)
- Unlimited servers($c = \infty^{\infty}$)

Service Times and the Service Mechanism

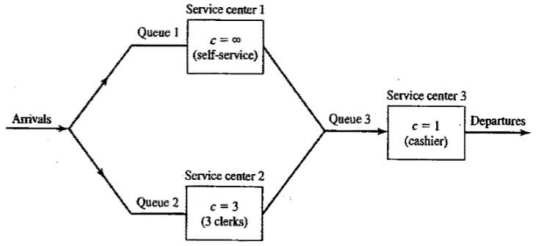


Figure 6.3 Discount warehouse with three service centers.

Service Times and the Service Mechanism

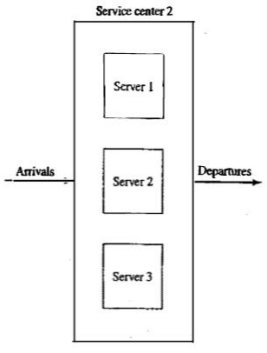


Figure 6.4 Service center 2, with $c = 3$ parallel servers.

Service Times and the Service Mechanism

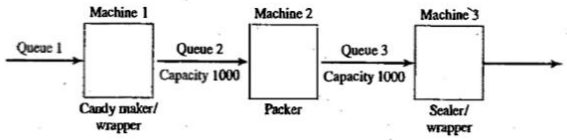


Figure 6.5 Candy-production line.

Summary

- We have learnt about the Service times and the Service mechanism.
- Credits: Jerry Banks book.

Module No.184:

Overview

- Here, we will learn about the Queueing notation.

Queueing Notation

- Proposed by Kendall [1953]
- For parallel server systems.
- A/B/c/N/K based on this format.
- “A” represents the inter-arrival-time distribution.
- “B” represents the service-time distribution.
- “C” represents the number of parallel servers.
- “N” represents the system capacity.
- “K” represents the size of the calling population.
- Common symbols for A and B include M (exponential or Markov)
- D (constant or deterministic).
- Ek (Erlang of order k).
- PH (phase-type).
- H (hyperexponential).
- G (arbitrary or general)
- Gl (general independent).

Queueing Notation

Table 6.2 Queueing Notation for Parallel Server Systems

P_n	Steady-state probability of having n customers in system
$P_n(t)$	Probability of n customers in system at time t
λ	Arrival rate
λ_e	Effective arrival rate
μ	Service rate of one server
ρ	Server utilization
A_n	Interarrival time between customers $n-1$ and n
S_n	Service time of the nth arriving customer
W_n	Total time spent in system by the nth arriving customer
W_n^Q	Total time spent in the waiting line by customer n
$L(t)$	The number of customers in system at time t
$L_Q(t)$	The number of customers in queue at time t
L	Long-run time-average number of customers in system
L_Q	Long-run time-average number of customers in queue
w	Long-run average time spent in system per customer
w_Q	Long-run average time spent in queue per customer

Summary

- We have learnt about the queueing notation.
- Credits: Jerry Banks book.

Module No.185:

Overview

- Here, we will learn about the long-run measures of performance of queueing systems.

LONG-RUN MEASURES OF PERFORMANCE

- The primary long-run measures of performance of queueing systems are:
- Long-run time-average number of customers in the system (L).
- In the queue (LQ).
- Long-run average time spent in the system (w).
- In the queue (wQ) per customer.
- Server utilization.
- A Proportion of time that a server is busy (p).
- Waiting line and mechanism is a system.
- System can also be sub-system of queue system.
- "queue" means waiting line alone.
- Other measurements include long-run proportion of customer who is delayed longer than $t^t{}_o$ time units.
- Customers turned away due to capacity constraints.
- Waiting line containing more than ko k_o customers.
- G/G/c/N/K is defined.
- Their performance relationships and simulation run.
- The two types of estimators are:

- An ordinary sample average.
- A time-integrated sample average.

Summary

- We have learnt about the long-run measures of performance of queueing systems.
- Credits: Jerry Banks book.

Module No.186:

Overview

- Here, we are going to learn about properties of random numbers.

Properties of Random Numbers

- Uniformity and independence are two important statistical properties.
- Each sample must be drawn from continuous uniform distribution between 0 and 1.

$$f(x)=\begin{cases}1, & 0 \le x \le 1\\ 0, & \text{otherwise}\end{cases}$$

- The density function and the expected value of R_i Ri is:

$$E(R)=\int_0^1 x\,dx=\left.\frac{x^2}{2}\right|_0^1=\frac{1}{2}$$

- And the variance is:

$$V(R)=\int_0^1 x^2\,dx-\left[E(R)\right]^2=\left.\frac{x^3}{3}\right|_0^1-\left(\frac{1}{2}\right)^2=\frac{1}{3}-\frac{1}{4}=\frac{1}{12}$$

- Generated by a digital computer.
- Individual computation is expensive so computationally efficient method should be used.
- Routine should be portable for different computers and programming languages.
- Routine having a long cycle.
- Replicable random numbers.
- Random numbers should closely approximate the ideal statistical properties.
- Techniques to generate random is easy to develop.
- Techniques that produce sequences that are independent and uniformly distributed are difficult.

Summary

- We have learnt about the properties of the random numbers.
- Credits: Jerry Banks book.

Module No.187:

Overview

- Here, we will learn about the linear congruential method.

Linear Congruential Method

- Widely used technique.
- Yields sequences with longer periods.
- Produces the sequence of integers.
- Between zero and m - 1^{m-1}.

$$X_{i+1} = (aX_i + c) \bmod m, \quad i = 0, 1, 2, \ldots$$

- xo^{x_o} = seed.
- a = $^{a=}$ multiplier.
- c = $^{c=}$ increment (c≠0)$^{(c \neq 0)}$.
- m= $^{m=}$ modulus.
- This is called mixed congruential method.
- When c = 0 then known as multiplicative congruential.
- Interval [0,1] is divided into n classes than expected observation for each interval is Nn $^{N/n}$.
- N is the total number of observations.
- Observing a value in a particular interval is independent.

Summary

- We have learnt about the linear congruential method.
- Credits: Jerry Banks book.

Module No.188:

Overview

- Here, we will learn about the generation of pseudo-random numbers.

Generation Of Pseudo-random Numbers

- False number generation.
- Removes the potential of true randomness.
- Set of random numbers can be replicated.
- Producing numbers between 0 and 1 that simulates the properties of uniform distribution and independence.

- Some errors or departures are:
- Not uniformly distributed random numbers.
- Not too high or too low variance.
- Generated numbers might not be continuous valued but are discrete valued.
- **Dependence:** Autocorrelation between numbers.
- Numbers successively higher or lower than adjacent numbers.
- Several numbers above the mean followed by the several numbers below the mean.
- Departures from uniformity and independence for a particular generation scheme often can be detected by tests.
- If detected then dropped in the favour of an acceptable generator.

Summary

- We have lear about the generation of pseudo-random numbers.
- Credits: Jerry Banks book.

Module No.189:

Overview

- Here, we will understand about the Tests for Random numbers.

Tests for Random numbers

- Two categories:
- 1. Uniformity.

$$H_0: \quad R_i \sim U[0, 1]$$
$$H_1: \quad R_i \not\sim U[0, 1]$$

- H_o denotes numbers distributed uniformly on the interval [0, 1].
- 2. Independence.

$$H_0: \quad R_i \sim \text{independently}$$
$$H_1: \quad R_i \not\sim \text{independently}$$

- H_o denotes the independent numbers.
- A level of significance α^{α} must be stated for each test.
- Where α^{α} is probability of rejecting the null hypothesis.

$$\alpha = P(\text{reject } H_0 \mid H_0 \text{ true})$$

- Software not specifically developed for simulation comes with random number generator pre-installed

Additional Tests:

- Good's serial test, the median-spectrum test, the run's test and a variance heterogeneity test.

Summary

- Here, we understood about the tests for random numbers.
- Credits: Jerry banks book.

Module No.190:

Overview

- Here, we will understand about the "Random Variate Generation".

Random Variate

- A random variate is a generated variable using uniformly distributed pseudorandom numbers.
- A random variate can be uniformly or nonuniformly distributed.
- Modeling activities is unpredictable or indeterminate.
- Random Variate Generations explain and illustrate some techniques for generating random
- Sometimes there is no availability of random-variate-generation libraries.
- It is up to the modeller to create an acceptable routine.
- This topic includes:
    - inverse-transform technique
    - Acceptance-rejection technique
- Special properties
- The composition method

Summary

- Here, we understood about Random Variate Generation.
- Credits: Jerry banks book.

Module No.191:

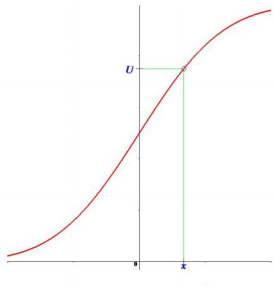
Overview

- Here, we will understand about the "Inverse Transform Technique".

Inverse Transform Technique

- The inverse-transform technique can be used to sample from the exponential.
- The uniform.
- The Weibull.
- The triangular and empirical distributions.
- The basic principle is to find the inverse function of F^{F} , F-1$^{F^{-1}}$ such that FF-1=I$^{FF^{-1} = I}$
- Where F-1$^{F^{-1}}$ denotes the solution of the equation r=Fx$^{r = F(x)}$ in terms of γ$^{\gamma}$.

Inverse Transform Technique



Inverse Transform Technique

- This technique explains:
    - Exponential Distribution
    - Uniform Distribution
    - Weibull Distribution
    - Triangular Distribution
    - Empirical Continuous Distribution
    - Continuous Distribution
    - Discrete Distribution

Summary

- Here, we understood about Inverse Transform Technique.
- Credits: Jerry banks book.

Module No.192:

Overview

- Here, We will understand “Acceptance Rejection Technique”.

Acceptance Rejection Technique

- Acceptance Rejection Technique is a method used to generate random variate using some steps explained next.
- Step 1. Generate a Random Number R^{R} .

- Step2a. If R≥14$R \geq 1/4$ Accept RR .
- Step2b. If R≤14$R \leq 1/4$ Reject RR .
- If another uniform random variate on [1/4, 1] is needed, repeat the procedure.
- The random variate generated using above methods is indeed uniformly distributed over [1/4, 1].
- Take any 14≤a<b≤1$1/4 \leq a < b \leq 1$ then

$$P(a < R \leq b | 1/4 \leq R \leq 1) = \frac{P(a < R \leq b)}{P(1/4 \leq R \leq 1)} = \frac{b-a}{3/4}$$

- Proves that probability for uniform distribution is on [1/4, 1].
- The efficiency of an acceptance-rejection technique rest on reducing the total of rejections.
- Which depends on computer being used.
- The skills of programmer.
- Rejected numbers.
- The Distributions which operates on acceptance rejection technique are:
    - Poisson Distribution
    - Non-stationary Poisson process
    - Gamma Distribution.

Summary

- Here, we understood about the Acceptance Rejection Technique.
- Credits, Jerry Banks book.

Module No.193:

- **Overview**
- Here, we will discuss about “Other topics in Randomness”.

Special Properties

- Special Properties are variate generation techniques based on a particular feature of probability distributions.

Direct Transformation

- Direct transformation is used for the normal and lognormal distributions.
- The cdf is given by:

$$\Phi(x) = \int_{-\infty}^{x} \frac{1}{\sqrt{2\pi}} e^{-t^2/2} dt, \qquad -\infty < x < \infty$$

- Box and Muller made this method and is easy to program in scientific programming languages.

Convolution Method

- The probability distribution of a sum of two or more independent random variables is called convolution method.
- This method uses two random variables.
- Sum them and generate a new random variable.
- This technique can be applied on Erlang and Binomial variates.

More Special Properties

- The other relationship among distributions include relation between gamma and beta distribution.
- We can generate beta variates, using two gamma variates with acceptance-rejection technique.

Summary

- Here, We understood about the Special Properties, Direct transformation, convolution method and other special properties.
- Credits: Jerry banks book.

Module No.194:

Overview

- Here, we will discuss about “Data Collection”.

Data Collection

- The relation between construction of the model and the collection of needed input data is constant.
- The required data elements changes with complexity of the model.
- Time taking nature of data collection.
- Which type of data is to be collected is dictated by the objectives of study.

Summary

- Here, We understood about the Data Collection.
- Credits: Jerry banks book.

Module No.195:

Overview

- Here, we will understand About “Identifying the Distribution with Data”.

Identifying the Distribution with Data

- Data distributions are graphical methods.
- These methods are used for selecting input distribution families.
- The specific distribution is often specified by estimating its parameters.

Summary

- We understood about Data Distribution and how data parameters are selected.
- Credits, Jerry Banks book.

Module No.196:

Overview

- Here, we will understand about the “Histograms”.

Histograms

- Histograms is a powerful technique for identifying how a distribution is shaped.
- Histogram is also known as frequency distribution.
- There are some methods for constructing a histogram.

  1. Divide the data ranges into intervals of equal width.

  2. Label the horizontal axis to follow the nominated intervals.

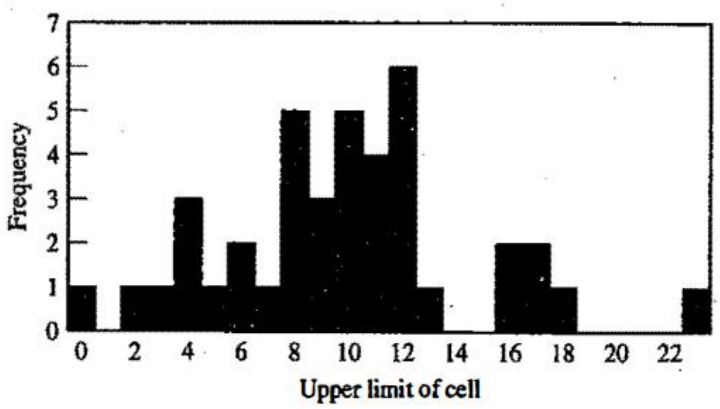
  3. Find the frequency of occurrences inside each intervals.

  4. Label the vertical axis to plot total occurrences for every interval.

  5. Plot the frequencies on the vertical axis.

- The number of class intervals is decided by the number of observations.
- Amount of scatter in the data.

Histograms



Summary

- Here, we understood about the "Histograms".
- Credits: jerry banks book.

Module No.197:

Overview

- Here, we will understand about the "Selecting the Family of Distributions".

Selecting the Family of Distributions

- The selection of a family of distributions is based on the context for what it is being investigated.
- The shape of the histogram.
- The Distributions which are frequently encountered and easy to analyze includes exponential, normal and poison distribution.
- Whereas difficult to analyze includes the beta, gamma and Weibull distributions.
- Out of many, Here are some examples of distributions which were created on the physical basis.
    -
        - Binomial
        - Negative Binomial
        - Poisson
        - Normal
        - Lognormal

- An input Model is an estimate of reality.
- There is no "true" distribution for any stochastic input process.

Summary

- Here, we understood about the Family of distributions, its types and input model.
- Credits: Jerry banks book.

Module No.198:

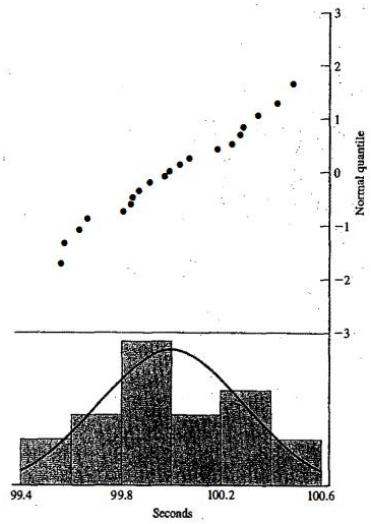
Overview

- Here, we will understand about the “Quantile-Quantile Plot”.

Quantile-Quantile Plot

- A Quantile-Quantile is a graphical method for comparison of two probability distributions by plotting their qualities against each other.
- If Fγ=PX≤γ=q$F(\gamma) = P(X \leq \gamma) = q$ for 0<q<1$0 < q < 1$
- Here x is random variable with cdf FF .
- Q-quantile of XX is value of .
- We write γ=F-1q$\gamma = F^{-1}(q)$ when FF has an inverse.
- If xi,i=1,2,3⋯n$\{x_i, i = 1, 2, 3 \cdot \cdots n\}$ and yj,j=1,2,3⋯n $\{y_j, j = 1, 2, 3 \cdot \cdots n\}$
- Where y1≤y2≤⋯≤yn$y_1 \leq y_2 \leq \cdots \leq y_n$
- Let j j denote the ranking of order
- j=1$j = 1$ **for smallest**
- j=n$j = n$ **for largest**
- Then
- yjy_j ≈$\approx$ F-1j-12n $F^{-1}\left(\frac{j-\frac{1}{2}}{n}\right)$

Quantile-Quantile Plot



Summary

- Here, we understood about the “Quantile-Quantile Plot”.
- Credits: jerry banks book.

Module No.199:

Overview

- Here, we will learn about the parameter estimation.

Parameter Estimation

- After selecting the family of distribution, the next step is to estimate the parameters of the distribution.
- Many useful distribution estimators are described here.
- Software packages integrated into simulation languages are available.

Summary

- We have learnt about the parameter distribution.
- Credits: Jerry Banks book.

Module No.200:

Overview

- Here, we will learn about the preliminary statistics: sample mean and sample variance.

Preliminary Statistics

- To estimate the parameters of a hypothesized distribution.
- When discrete and continuous raw data is available then the following equation is used:

$$\bar{X} = \frac{\sum_{i=1}^{n} X_i}{n}$$

- x $\bar{x}$ = sample mean.
- x1,x2,...,xn $x_1, x_2, \ldots, x_n$ = observation in a sample of size n.
- The sample variance s2 s^2 is defined as:

$$S^2 = \frac{\sum_{i=1}^{n} X_i^2 - n\bar{X}^2}{n-1}$$

- When discrete or continuous raw data are available following modified equations are used:

$$\bar{X} = \frac{\sum_{j=1}^{k} f_j X_j}{n}$$

- Where x $\bar{x}$ is the sample mean.
- And the sample variance is :

$$S^2 = \frac{\sum_{j=1}^{k} f_j X_j^2 - n\bar{X}^2}{n-1}$$

- Here k is the number of distinct values of X.
- F_j is the observed frequency of the value X_j of X.
- When data is discrete or continuous and have been placed in class intervals then the sample mean and the sample variance are approximated from the following equations:

$$\bar{X} = \frac{\sum_{j=1}^{c} f_j m_j}{n}$$

- For the sample variance the following equation is used.

$$S^2 = \frac{\sum_{j=1}^{c} f_j m_j^2 - n\bar{X}^2}{n-1}$$

- fj f_j observed frequency of j_{th} jth class interval.
- mj m_j midpoint of j_{th} jth interval.
- c c = number of intervals.

Summary

- We have learnt about the preliminary statistics: sample mean and sample variance.
- Credits: Jerry Banks book.

Module No.201:

Overview

- **Here, we will learn about the evolution of computer systems architectures**

Evolution of Computer Systems Architectures

- ENIAC was the first computer system with no operating system and was large in size. They used vacuum tubes.
- Early computer doesn't have a good operating system, databases.
- They doesn't have any high-level programming language for simplifying program.

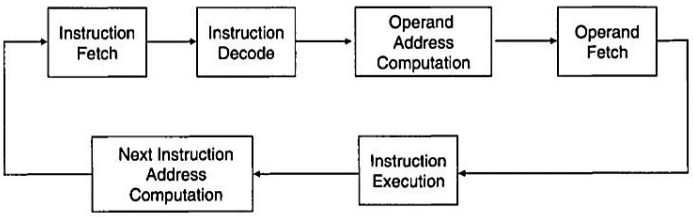
Systems Architecture

- Central Processing Unit.
- Graphics Processing Unit.
- Temporary memory.
- Permanent Memory.
- Input unit.
- Output unit.
- Control element.
- Disk Drives.

CPU Architectures

- Central processing unit.
- CU fetch functions.
- ALU performs functions.
- Registers to store the operands.
- L1, L2, L3 Cache.
- CPU Execution cycle.

Instruction architectures



Instructions and Memory Architectures

- Reduced instruction set computer (RISC).
- Highly optimized instructions.
- Complex instruction set computer (CISC).
- Memory address register.
- Memory data register.

Secondary Storage and Peripheral Device Architecture

- I/O devices control secondary storage.
- Magnetic tape drives.
- Magnetic disk drives.
- Compact optical disk drives.
- Disk jukeboxes.

- High density data storage.

Network Architecture

- Share information and processing capacity in real life.
- Another path to send or receive data.
- Central switching element. Central storage repository.
- Connected using intelligent interface units.

Summary

- We have learnt about the evolution of computer systems architectures.
- Credits: Paul Fortier book.

Module No.202:

Overview

- Here, we will learn about the evolution of database systems.

Evolution of Database Systems

- Powerful tools to manage information.
- Early computers lag in online data storage.
- System architects have to reply on external storage.
- Initial storage was constructed using simple direct addressing.
- File system used coarse semantic means.
- Information Centralization.
- Security.
- Integrity.
- File systems added finer-grained definitions of stored information.
- Relational database system was introduced.
- Database must be entered at a specific entry point.
- Must traverse the path leading to the specific item.
- An issue for network is its legacy.
- Database system's demise began with development.
- Publication of Codd’s relational database model.
- Information can be formed into tables called relations.
- Have to use calculus and algebra to operate.
- Concurrent access was not present in early network-based databases.
- The serializability theory and concurrency control led to further improvements in database technology.
- “ACID” properties.
- Object-oriented database was developed.
- For application developers to store data and data types efficiently.
- But did not possess some fundamental concepts.
- Object relational databases.
- Embraced by the U.S and international venders.
- Next change will be in the area of transactions and transaction processing.
- ACID properties may be altered.

Summary

- We have learnt about the evolution of database systems.
- Credits: Paul Fortier book.

Module No.203:

Overview

- **Here, we will learn about the evolution of operating systems.**

Evolution of Operating Systems

- Operating system is computer's software, firmware, and hardware.
- Interacts at low-level.
- Manage sharing of computer's resources.
- The most privileged of software elements on the system.
- Requires hardware.
- Early coders load the software in a location then make hardware to start processing.
- No automated means to switch between different jobs.
- Monitor or batch operating system provided the solution to transition between processing jobs.

Services of Operating System

- Process and Hardware management.
- Memory management.
- I/O management.
- Inter-process synchronization and communications.
- Resource allocation.
- Protection of system and user resources.

Evolution of Operating Systems

- interrupt service routine checks everything and determines what to perform next.
- Efficient use of computing facilities.
- Convenient interface to user by hiding the details of machine.
- Transparent use of computer resources.
- Improved architectures and instruction execution schemes.
- Improve the performance of the processor.
- Not to depend on a single processor
- Add entire new processors.
- Distributed processors.
- Processors operating system is the single global operating system.
- In one case the entire operating system can be replicated on each site.
- Each processor has a specific function.
- New concepts are still being examined.
- Client/server processing.

- Multiprocessing operating system forum.

Summary

- We have learnt about the evolution of operating systems.
- Credits: Paul Fortier book.

Module No.204:

Overview

- Here, we will learn about the evolution of computer networks

Evolution of Computer Networks

- The term network can mean many things.
- In computer it refers to combination of interconnected resources.
- ARPANET was an early computer network.
- A loose coalition of devices that share information.
- All this evolved into internet.
- ARPANET provided early research into communication protocols.
- Greater availability and access to a wider range of devices.
- As more applications were added.
- Information access and manipulation took on greater emphasis.
- As the networks improve new applications arose.
- Remote job execution was added with information exchange.
- But still real-time was not provided.
- They had to develop more advance protocols and network operating systems.
- It was time consuming.
- Local area network was introduced in 1970s.
- LAN became more widely available.
- LAN designs have been produced to fit an extremely wide spectrum of user requirements.
- LAN is widely used in all aspects of society.
- LAN had rapid growth in 1990s and early 2000s.
- LAN can connect a device from anywhere.
- LAN is used to link factory robots.
- LAN provide sensor data, control data and feedback.
- Walt Disney World used LAN for different services.
- Large banks also use LAN for interconnection of their local sites.
- Wireless technology has improved.
- Began in 1970s with Aloha net as foundation.
- Wireless phone network development has opened the door for computer networks.
- More development will be made to provide more applications.

Summary

- We have learnt about the evolution of computer networks.
- Credits: Paul Fortier book.

Module No.205:

Overview

- **Here, we will learn about the need for performance evaluation**

Need for Performance Evaluation

- Selecting a specific computer architecture for an application.
- An operating system.
- A database system.
- A wide area or local area network system.
- If not used properly then decrease productivity.
- Operating system can increase the concurrency access.
- Or can block access.
- A database can provide efficiently sharing information among many applications.
- Or blocking of information by dropping data availably.
- LAN provides means for streamline information processing and eliminate redundancies.
- But it may also deter users from logging on because of link or protocol problems.
- We need to select the computer system mapped to the needs.
- A computer system can be very simple with simple processor etc.
- A computer can be highly elaborate with multiple processors etc.
- A purchaser must decide according to motherboard its using.
- While purchasing the system all this must be kept in mind.

Summary

- We have learnt about the Need for performance evaluation.
- Credits: Paul Fortier book.

Module No.206:

Overview

- Here, we will learn about role of performance evaluation in computer engineering.

Role of Performance Evaluation in Computer Engineering

- Client/server architectures.
- These systems are from Government, industry and academia.
- There is constant demand for improved reliability and availability.
- Present systems provide more features.
- Sharing of resources on global scale.
- Ability to bring computing power to the user.
- The optimal design or selection of computer system is therefore very important.

Summary

- We have learnt about the role of performance evaluation in computer engineering.
- Credits: Paul Fortier book.

Module No.207:

Overview

- Here, we will learn about the overview of performance evaluation methods.

Overview of Performance Evaluation Methods

- Models provide a tool to define a system.
- Define its problems in a concise fashion.
- This provide an environment to study a system before its existence.
- Developed on the base of theoretical laws and principles.
- Modeling is better if:
- Physical laws are available.
- Pictorial representation is possible.
- Input/output elements are manageable.
- But we don't have clear physical laws.
- Faithful model of a system must be constructed.
- Model a slice of the real-world system.
- Which element is important to understand in faithful system.
- We can say its intuition.

Summary

- We have learnt about the overview of performance evaluation methods.
- Credits: Paul Fortier book.

Module No.208:

Overview

- Here, we will learn about performance metrics and evaluation criteria**.**

Performance Metrics and Evaluation Criteria

- Performance metrics and evaluation criteria are required for systems.
- User must follow a methodology.
- The motivations.
- The environment.
- Technological boundaries.
- Must understand the usage of the system.
- Needs and uses must be defined.
- Must compile a wish list of all potential uses.
- User must know processing requirements.

- Communication transfer.
- Management requirements.
- Define or know why we need something at first priority.
- Checking in which environment the system fits.
- Checking in which technological aspect it fits.
- Connecting to diverse set of company assets.
- Link planned new resources.
- Need large volume of data for system requirements.
- How this data assist in the selection?
- Collection of data can be divided in quantitative and qualitative.
- Specific performance measures can be derived from one set of data.
- Subjective measures can be derived from another.

Summary

- We have learnt about the performance metrics and evaluation criteria.
- Credits: Paul Fortier book.

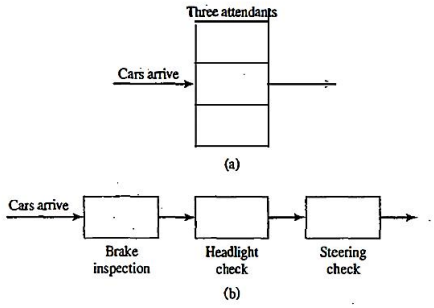
Module No.209:

Overview

- Here, we will understand about the "Comparison of Two System Designs".

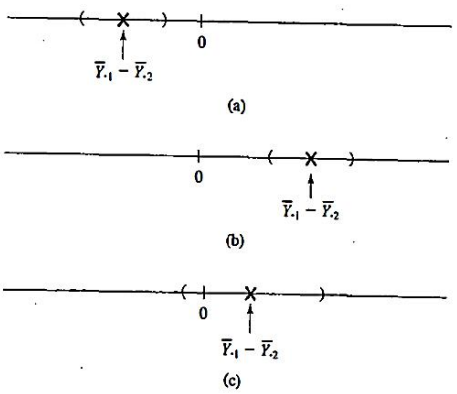
Comparison of Two System Designs

- A simulation analyst can compare two possible configurations.
- Using supply chain inventory system there ordering policies will be compared.
- Replications is used to analyze the output data.
- The mean performance measure for system 'I' will be denoted by θi=$\hat{i}$=1,2.$\theta_i = (i = 1,2).$
- The goal of the simulation experiments is to obtain point.
- Interval estimates of the difference in mean performance.
- Namely θ1-θ2$(\theta_1 - \theta_2)$.
- Inverse Transform Technique
- Replications is used to analyze the output data.
- The mean performance measure for system 'I' will be denoted by θi=$\hat{i}$=1,2.$\theta_i = (i = 1,2).$
- Vehicle Safety Inspection:



- In comparing two systems, number of replications RiR_i to be made and run length TEi$T_E^{(i)}$ for each model i=1,2$(i = 1,2)$ is decided.

System	Replication 1	2	...	R_i	Sample Mean	Sample Variance
1	Y_{11}	Y_{21}	...	$Y_{R1}1$	$\overline{Y}_{\cdot 1}$	S_1^2
2	Y_{12}	Y_{22}	...	$Y_{R2}2$	$\overline{Y}_{\cdot 2}$	S_2^2



Summary

- Here, we understood about Comparing two system designs.
- Credits: Jerry banks book.

Module No.210:

Overview

- **Here, We will understand "Independent Sampling with Equal Variances".**

Independent Sampling with Equal Variances

- Independent Sampling refers to different and independent random numbers streams to simulate the two systems.
- **Acceptance-Rejection**

- The observations of simulated system 1, Yr1,r=1,⋯,R1 $\{Y_{r_1}, r = 1, \cdots, R1\}$ are all statistically independent of simulated system 2, , Yr2,r=1,⋯,R2 $\{Y_{r_2}, r = 1, \cdots, R2\}$.
- Yi Y_i is given by:

$$V(\bar{Y}_{.i}) = \frac{V(Y_{ri})}{R_i} = \frac{\sigma_i^2}{R_i}, \quad i = 1, 2$$

- Where Y1 $\overrightarrow{Y_1}$ and Y2 $\overrightarrow{Y_2}$ are statistically different as:

$$V(\bar{Y}_{.1} - \bar{Y}_{.2}) = V(\bar{Y}_{.1}) + V(\bar{Y}_{.2}) = \frac{\sigma_1^2}{R_1} + \frac{\sigma_2^2}{R_2}$$

- It implies two variances are equal (but unknown in value) σ12=σ22 $\sigma_1^2 = \sigma_2^2$.
- If two-sample-t confidence interval approach is used. The point estimate of the mean difference is: Y1 $\overrightarrow{Y_1}$ - Y2 $\overrightarrow{Y_2}$
- Where Yi $\overrightarrow{Y_i}$ is given as:

$$S_i^2 = \frac{1}{R_i - 1}\sum_{r=1}^{R_i}(Y_{ri} - \bar{Y}_{.i})^2 = \frac{1}{R_i - 1}\left(\sum_{r=1}^{R_i} Y_{ri}^2 - R_i\bar{Y}_{.i}^2\right)$$

- Here, Si2 S_i^2 is an estimator of variance σi2 σ_i^2. Assuming σ12=σ22=σ 2 $\sigma_1^2 = \sigma_2^2 = \sigma^2$ a pool estimate of σ 2 σ^2 is obtained as:

$$S_p^2 = \frac{(R_1 - 1)S_1^2 + (R_2 - 1)S_2^2}{R_1 + R_2 - 2}$$

- Where v=R1+R2-2 $v = R_1 + R_2 - 2$
- The c.i. θ1-θ2 $\theta_1 - \theta_2$ for with the standard error is computed by:

$$s.e.(\overline{Y_1} - \overline{Y_2}) = S_p\sqrt{\frac{1}{R_1} + \frac{1}{R_2}}$$

- The standard error is an estimate of the standard deviation of the point given as: σ1R1+1R2$\sigma\sqrt{1/R_1 + 1/R_2}$.

Summary

- Here, we understood about the Independent Sampling.
- Credits, Jerry Banks book.

Module No.211:

Overview

- Here, we will discuss about "Independent Sampling with Unequal Variances".

Independent Sampling with Unequal Variances

- If the assumption of equal variances cannot safely be made, an approximate 1001-α% $100(1-\alpha)\%$ for θ1-θ2$\theta_1 - \theta_2$ can be computed as:

$$s.e.(\overline{Y_1} - \overline{Y_2}) = \sqrt{\frac{S_1^2}{R_1} + \frac{S_2^2}{R_2}}$$

- Degrees of freedom, v can be as:

$$v = \frac{(S_1^2/R_1 + S_2^2/R_2)^2}{[(S_1^2/R_1)^2/(R_1-1)] + [(S_2^2/R_2)^2/(R_2-1)]}$$

- A minimum number of replications R1≥6$R_1 \geq 6$ and R2≥6$R_2 \geq 6$ is recommended for this procedure.

Summary

- Here, We understood about Independent Sampling with Unequal Variances.
- Credits: Jerry banks book.

Module No.212:

Overview

- Here, we will understand About “Common Random Numbers”.

Common Random Numbers

- CRN means, for each replication, the same random numbers are used to simulate both systems.
- Therefore, R1 and R2 must be equal, say R1 = R2= R.
- The purpose of CRN is to induce positive correlation betweeny1 and y2y_1 and y_2 .
- A variance reduction in point estimator of mean difference is given by:

$$V(\bar{Y}_{.1}-\bar{Y}_{.2}) = V(\bar{Y}_{.1})+V(\bar{Y}_{.2})-2\text{cov}(\bar{Y}_{.1},\ \bar{Y}_{.2})$$
$$= \frac{\sigma_1^2}{R}+\frac{\sigma_2^2}{R}-\frac{2\rho_{12}\sigma_1\sigma_2}{R}$$

- Compare variance of Y1-Y2$\overrightarrow{Y_1}-\overrightarrow{Y_2}$, call it V_{crn} given as :

$$V_{CRN} = V_{IND} - \frac{2\rho_{12}\sigma_1\sigma_2}{R}$$

- Which implies:

$$V_{CRN} < V_{IND}$$

- To compute a 100(1 - a)% c.i compute the differences:

$$D_r = Y_{r1} - Y_{r2}$$

- Compute Sample mean difference:

$$\bar{D} = \frac{1}{R}\sum_{r=1}^{R} D_r$$

- The sample variance of the difference $\{D_r\}$ is computed as:

$$S_D^2 = \frac{1}{R-1}\sum_{r=1}^{R}(D_r - \bar{D})^2$$
$$= \frac{1}{R-1}\left(\sum_{r=1}^{R} D_r^2 - R\bar{D}^2\right)$$

- Where standard error of Y1-Y2 $\overrightarrow{Y_1} - \overrightarrow{Y_2}$ is estimated by:

$$\text{s.e.}(\bar{D}) = \text{s.e.}(\bar{Y}_{.1} - \bar{Y}_{.2}) = \frac{S_D}{\sqrt{R}}$$

- Implementation of common random number:

1. Dedicate a random-number stream to a specific purpose.
2. For system with external arrivals, dedicate one random stream to one of arrival and their attributes.

1. For systems having an entity performing given activities in a cyclic fashion, assign a random-number stream to this entity.
2. If synchronization is not possible use independent steams of random numbers for this subset.

Summary

- We understand about Common Random Numbers.
- Credits, Jerry Banks book.

Module No.213:

Overview

- Here, we will understand about the “Confidence Intervals with Specified Precision”.

Confidence Intervals with Specified Precision

- Confidence intervals for the difference between two systems' performance can be achieved in an parallel way.
- If the goal is to obtain error in our estimate θ1-θ2 $\theta_1 - \theta_2$ to be less than $\pm\varepsilon$ $\pm\varepsilon$ then number of replications R should be:

$$H = t_{\alpha/2,\nu}\,\text{s.e.}(\bar{Y}_{.1} - \bar{Y}_{.2}) \leq \epsilon$$

- Using R0≥2$R_0 \geq 2$ we obtain an initial estimate.
- We solve total number of replication R≥R0$R \geq R_0$ to achieve half-length criterion.
- We make replications R≥R0$R \geq R_0$ for each system to compute confidence interval and half-length criterion.

Summary

- Here we understand about Confidence Intervals with Specified Precision.
- Credits: Uri Wilensky book

Module No.214:

Overview

- Here, we will understand about the "COMPARISON OF SEVERAL SYSTEM DESIGNS".

COMPARISON OF SEVERAL SYSTEM DESIGNS

- To compare "K" alterative system designs.
- Many different statistical procedures have been developed.
- These are used to analyze simulation data and to draw statistically sound interfaces about parameters θiθ_i .
- These procedures are classified as:
    - Fixed-sample size.
    - Sequential sampling.

- **Fixed Sample Size**

  A predetermined sample size is used to draw inferences via hypothesis tests or confidence intervals.

Example:

-
    - The interval estimation of a mean performance measure.

Advantages:

-
    - An easily estimated or known cost in terms of computer time before running the experiments.
    - Can be used when :
    - Computer time is limited.
    - A pilot study is being conducted.

Disadvantages

-
    - A strong Conclusion could be impossible.
    - A hypothesis test may lead to a failure to reject the null hypothesis.

- **Sequential Sampling**
    - In sequential sampling more and more data are collected until an estimator with a pre-specified precision is achieved.
    - In two stage procedure, an initial sample estimates additional observations needed to draw conclusions with a specified precision.
    - Goal of simulation analyst
    - Estimation of each parameter, θiθ_i .
    - Comparison of each performance measure.
    - All pair wise comparisons.
    - selection of the best θiθ_i .

Summary

- Here, we understood about the "COMPARISON OF SERAL SYSTEM DESIGNS".
- Credits: jerry banks book.

Module No.215:

Overview

- Here, we will understand about the "Bonferroni Approach to Multiple Comparisons".

Bonferroni Approach to Multiple Comparisons

- If C confidence intervals are computed and that the ith^{it_h} interval has confidence coefficient 1-αi$^{1-\alpha_i}$.
- The Bonferroni inequality states that:

$$P(\text{all statements } S_i \text{ are true}, i = 1, \ldots, C) \geq 1 - \sum_{j=1}^{C} \alpha_j = 1 - \alpha_E$$

- Where

$\alpha_E = \sum_{j=1}^{C} \alpha_j$ is called overall error probability.

- **This is equivalent to:**

$$P \text{ (one or more of the } C \text{ confidence intervals does not contain the parameter being estimated)} \leq \alpha_E$$

- Here, aE$^{\alpha^E}$ provides the upper bound.

- A major advantage of Bonferroni approach is that runs with independent sampling and or common random numbers.
- The major disadvantage of the Bonferroni approach in making a large number of comparisons is the increased width of each individual interval.
- The width of a c.i. is a measure of the precision of the estimate.
- 3 possible ways of using the Bonferroni Inequality when comparing K alterative system designs:

1. Construct a 1001-αi%$^{100(1-\alpha_i)\%}$ c.i. for parameter θi $^{\theta_i}$, where number of intervals is C = K.

2. Compare all designs to one specific design that is:

Construct a 1001-αi%$^{100(1-\alpha_i)\%}$ c.i. for θi -θ1$^{\theta_i - \theta_1}$.

Here, the number of intervals is C = K – 1.

3. Compare all designs to each other. That is :

- Construct a 1001-αi% $^{100(1-\alpha_i)\%}$ c.i. for θi -θj$^{\theta_i - \theta_j}$.
- With K designs, the number of intervals is computed as C = K (K – 1) / 2.

Summary

- Here, we understood about Bonferroni Approach.
- Credits: Jerry banks book.

Module No.216:

Overview

- Here, We will understand "Bonferroni Approach to Selecting the Best".

Bonferroni Approach to Selecting the Best

- Gross Level

- If there are K system designs, and the iTH i_{TH} design has expected performance θiθ_i .
- Then , we're interested in best system.
- Where "best" means maximum expected performance.
- Refined
- At a refined level, we are also interesting in knowing how much better the best is relative to alternatives.
- It helps us in choosing a inferior system not deficient by much.
- **Difference**
- If system design i^i is the best, then $\theta_i - \max_{j \neq i} \theta_j$ is equal to the difference in performance between the best and the second best.
- If system design i^i is not the best then, $\theta_i - \max_{j \neq i} \theta_j$ is equal to the difference between system i^i and the best.
- i^{*i*} denote index of the best system.
- Smaller the difference $\theta_i - \max_{j \neq i} \theta_j$ is, the more certain we find the best system.
- If the difference between system i^{*i*} and the other is significant, then we try to achieve i^{*i*} with high probability.
- The best system probability should be at least : $1 - \alpha$ whenever $\theta_{i^*} - \max_{j \neq i^*} \theta_j \geq \epsilon$
- We select best of one of near best if multiple system are within ε$^{\varepsilon}$ of the best.
- **Achievement**
- This procedure achieves the desired probability and also forms 1001-a%$^{100(1-a)\%}$ confidence intervals for $\theta_i - \max_{j \neq i} \theta_j$

Summary

- Here, we understood about Bonferroni Approach to Selecting the Best.
- Credits, Jerry Banks book.

Module No.217:

Overview

- Here, we will discuss about "Bonferroni Approach to Screening".

Bonferroni Approach to Screening

- When there exist multiple systems and two stage procedure is not possible.
- Then, divide set of systems into the best.

- This is achieved by using screening or subset selection procedure.
- Independent sampling with unequal variances
- This promises that stayed subset contains best system.
- With probability ≥1-a $\geq 1-a$.
- When the data is distributed normally, by independent sampling or CRN is used.
- This subset may contain all K systems, only one or some.
- It depends on the number of replications and sample variances.
- Procedure
- Specify correct probability selection and sample size.
- Make R replications of the system.
- Calculate the sample and sample variance of the difference.
- If bigger is better then retain selected subset system.
- If smaller is better then retain selected subset system.
- All design not retained can be eliminated from further considerations.

Summary

- Here, We understood about Bonferroni Approach to Screening.
- Credits: Jerry banks book.

Module No.218:

Overview

- Here, we will understand About “Metamodeling”.

Metamodeling

- Metamodeling is the process of generating metamodels.
- Where metamodel refers to model of a model.
- There is a simulation output variable Y.
- It is related to k independent variables.
- Dependent variable Y is random variable.
- Independent variable is design variable.
- Relation between independent and dependent variable is represented by simulation model.
- Metamodel simplifies this relationship by simpler mathematical function.
- This relationship is represented by as function Y=f(x) $Y = f(x)$.
- Most of the time relationship is unknown.
- Function is created using unknown parameter.
- Regression analysis estimates this parameter.

Summary

- We understood about Metamodeling.
- Credits, Jerry Banks book.

Module No.219:

Overview

- Here, we will understand about the “Simple Linear Regression”.

Simple Linear Regression

- The relationship is between independent variable x and a dependent variable y.
- Linear relationship.
- Value of y for given x is:

$$E(Y \mid x) = \beta_0 + \beta_1 x$$

- β0β_0 intercept on y axis.
- β1β_1 is the slope.
- Y can be described as:

$$Y = \beta_0 + \beta_1 x + \epsilon$$

- εε is random error with mean zero.
- This model is commonly called a simple linear regression model.
- nn pair of observations y1,x1,y2,x2...yn,xn$(y_1, x_1), (y_2, x_2) \dots (y_n, x_n)$
- β0β_0 and β1β_1 estimates the observation.
- Sum of squares of the deviation and regression line is minimized.
- The individual observations:

$$Y_i = \beta_0 + \beta_1 x_i + \epsilon_i, \; i = 1, 2, \dots, n$$

- ε1,ε2...$\varepsilon_1, \varepsilon_2 \cdots$ are uncorrelated random variables.
- εiε_i is given by:

$$\epsilon_i = Y_i - \beta_0 - \beta_1 x_i$$

- It shows how εiε_i is related to xi,Yix_i, Y_i and EYixi$E(Y_i | x_i)$.
- The sum of squares of the deviation is :

$$L = \sum_{i=1}^{n} \epsilon_i^2 = \sum_{i=1}^{n} (Y_i - \beta_0 - \beta_1 x_i)^2$$

- Where L is called least-squares function.
- Rewriting YiY_i as:

$$Y_i = \beta_0' + \beta_1(x_i - \bar{x}) + \epsilon_i$$

- Where:

$$\beta_0' = \beta_0 + \beta_1 \bar{x} \text{ and } \bar{x} = \sum_{i=1}^{n} x_i / n.$$

- It is often called as Transformed linear regression model.

$$L = \sum_{i=1}^{n} [Y_i - \beta_0' - \beta_1(x_i - \bar{x})]^2$$

- Taking partial derivatives and setting each to zero.

$$n\hat{\beta}_0' = \sum_{i=1}^{n} Y_i$$

$$\hat{\beta}_1 \sum_{i=1}^{n} (x_i - \bar{x})^2 = \sum_{i=1}^{n} Y_i(x_i - \bar{x})$$

- These are called normal equations.

Summary

- Here we understand about Simple Linear Regression.
- Credits: Uri Wilensky book

Module No.220:

Overview

- Here, we will understand about the “Testing for Significance of Regression”.

Testing for Significance of Regression

- Testing for the significance of regression provides another means for assessing the adequacy of the model.
- Error component εiε_i is normally distributed.
- Testing for Significance of Regression
- The adequacy of the model is checked by residual analysis.
- Developed by variance properties of and β0β_0 and β1β_1 .
- **Alternative hypothesis:**

$$H_0 : \beta_1 = 0$$
$$H_1 : \beta_1 \neq 0$$

- H0H_0 indicates no relationship between x and Y.
- x is of little value in explaining the variability in Y.
- Illustration Best Estimator:

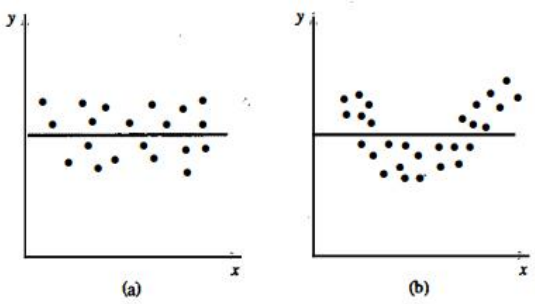


Figure 12.5 Failure to reject $H_0 : \beta_1 \simeq 0$.

- Illustration Non-linear relationship:

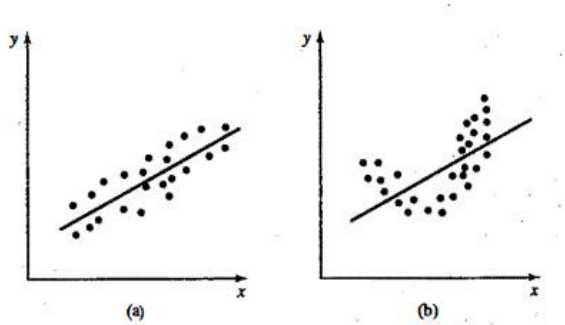


Figure 12.6 $H_0 : \beta_1 = 0$ is rejected.

- Statistic for significance of regression:

$$t_0 = \frac{\hat{\beta}_1}{\sqrt{MS_E / S_{xx}}}$$

- MSεMS_ε is mean squared error given by:

$$\mathrm{MS}_E = \sum_{i=1}^{n} \frac{e_i^2}{n-2}$$

- Testing for Significance of Regression
- The direct method calculates i=1nei2 $\sum_{i=1}^{n} e_i^2$.

$$\sum_{i=1}^{n} e_i^2 = S_{yy} - \hat{\beta}_1 S_{xy}$$

- Where SyyS_{yy} is corrected sum of squares Y given by:

$$S_{yy} = \sum_{i=1}^{n} Y_i^2 - \frac{\left(\sum_{i=1}^{n} Y_i\right)^2}{n}$$

Summary

- Here, we understood about the "Testing for Significance of Regression".
- Credits: Jerry banks book.

Module No.221:

Overview

- Here, we will understand about the "Optimization via Simulation".

Optimization via Simulation

- Optimization is a key tool.
- Well developed algorithms for classes of problems.
- Linear Programming is Most famous.
- Much of optimization deals with performance of design.
- In discrete event simulation, result is a random variable.
- Let x1,x2⋯xn$x_1, x_2 \cdots x_n$ **be** mm controllable design variables.
- Let Y(x1,x2⋯xn)$Y(x_1, x_2 \cdots x_n)$ be observed simulation output performance.
- x1,x2⋯xn$x_1, x_2 \cdots x_n$ might denote number of Automated Guided Vehicles.
- Y(x1,x2⋯xn)$Y(x_1, x_2 \cdots x_n)$ could be total Material Handling System acquisition and operation cost.
- Y(x1,x2⋯xn)$Y(x_1, x_2 \cdots x_n)$ **with respect to** x1,x2⋯xn$x_1, x_2 \cdots x_n$ **.**
- **Y is random variable, so cannot optimize actual value of Y.**
- **Definition of optimization.**

$$\text{maximize or minimize } E(Y(x_1, x_2, \ldots, x_m))$$

- **E(**Y(x1,x2⋯xn))$E(Y(x_1, x_2 \cdots x_n))$ is long-run average cost of operating the MHS .
- x1 x_1 AVGs**,** x2x_2 load per average, and routing algorithm x3x_3 .
- Formulation of performance measure.

$$Y'(x_1, x_2, x_3) = \begin{cases} 1, & \text{if } Y(x_1, x_2, x_3) \le D \\ 0, & \text{otherwise} \end{cases}$$

1. Combine all of the performance measures into a single measure.

-

- The most common being cost.

2. Optimize with respect to one key performance measure.

-
    - Evaluate with secondary performance measures.

3. Optimize with respect to one key performance measure.

-
    - Consider those meeting certain constraints on performance.

- Expected cost and cycle time.

Summary

- Here, we understood about Optimization via Simulation.
- Credits: Jerry banks book.

Module No.222:

Overview

- Here, We will understand "Why is Optimization via Simulation Difficult?"

Difficulties in Optimization via Simulation

- If design variables are numerous optimization can be difficult.
- A diverse collection of design variable types is known as the structure of performance function
- The performance can not be evaluated , but can be estimated.
- No conclusion with estimates.
- It can frustrate optimization algorithms that try to move in improving directions.
- It can be eliminated using many replications.
- Or using long runs.
- At each design point that the performance estimate has essentially no variance.
- It causes other designs to be explored.
- Standard sampling variability force optimizations are as following.
- Guarantee a prespecified probability of correct selection.
- Guarantee asymptotic convergence.
- Optimal for deterministic counterpart.
- Robust heuristics.
- Robust heuristics are the most common algorithms.
- It is found in commercial optimization via simulation software.

Summary

- Here, we understood about Bonferroni Approach to Selecting the Best.
- Credits, Jerry Banks book.

Module No.223:

Overview

- Here, we will discuss about “Using Robust Heuristics”.

Using Robust Heuristics

- It means a procedure that does not depend on strong problem structure.
- Can be applied on decision variables and sampling variability.
- Generic algorithms and tabu search are examples.
- There are k possible solutions to the optimization via simulation problem.
- On each iteration a GA operates on a "population" of p solutions.
- Basic Genetic Algo
- Set the iteration counter j =

  0. p0=x10⋯xp0$p(0) = \{x_1(0) \ldots x_p(0)\}$
- Ruin simulation experiments.
- Select population p from P(j).
- Recombine Solutions via P(j + 1).
- Set j = j + 1 and repeat.
- The G.A can be terminated after some number of iterations.
- At termination X* with smallest Y(x) is choosen as best.
- **Basic Tabu Search**
- Set the iteration counter j = 0. select initial x* in X
- Find solution x’ which minimizes Y(x).
- If Y(x‘) < Y (x*) then x* = x’.
- Update the list of Tabu and repeat from step 2.
- Tabu search can be terminated when some number of iterations has passed without changing x*.
- T.S is fundamentally a discrete-decision-variable optimizator.

Summary

- Here, we have understood concepts about Robust Heuristics
- Credits: Jerry banks book.

Module No.224:

Overview

- Here, we will understand About “An Illustration: Random Search”.

Random Search: An Illustration

- Random search provides guaranteed asymptotic converges in certain conditions.

- Convergence can be slow and memory consumptive.
- random search is not a Robust Heuristic.
- Random search algorithm requires finite number of possible designs.
- Rule out problem with continuous decision variables.
- Algorithms designed for continuous variable problem is there.
- On each iteration, we compare a current solution to a randomly chosen competitor.
- If competitor is better it becomes current good solution.
- **Step 1.**
    - Initialize random variables Ci=0$C(i) = 0$.
    - Initial solution $i0^{i^0}$ and set Ci0=1$C(i^0) = 1$.
    - Ci $C(i)$ count number of times i i is visited.
- **Step 2.**
    - Chose another solution except i i.
    - Such as each solution has equal chance of being selected.
- **Step 3.**
    - Run two simultaneous experiments.
    - Use $i0$ i^0 and i^i to obtain Y(i0)$Y(i^0)$ and Y(i)$Y(i)$
    - Set $i0^{i^0} = i^i$
- **Step 4.**
    - Set Ci0=Ci°+1$C(i^0) = C(i^\circ) + 1$
    - If not done go to step 2.
    - If done, Then select xi* xi^* such that C(i*)$C(i^*)$ is largest count.

Summary

- We understood about Random search.
- Credits, Jerry Banks book.

Module No.225:

Overview

- Here, we will see the introduction to performance evaluation.

Introduction to performance evaluation

- Understanding all elements of the computer system.
- All aspects are important while understanding the issues.
- Realizing the requirements.
- Computer system must function correctly.
- Performing the intended function efficiently.

- Describing the computer system's design.
- Construction.
- Fielding.
- Life-cycle maintenance.

Summary

- We have learnt about the introduction to the performance evaluation.
- Credits: Paul Fortier book.

Module No.226:

Overview

- **Here, we will understand about the "Events".**

Events

- Time is an important measure.
- Useful if we can measure under computer evaluation.
- An event describes an entity of interest in our system.
- E.g. Clock cycle.

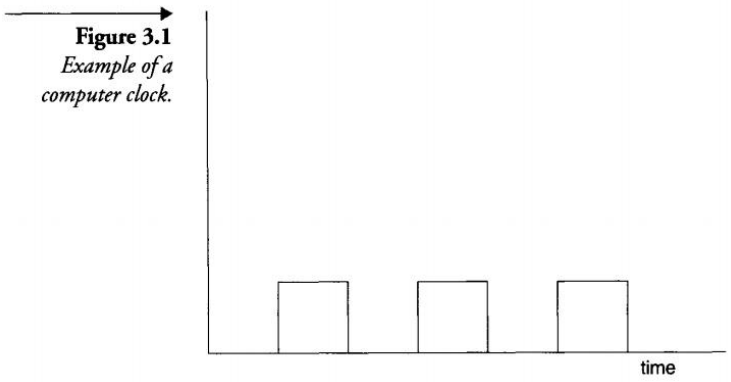
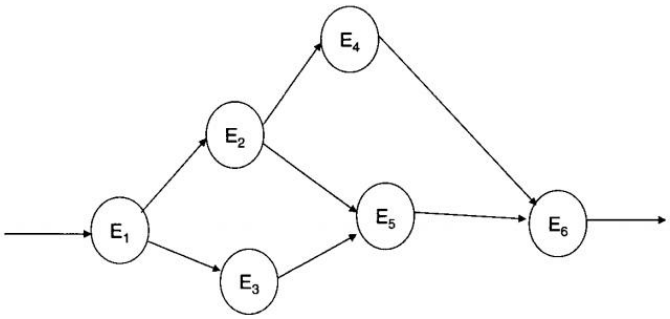


Figure 3.1
Example of a computer clock.

- All events must be controlled.
- Each action is designed to synchronize with other actions.
- Small actions makes larger events.
- System analyst must have understanding of the events.
- And the relationship among different events.



Summary

- Here we understand about Events.
- Credits: Uri Wilensky book

Module No.227:

Overview

- Here, we will learn about the measurements (sampling).

Measurements (sampling)

- Events represent all of the real actions that occur in the computer system.
- Must know all the possible conditions.
- Measurements (sampling)
- State is an important concept in any computer system.

$$S = \{E_1(\text{value}), E_2(\text{value}), E_3(\text{value}), \ldots, E_n(\text{value})\}$$

- Each event must have valid components.
- Primary types of measures are:
- Count a number of items over a given time period.
- Measures all state variables.
- Measures the fraction of time the system is within a state.
- Valid state must reach.
- Many ways are there to measure system events and decision depends on many factors.
- In hardware monitoring system analyst has ability to add instrumentation.
- Can also use integral test hardware.
- The hardware monitoring be designed as an integral component of the system.
- Must determine and define all aspects.
- Software monitoring requires designers to system hardware elements.
- As well as low level software elements.
- Need to have access to low level operating system calls.
- Hybrid monitoring uses the concepts of both software and hardware monitoring.

- Synchronization is required.

Summary

- We have learnt about the Measurements (sampling).
- Credits: Paul Fortier book.

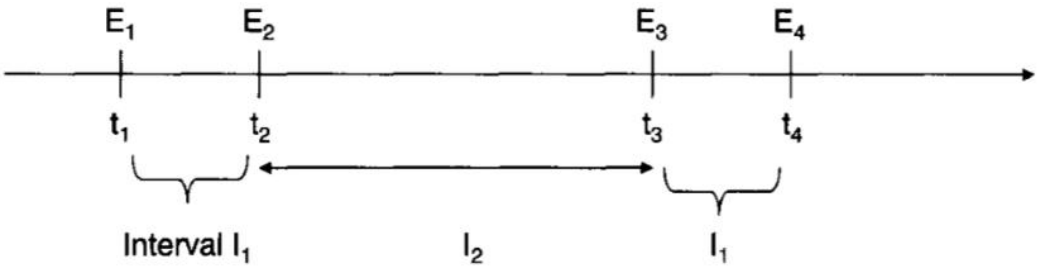
Module No.228:

Overview

- Here, we will learn about the intervals.

Intervals

- An interval represents a period of time.
- System environment is system clocks.
- Instruction execution cycle.
- Intervals are same if they represent the same sequence of events.
- Separate sequence are represented by two intervals then they are unrelated.
- **Intervals**



Summary

- We have learnt about the Intervals.
- Credits: Paul Fortier book.

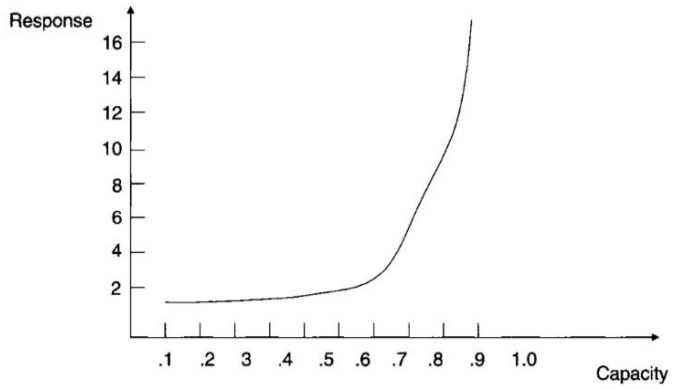
Module No.229:

Overview

- Here, we will learn about the response.

Response

- Represent a measure of period of time a user or application to wait before task assignment.
- Typical measure may pit the response time.
- **Response**



- Load increases above specific capacity the response increases.
- Study the measurable sequences against system load.
- How these loads impact each other.

Summary

- We have learnt about the response.
- Credits: Paul Fortier book.

Module No.230:

Overview

- Here, we will learn about the Independence.

Independence

- Occurrence if one dose not influences the outcome of other.
- Important concept while evaluating systems.
- Two programs that cannot run concurrently must be consider independent.
- Running on the same operating system and hardware.
- Cannot interfere each other then they are separate and unrelated.
- Important in modeling to define all elements and their relationship.

Summary

- We have learnt about the Independence.
- Credits: Paul Fortier book**.**

Module No.231:

Overview

- Here, we will learn about randomness.

Randomness

- Randomness is a property of an event and its reoccurrence.

- Have no pattern.
- Realization of the property of independence.
- Its a mathematical concept.
- External sources can be viewed as random events in systems.
- Important to analyze systems using mathematical concepts.

Summary

- We have learnt about the randomness.
- Credits: Paul Fortier book.

Module No.232:

Overview

- Here, we will learn about the workloads.

Workloads

- Simply we can say load.
- Event or event sequences presented to system to model.
- Represent the number of events offered to system for execution.
- Arranging the requests and perform them by the system.
- The duration could be endless.
- The load is re-entered after prescribed period of time.
- The database community has developed a set of transactional workloads.

Summary

- We have learnt about the workloads.
- Credits: Paul Fortier book.

Module No.233:

Overview

- Here, we will understand about the problems encountered in model development and use.

Problems encountered in model development and use

- Must start with having concept that what we are doing.
- Determining what to do.
- Two main classes performance assessment.
- First take an existing system and design some experiments.
- Second is either analytical modeling or simultaneous.
- Analyst uses following measures:
- Responsiveness and use levels.
- Mission-ability and dependability.
- Productivity and predictability.

- Analyst must avoid the following mistakes:
- Having no goals.
- Setting biased goals.
- Unsystematic approach to development.
- Choosing of incorrect performance metrics.
- Choosing non-stressful workload.
- To solve these problems the analyst must ask some questions to himself/herself.

Summary

- We have learnt about the problems encountered in model development and use.
- Credits: Paul Fortier book.

Module No.234:

Overview

- Here, we will understand about how to conduct a case study

A Case Study

- First Step: Define the System that we wish to study
- E.g. Remote pipes vs. Remote Procedures
- Focus on Services – small and large data transfers
- Select Metrics
- Rate of access
- Time for Performance
- Resource requirements per service
- Evaluate the developed model and start over again
- Trial and error
- Conducting our own: Modeling a ? ? ?
- A Bus system
- A railway terminal
- A hospital
- A human cell
- Human brain
- Ecosystem

Summary

- We have learnt about conducting a case study.
- Credits: Paul Fortier book.

Module No.235:

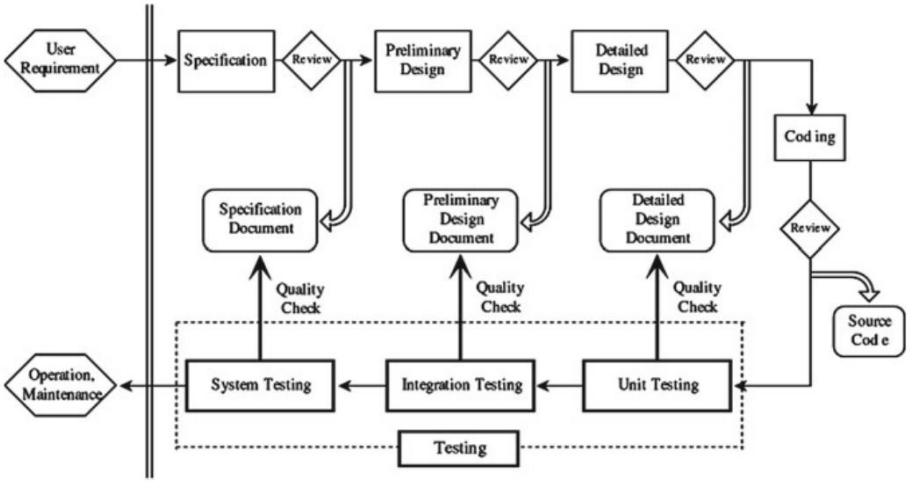
Overview

- Here, we will understand about software reliability

Introduction to Software Reliability

- Software fails on a regular basis
- This is a serious issue
- Especially in case of critical systems – nuclear reactors, patient care etc.
- Problems with X-Ray machine
- Problems with SCUD Missiles
- Problems with rocket launch
- Etc.

Introduction to Software Reliability



Summary

- We have learnt about software reliability.
- Credits: Yamada book.

Module No.236:

Overview

- Here, we will understand about definitions about reliability

Definitions and Software Reliability Model

- Generally, a software failure caused by software faults latent in the system cannot occur except on a specific occasion when a set of specific data is put into the system under a specific condition, i.e. the program path including software faults is executed.
- A software system is a product which consists of the programs and documents produced through a software development process
- Software failure is defined as an unacceptable departure of program operation from the program requirements.

- The cause of software failure is called a software fault.
- Then, software fault is defined as a defect in the program which causes a software failure.
- The software fault is usually called a software bug.
- Software error is defined as human action that results in the software system containing a software fault
- **Definitions and Software Reliability Model**

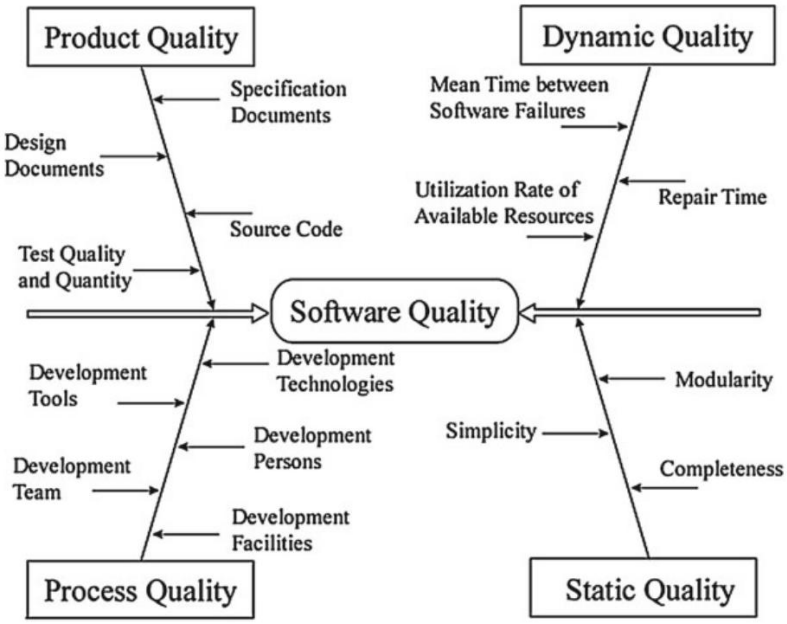
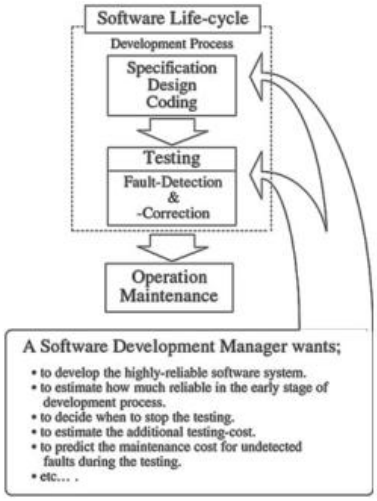


Fig. 1.2 Elements of software quality based on a cause-and-effect diagram

- **Definitions and Software Reliability Model**

Fig. 1.3 Aim of software quality/reliability measurement and assessment



- **Definitions and Software Reliability Model**

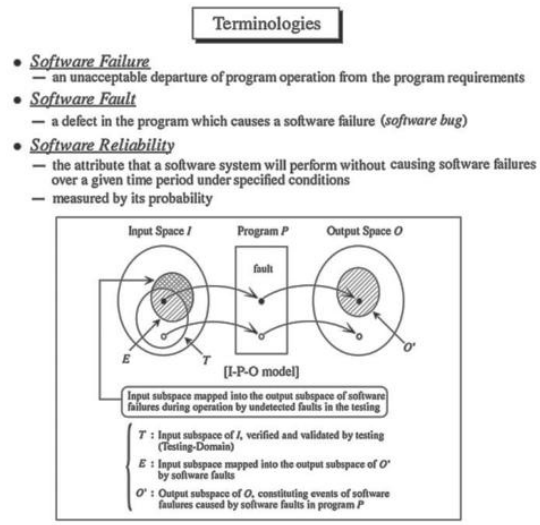


Fig. 1.4 An input-program-output model for software behavior

- **Definitions and Software Reliability Model**

Software Reliability	Hardware Reliability
(1) Software failures can be due to no wearout phenomenon.	(1) Hardware failures can be due to wear.
(2) Software reliability is, inherently, determined during the earlier phase of the development process, i.e. specification analysis and design phases.	(2) Hardware reliability is affected by deficiencies injected during all phases of the development, operation, and maintenance.
(3) Software reliability can not be improved by redundancy with identical versions.	(3) Hardware reliability can be improved by redundancy with identical units.
(4) A verification method of software reliability has not been established.	(4) A testing method of hardware reliability is established and standardized.
(5) A maintenance technology is not established since the market of software products is rather recent.	(5) A maintenance technology is advanced since the market of hardware products is established and the user environment is seized.

Fig. 1.5 Comparison between the characteristics of software reliability and hardware reliability

Summary

- We have learnt about definitions related to reliability.
- Credits: Yamada book.

Module No.237:

Overview

- Here, we will understand about software reliability growth modeling

Software Reliability Growth Modeling

- Generally, a mathematical model based on stochastic and statistical theories is useful to describe the software fault-detection phenomena or the software failure-occurrence phenomena and estimate the software reliability quantitatively.
- **Software Reliability Growth Modeling**

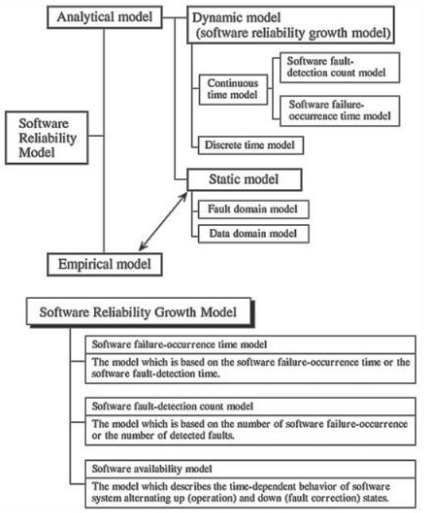


Fig. 1.6 Hierarchical classification of software reliability models

- **Software Reliability Growth Modeling**

$N(t)$ the cumulative number of software faults (or the cumulative number of observed software failures) detected up to time t,

S_i the ith software-failure occurrence time ($i = 1, 2, \ldots; S_0 = 0$),

X_i the time-interval between $(i-1)$-st and ith software failures ($i = 1, 2, \ldots; X_0 = 0$).

- **Software Reliability Growth Modeling**

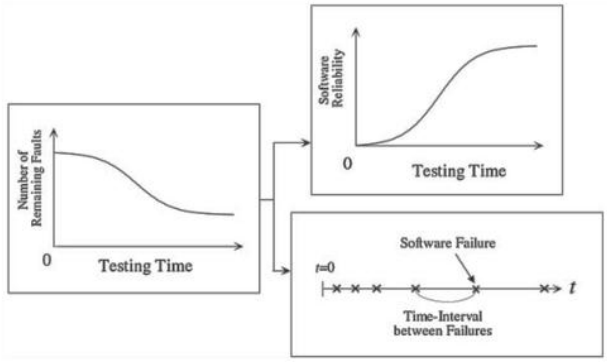


Fig. 1.7 Software reliability growth

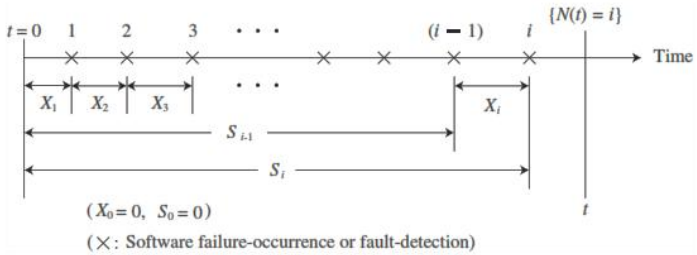


Fig. 1.8 The stochastic quantities related to a software fault-detection phenomenon or a software failure-occurrence phenomenon

Summary

- We have learnt about software reliability growth modeling.
- Credits: Yamada book.

Module No.238:

Overview

- Here, we will understand about imperfect debugging modeling

Imperfect Debugging Modeling

- Most software reliability growth models proposed so far are based on the assumption of perfect debugging
- i.e. All faults detected during the testing and operation phases are corrected and removed perfectly.
- However, debugging actions in real testing and operation environment are not always performed perfectly.
- For example, Typing errors invalidate the fault-correction activity or fault-removal is not carried out precisely due to incorrect analysis of test results
- As a result it is preferable to develop an incorrect debugging model.
- This will result in a better estimation of reliability

Summary

- We have learnt about imperfect debugging modeling.
- Credits: Yamada book.

Module No.239:

Overview

- Here, we will understand about imperfect debugging model with perfect correction rate

Imperfect Debugging Model with Perfect Correction

- To model an imperfect debugging environment, the following assumptions are made:

1. Each fault which causes a software failure is corrected perfectly with probability $p(0 \leq p \leq 1)$. It is not corrected with probability $q(= 1 - p)$. We call p the perfect debugging rate or the perfect correction rate.
2. The hazard rate is given by (1.5) and decreases geometrically each time a detected fault is corrected (see Fig. 1.14).
3. The probability that two or more software failures occur simultaneously is negligible.
4. No new faults are introduced during the debugging. At most one fault is removed when it is corrected, and the correction time is not considered.

- **Imperfect Debugging Model with Perfect Correction**

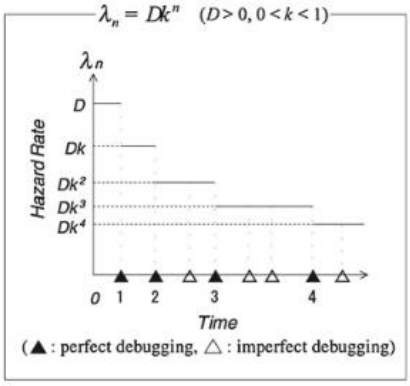


Fig. 1.14 Behavior of hazard rate

- **Imperfect Debugging Model with Perfect Correction**

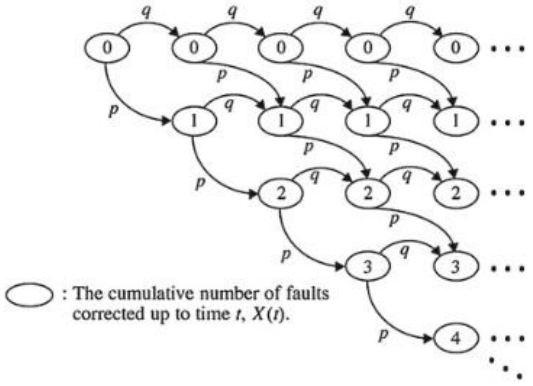


Fig. 1.15 A diagrammatic representation of transitions between states of $X(t)$

- Let X (t) be a random variable representing the cumulative number of faults corrected up to the testing time t. Then, X (t) forms a Markov process.
- That is, from assumption 1, when i faults have been corrected by arbitrary testing time t,

$$X(t) = \begin{cases} i, & \text{with probability } q, \\ i+1, & \text{with probability } p, \end{cases} \qquad (1.14)$$

Summary

- We have learnt about imperfect debugging modeling with correction.
- Credits: Yamada book.

Module No.240:

Overview

- Here, we will understand about imperfect debugging modeling with introduced faults

Imperfect Debugging with Introduced Faults

- Besides the imperfect debugging factor above in fault-correction activities, we consider the possibility of introducing new faults in the debugging process.
- It is assumed that two kinds of software failures exist in the dynamic environment
- (F1) software failures caused by faults originally latent in the software system prior to the testing (which are called inherent faults)
- (F2) software failures caused by faults introduced during the software operation owing to imperfect debugging.
- In addition, it is assumed that one software failure is caused by one fault
- And that it is impossible to discriminate whether the fault that caused the software failure that has occurred is F1 or F2.
- As to the software failure-occurrence rate due to F1, the inherent faults are detected with the progress of the operation time.
- In order to consider two kinds of time dependencies on the decreases of F1, let
- ai(t) = (1,2) denote software failure occurrence rate for F1
- For F2, it is constant (Lambda > 0)

Summary

- We have learnt about imperfect debugging modeling with introduced faults.
- Credits: Yamada book.

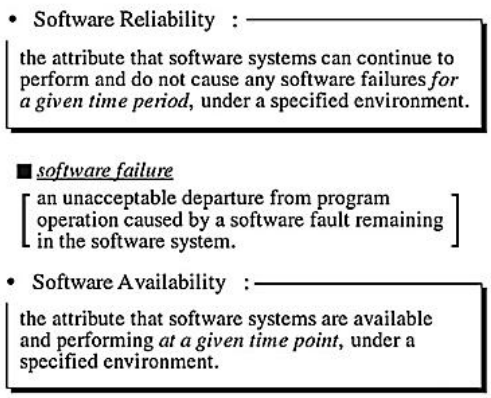
Module No.241:

Overview

- **Here, we will understand about software availability modeling**

Software Availability Modeling

- Recently, software performance measures such as the possible utilization factors have begun to be interesting for metrics as well as the hardware products.
- That is, it is very important to measure and assess software availability,
- It is defined as the probability that the software system is performing successfully, according to the specification, at a specified time point
- The actual operational environment needs to be more clearly reflected in software availability modeling, since software availability is a customer-oriented metrics.
- **Software Availability Modeling**



- Software fails on a regular basis
- This is a serious issue
- Especially in case of critical systems – nuclear reactors, patient care etc.

Summary

- We have learnt about conducting a case study.
- Credits: Paul Fortier book.

Module No.242:

Overview

- Here, we will understand about model description for software availability modeling

Model Description

- **Assumptions**

1. The software system is unavailable and starts to be restored as soon as a software failure occurs, and the system cannot operate until the restoration action is complete

2. The restoration action implies debugging activity, which is performed perfectly with probability a

- $(0 < a \leq 1)$ and imperfectly with probability $b(= 1 - a)$.
- a is the perfect debugging rate
- One fault is corrected and removed from the software system when the debugging activity is perfect.

3. When n faults have been corrected, the time to the next software failure-occurrence and the restoration time follow exponential distributions

4. The probability that two or more software failures will occur simultaneously is negligible

Summary

- We have learnt about model description.
- Credits: Yamada book.

Module No.243:

Overview

- **Here, we will understand about software availability measures**

Model Description

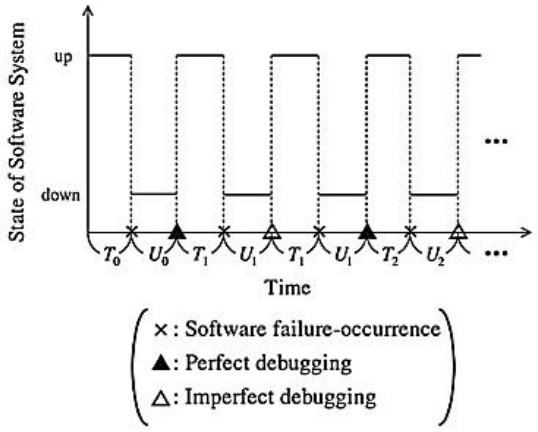


Fig. 1.17 Sample behavior of the software system alternating between up and down states

- **Software Availability Measures**

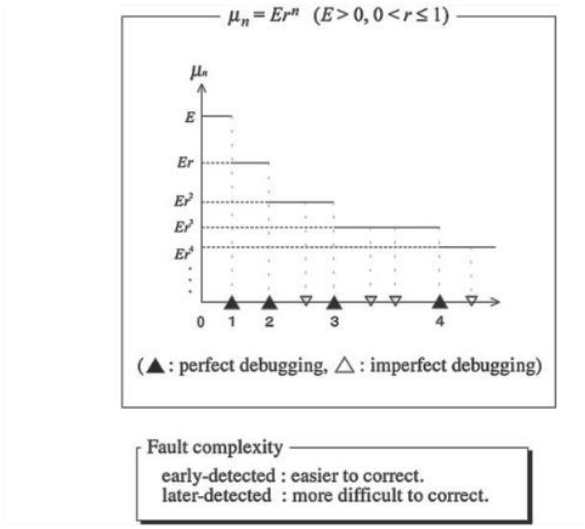
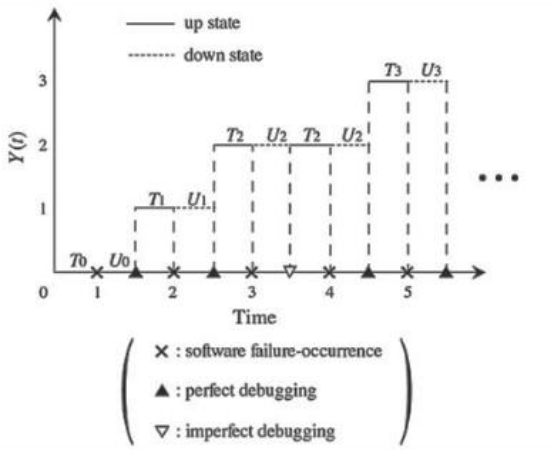


Fig. 1.18 Behavior of restoration rate

- **Software Availability Measures**



- **Software Availability Measures**

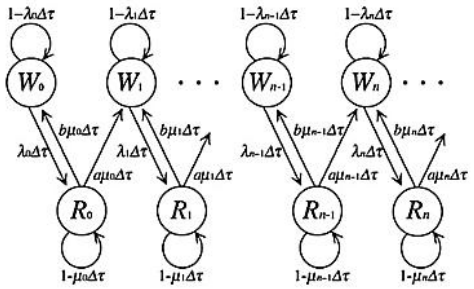


Fig. 1.20 A state transition diagram for software availability modeling

- **Software Availability Measures**

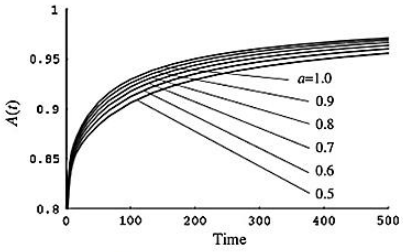


Fig. 1.21 Dependence of perfect debugging rate a on $A(t)$

- **Software Availability Measures**

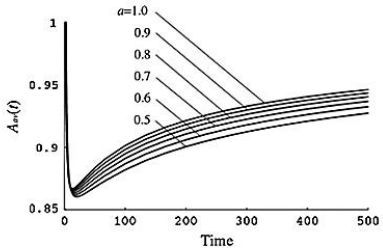


Fig. 1.22 Dependence of perfect debugging rate a on $A_{av}(t)$

Summary

- We have learnt about software availability measures.
- Credits: Yamada book.

Module No.244:

Overview

- Here, we will understand about software reliability assessment

Application of Software Reliability Assessment

- It is very important to apply the results of software reliability assessment to management problems with software projects for attaining higher productivity and quality.
- We discuss three software management problems as application technologies of soft- ware reliability models.
- Optimal Software Release Problem
- Statistical Software Testing-Progress Control
- Optimal Testing-Effort Allocation Problem

Summary

- We have learnt about conducting a case study.
- Credits: Paul Fortier book.

Module No.245:

Overview

- Here, we will understand about optimal software release issues

Optimal Software Release Problem

- Recently, it is becoming increasingly difficult for the developers to produce highly reliable software systems efficiently.
- Thus, it has been necessary to control a software development process in terms of quality, cost, and release time.
- In the last phase of the software development process, testing is carried out to detect and fix software faults, introduced by human work, prior to its release for the operational use.
- The software faults that cannot be detected and fixed remain in the released software system after the testing phase.
- Thus, if a software failure occurs during the operational phase, then a computer system stops working and it may cause serious damage.
- If the duration of software testing is long, we can remove many software faults in the system and its reliability increases. However, this increases the testing cost and delays software delivery.
- In contrast, if the length of software testing is short, a software system with low reliability is delivered and it includes many software faults which have not been removed in the testing phase.

- Thus, the maintenance cost during the operation phase increases.
- It is therefore very important in terms of software management that we find the optimal length of software testing, which is called an optimal software release problem
- These decision problems have been studied in the last decade by many researchers.
- In optimal software release problems we consider both a present value and a warranty period (in the operational phase)

Summary

- We have learnt about optimal software release.
- Credits: Yamada book.

Module No.246:

Overview

- Here, we will understand about statistical software testing-progress control

Statistical Software Testing-Progress Control

- As well as quality/reliability assessment, software-testing managers should assess the degree of testing-progress.
- We can construct a statistical method for software testing-progress control based on a control chart method
- This method is based on several instantaneous fault-detection rates derived from software
- **Statistical Software Testing-Progress Control**

The procedure of testing-progress control is shown as follows:

Step 1: An appropriate model is selected to apply and the model parameters are estimated by the method of least-squares.
Step 2: To certify goodness-of-fit of the estimated regression equation for the observed data, we use the F-test.
Step 3: Based on the result of the F-test, the central line and upper and lower control limits of the control chart are calculated. The control chart is drawn.
Step 4: The observed data are plotted on the control chart and the stability of the testing-progress is judged.

- **Statistical Software Testing-Progress Control**

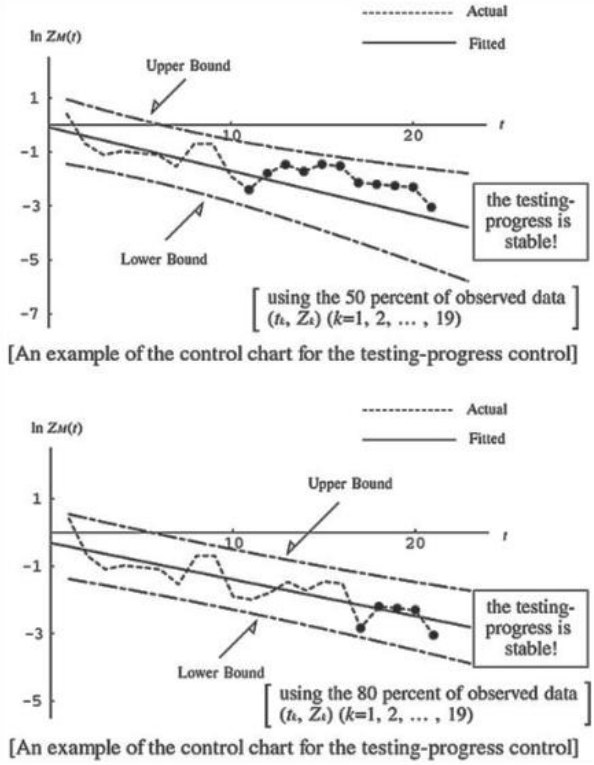


Fig. 1.25 Examples of the control chart for the testing-progress control

Summary

- We have learnt about statistical software testing-progress control.
- Credits: Yamada book.

Module No.247:

Overview

- **Here, we will understand about Optimal Testing-Effort Allocation**

Optimal Testing-Effort Allocation

- We discuss a management problem to achieve a reliable software system efficiently during module testing in the software development process by applying a testing-effort-dependent software reliability growth model
- We take account of the relationship between the testing-effort spent during the module testing and the detected software faults where the testing-effort is defined as resource expenditures spent on software testing, e.g. manpower, CPU hours, and executed test cases.
- The software development manager has to decide how to use the specified testing-effort effectively in order to maximize the software quality and reliability.
- That is, to develop a quality and reliable software system, it is very important for the manager to allocate the specified amount of testing-effort expenditure for each software module under some constraints.

- We can observe the software reliability growth in the module testing in terms of a time-dependent behavior of the cumulative number of faults detected during the testing stage.
- Based on the testing-effort dependent software reliability growth model, we con- sider the following testing-effort allocation problem:
- Based on the testing-effort dependent software reliability growth model, we con- sider the following testing-effort allocation problem:

1. The software system is composed of M independent modules. The number of software faults remaining in each module can be estimated by the model.
2. The total amount of testing-effort expenditure for module testing is specified.
3. The manager has to allocate the specified total testing-effort expenditure to each software module so that the number of software faults remaining in the system may be minimized.

Summary

- We have learnt about Optimal Testing-Effort Allocation.
- Credits: Yamada book.

Module No.248:

Overview

- **Here, we will understand about the "formal modeling".**

Formal modeling

- Petri nets PNs provide graphical tool and notation method.
- Include arrival rate and service rate.
- Individual state information.
- Contention and concurrency.
- Petri Nets have place in computer systems performance assessment.
- Between analytical queuing theory and computer simulation.
- System state more complete than analytical models.
- But not as simulations.
- Have fundamental theory but act like simulations.
- Single entities, movement and affect effect of the state of the entire system.
- Petri Nets was introduced in 1966.
- It describes concurrent systems.
- Followed by continuous improvements.
- Timing of transitions.
- Priority of transitions.
- Types to Token.
- Colors depicting.
- Analysis tools and modelling aid.

Summary

- Here, we understood about the “Testing for Significance of Regression”.
- Credits: Paul Fortier book.

Module No.249:

Overview

- Here, we will understand about the “Basic notation”?

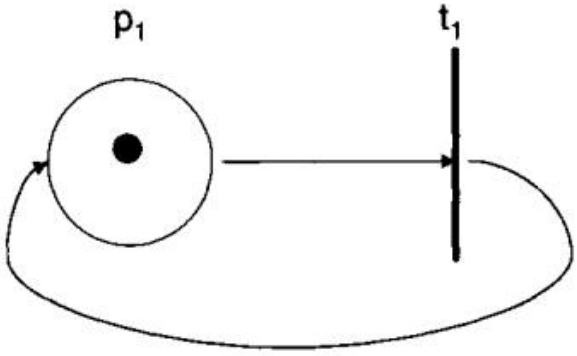
Basic notation

- Petri Nets provides abstraction and information flow.
- Using four fundamental components.
- Places as Circles.
- Transition as bars.
- Arc as line segments.
- Token as dots.
- Places represents useful system components and their state e.g. disk drive.
- Transitions represents events.
- Action of reading an item from disk drive.
- Arcs represents relationship between transitions and places.
- Proved path for activation of transition.
- Tokens define state of Petri net.

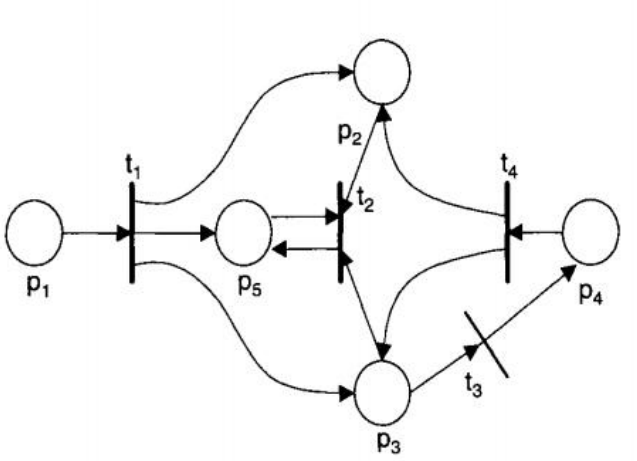
Forever Cycling Petri Net

The place is connected to the transition by an arc, and the transition is likewise connected to the place by a second arc.

The former arc is an input arc, while the latter arc is an output arc.
The placement of a token represents the active marking of the Petri net state
The Petri net shown represents a net that will continue to cycle forever.



Petri Net Graphical Representation



Places(P), Transitions(T) Arc(input I, Output O), and Tokens for net (MP).

Five tuple M=(P,T,I,O,MP).

P represent set of Places P$\{p_1, p_{2,...} p_n\}$.

P represent set of Transitions P$\{t_1, t_{2,...} t_n\}$.

I as input functions I = $\{I_{t1},t_{t2...} I_{tn}\}$ and as output functions O = $\{O_{t1},O_{t2...} O_{tn}\}$.

MP represents marking of places with tokens.

MP_O is represented as ordered tuple of magnitude where n represents number of places in petri net.

Summary

- Here, we understood about Basic notation.
- Credits: Paul Frontier book.

Module No.250:

Overview

- Here, we will understand about the “Basic notation”.

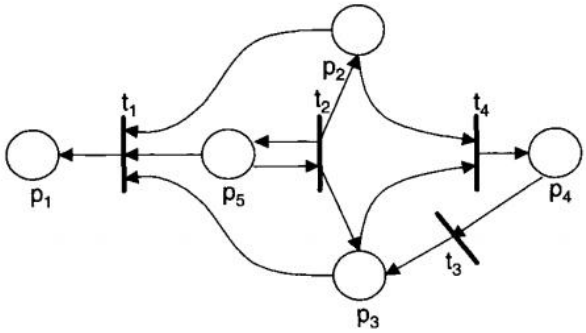
Basic notation-II[II]

Petri Net Dual

- Petri net model also has dual.
- In dual of petri, transitions are changed to places and places to transitions.
- The input becomes output function and output becomes output function.

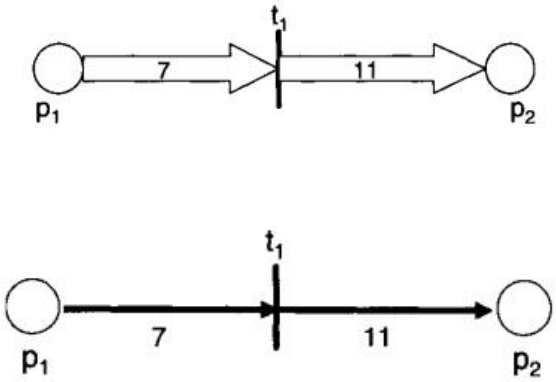
Petri Net Inverse

- A petri net model is also defined for inverse.
- Places and transitions are kept same.
- Whereas, input functions switches to output functions.
- **Petri Net Inverse Representation**



- **Petri Net Multigraphs**
- A petri net model is also defined for Multigraphs.
- It implies there could be several arcs between a single place and a transition.
- Can be represented as:
    - Multiple arc as thick arc with numbers inside.
    - A bold arc with number attached to it.

- **Basic Petri Net Multigraphs Representation**



Petri Net State

- A petri net also has a state defined by the cardinality if tokens.
- Represented as function, μ^{μ} .

$$\mu : p \rightarrow N$$

- μ^{μ} can be defined as vectors.

$$\mu = (\mu_1, \mu_2, \mu_3, \ldots, \mu_n),$$

Petri Net Multigraphs

- Where.

$$n = |P| \text{ and each } \mu_i \in N, i = 0, \ldots, n$$

- Place marking function and vector relation:

$$\mu(p_i) = \mu_i.$$

- Petri net structure:

$$M = (P, T, I, O, \mu_t)$$

- It represents state of petri net at time t.

Summary

- Here, we understood about the “Basic Notaions”.
- Credits: Paul Fortier book.

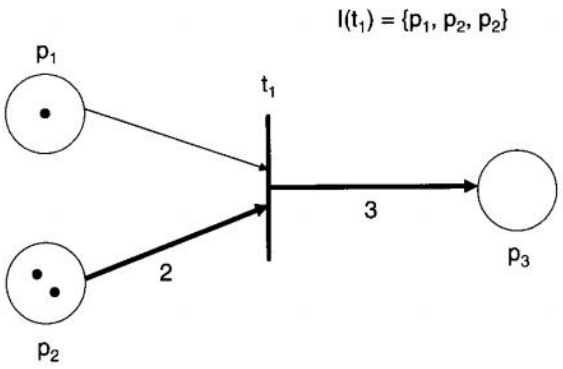
Module No.251:

Overview

- Here, we will understand about the “Classical Petri Nets”?

Classical Petri Nets

- By using notations Petri nets can be used in modelling.
- A new petri net state transition notation is required.
- These transitions are helpful in movements.
- The exact firing moment can be captured as a clock signal.
- During Clock cycle, all gates start signal execution.
- In petri net all transitions fire once during this cycle.

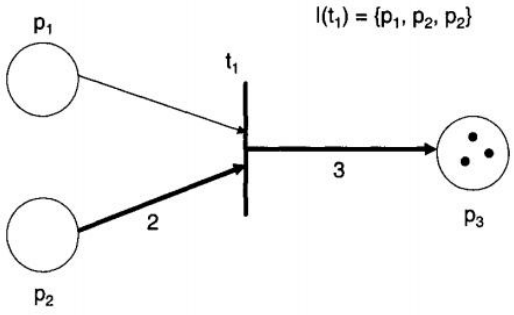


The Petri net shown has the marking g0 = (1,2, 0), input function I(tl) - {PI,P2,P3}, and output function O(t1) = {P3,P3,P3}" .

Given this initial marking and the defined input and output functions, transition t I is enabled, since it requires one token contributed by P1 and two tokens contributed by P2 to have all of its input functions satisfied with tokens available in the named places and in the required quantities.

transition is enabled at the beginning of a Petri net's firing cycle, it will fire the transition, causing the movement of the number of tokens from its input places to its output places The result of this firing will be a new Petri net state, B1. For example, in given the initial state B0, transition t 1 will fire at the beginning of the firing cycle, removing one token from place Pl and two tokens from
place P2 and placing these three tokens into place P3"

Forever Cycling Petri Net



The resulting new Petri net state is represented by the marking B1 -- (0,0,3), and is depicted in the figure. The firing action is considered an atomic action, in that it appears as if all of the tokens are removed from the input places and deposited into the output places instantaneously.
The firing of the Petri net provides the movement from one state to a new state. The new state is the only state reachable in a single total transition firing of all the enabled transitions. The collection of all possible states that can be represented by this Petri net and its initial markings is called state-space.
This implies that a Petri net can only fire enabled transitions, and, after they fire, they cannot fire any newly enabled transitions until the next cycle.

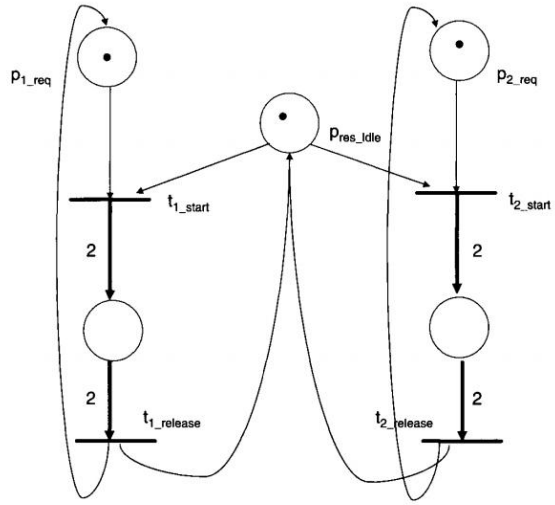
Petri Net Set Notation Representation

Petri nets enables modeling not available through queuing theory.

- Synchronization.
- Conflict.
- Concurrency.

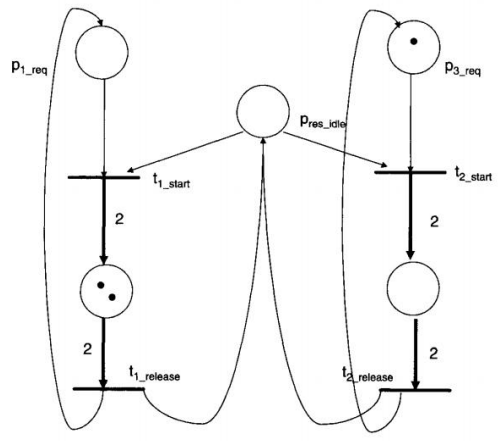
These are not easily defined and modeled by queuing theory.

Resource Sharing



We model two user processes requesting a specific resource (P_{res_idle}). If the resource is idle, there is a token present in the (P_{res_idle}) place. If a process wanting this resource has a token in its place (e.g., P_{p1_req}), and the resource is idle (a token in P_{res_idle}), then the transition (e.g., t_{1_start}) is enabled and can fire on the next cycle. When the transition fires, the resource now
becomes unavailable (it is busy), and the second process must wait until the resource is released.

Allocated Resources



Summary

- Here, we understood about Classical Petri Nets.
- Credits: Paul Frontier book.

Module No.252:

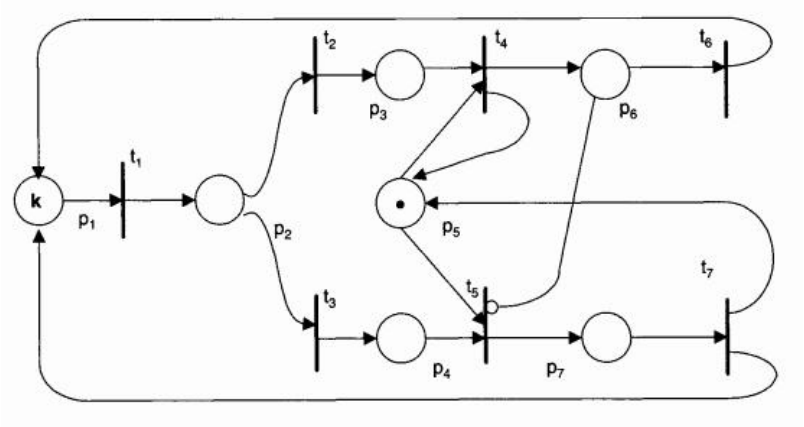
Overview

- Here, We will understand "Classical Petri Nets".

Classical Petri Nets-II^{II}

Classical Petri Nets

- A Petri net's state, is said to be reachable from some other state, u'u' .
- There exists some finite number of firings of the Petri net beginning at state uu .



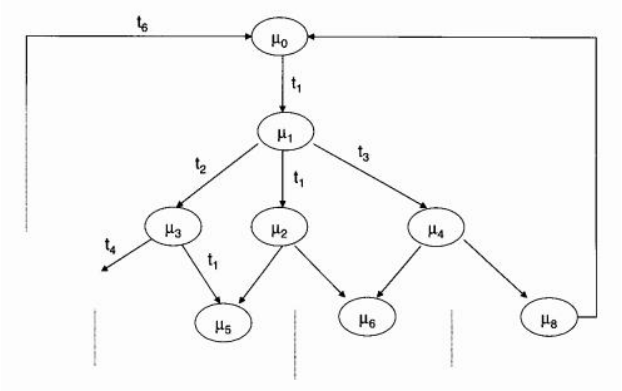
The reachability set for a Petri net with an initial marking, μ0μ_0 , is denoted RS(μ0μ_0) and is defined as the smallest set of markings, so that:
μ0μ_0 εε RS (μ0μ_0) and
μ0μ_0 RS (μ0μ_0)

μ2ε RS$\mu_2 \varepsilon RS$ (N0N_0)

To determine the reachability set, we must begin from the initial state, μ0μ_0 , and incrementally define each step possible emanating from this initial state and all states derivable from this state .

The reachability set contains no information about which transitions fired to reach the state markings. Such information can be found in a reachability graph shown next.

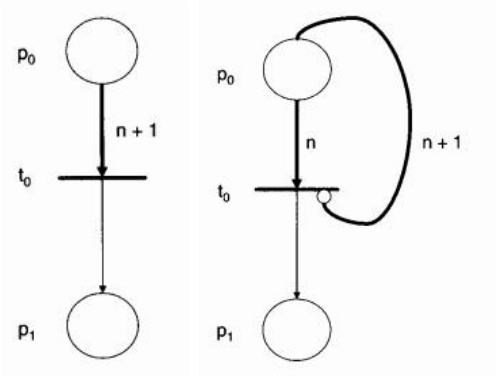
- **Reachability Graph**



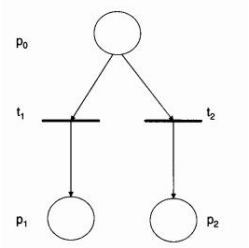
In this graph each node represents a state of the Petri net and each arc represents a direct transition, which is possible from one end of the directed arc to the other end due to a single transitions firing. For example, you can see that if we fire transition t1 from μ0μ_0 we can get to a new state, μ1μ_1, where a token has moved from place 1 to place 2.

It is often desirable to model logical conditions. For example, to only fire a transition when there are more than n tokens in a place, we simply need to include an arc with arity n + 1. Since the basic condition on a transition firing is that its connected input places meet the conditions of the transitions input function, by including n + 1 redundant set items for the input place we can accomplish what we need shown in next figured

- **Component Test**



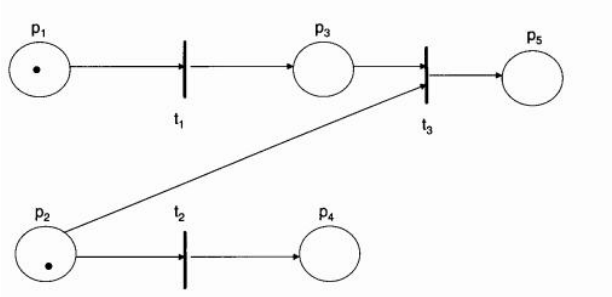
An important property required when modeling computer systems is that of conflict shown next.



In this example, when there is a token in place P0, both transitions t1 and t2 are enabled.

However, only one of them may fire, since there is only one token available. As soon as one fires, say t1, it removes the token from place P0 and transfers it to place P1. As soon as transition

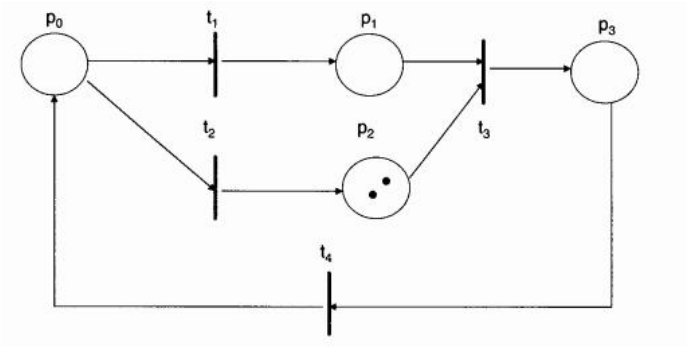
t1 removes the token, transition t2 is no longer enabled. If there were two tokens in place P0, then both of the transitions would be enabled and could fire during this firing cycle.



Very often a computer system can have both conflict and concurrency occur at the same time as shown.

we have an initial marking, u = (1,1,0,0,0), which results in transitions t1 and t2 being enabled, the condition of concurrent transitions. If t1 fires first, then we now have two transitions enabled, t2 and t3. This then depicts conflict, since the token from P2 can only satisfy one of the necessary conditions for the two transitions it is enabling at this point in time.

A Petri net can have a variety of other qualities as well.



A Petri net is deadlocked if there are no transitions in the net that are enabled. In the example shown, the net has an initial marking, u0 = (0,0,2,0). This marking results in no transitions being enabled and no hope after an infinite amount of time of becoming enabled. Conversely, a Petri net is considered live if there are any transitions enabled.

Summary

- Here, we understood about Classical Petri Nets

- Credits, Paul Frontier Book.

Module No.253:

Overview

- Here, we will learn about the timed petri nets.

Timed Petri nets

- Every action takes some time to complete.
- Time addition provides Petri net modeler another powerful tool.
- Time in Petri net modeling is associated with transitions.

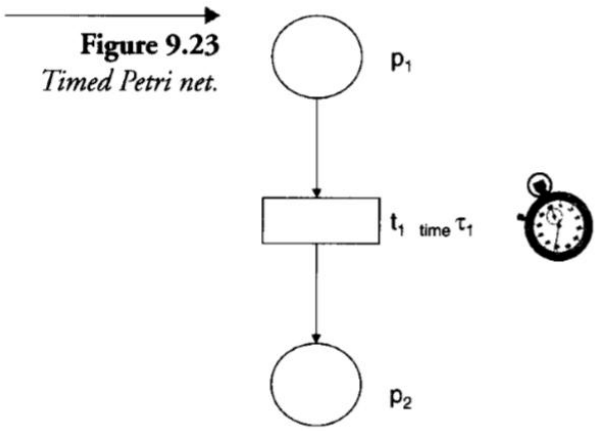


Figure 9.23
Timed Petri net.

Transition t1t_1 = timed transition.

Semantic of the firing is different.

Once transition enabled, its time period clock timer is set.

Transition fires when timer reaches 0.

When token reaches p1p_1 the timer for transition t1t_1 is set to τ1.τ_1.

Token moves to p2p_2 once τ1 τ_1 time is passed.

The decrement of the timer must be constant speed.

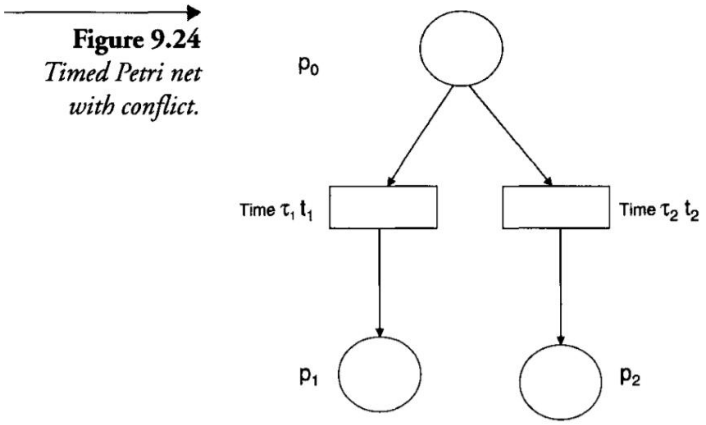


Figure 9.24
Timed Petri net with conflict.

If t1< t2.$t_1 < t_2$.

Two timers would begin when p2p_2 receives a token.

When t1t_1 reached 0 then t2 t_2 becomes disabled.

Reset the timer.

Allowing transition t2 t_2 's timer to maintain the present clock.

Summary

- Here, we have learnt about the two conditions of the timed Petri nets.
- Credits: Paul Fortier book.

Module No.254:

Overview

- Here, we will learn about the timed Petri nets.

Timed Petri nets

- Some Petri net models have proposed using state transition timing graphs.
- Each state μμ is enumerated.
- Time period is set for each state in sequence.

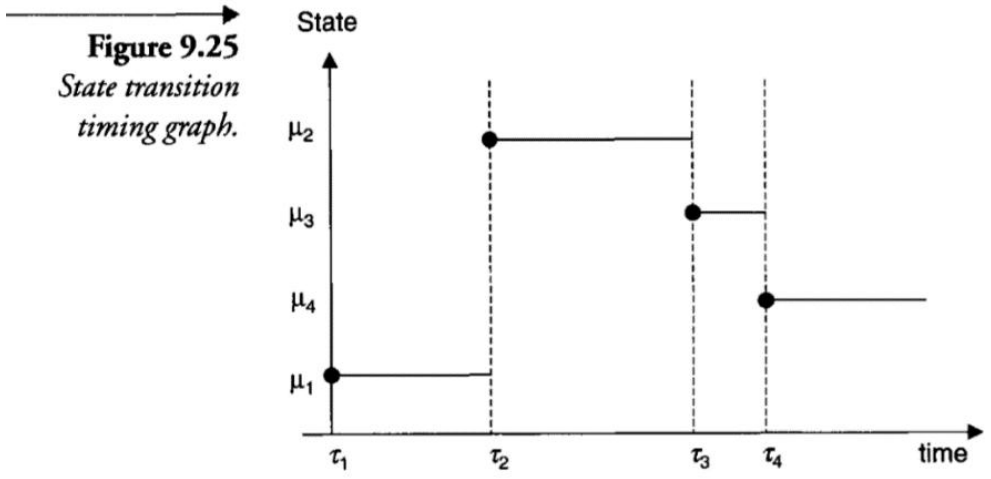


Figure 9.25
State transition timing graph.

- Time units can be represented using time transition sequences.
- [τ1,t2;(τ2$[(\tau_1, t_2); (\tau_2$, t2);...;τj,tj;...]$t_2); \ldots; (\tau_j, t_j); \ldots]$

Time units are require to move from μ1μ_1 to μ2μ_2 .

Transitions Can be also be represented as immediate transitions.

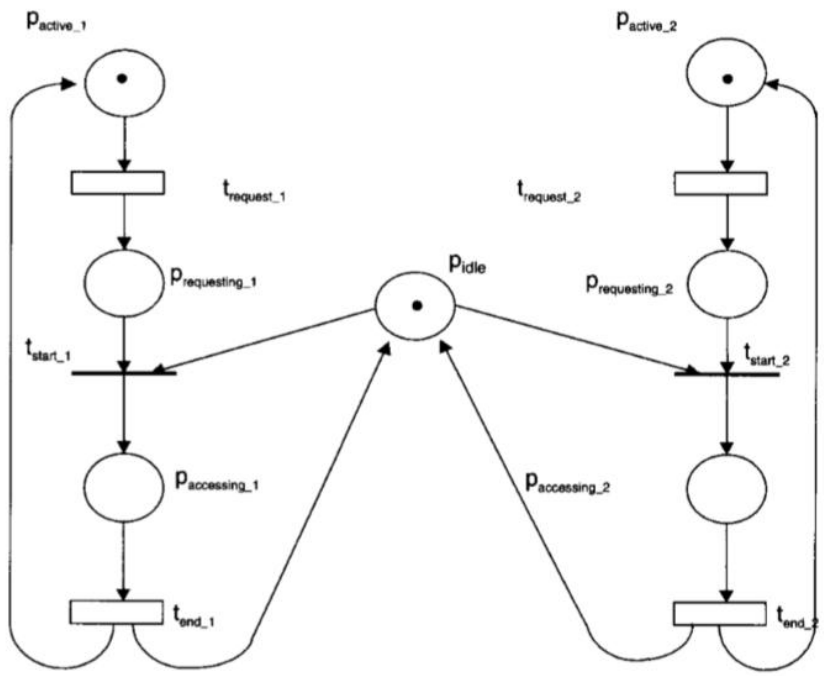


Figure 9.26
Timed Petri net with immediate transitions.

Immediate transitions are used to capture resources.

Acts like a semaphores.

Active process will win it and other processes have to wait until resources become free.

Summary

- We have learnt about the timed Petri nets.
- Credits: Paul Fortier book.

Module No.255:

Overview

- Here, we will learn about the Priority-based Petri nets.

Priority-based Petri nets

- The Petri net is described by nine tuple.
- M=P,T,I,O,H,Π,**PAR**,Pred,μ $M = \begin{pmatrix} P, T, I, O, H, \Pi, \mathbf{PAR}, \\ Pred, \mu \end{pmatrix}$

  P represents the set of places.

  T is the set of transitions.

  I represents the set of input functions.

  O represents the set of output functions.

  H represents the inhibitor functions.

  Par represents the parameter set for this Petri net.

  Pred represents the predicates defining how the parameter set can be distributed.

  μμ represents the set of markings.

  ΠΠ represents the priority function

Petri nets are enabled with right amount of tokens.

No additional transitions area enabled.

t1t_1 has lowest priority at 1.

t2t_2 has lowest priority at 2.

t3 and t4t_3 and t_4 have the highest priority at 3.

If we start initial marking μ=1,0,0,1.$\mu = (1,0,0,1).$

Only t2t_2 will be enabled.

t1t_1 cannot be enabled.

Figure 9.27
Priority-based Petri net.

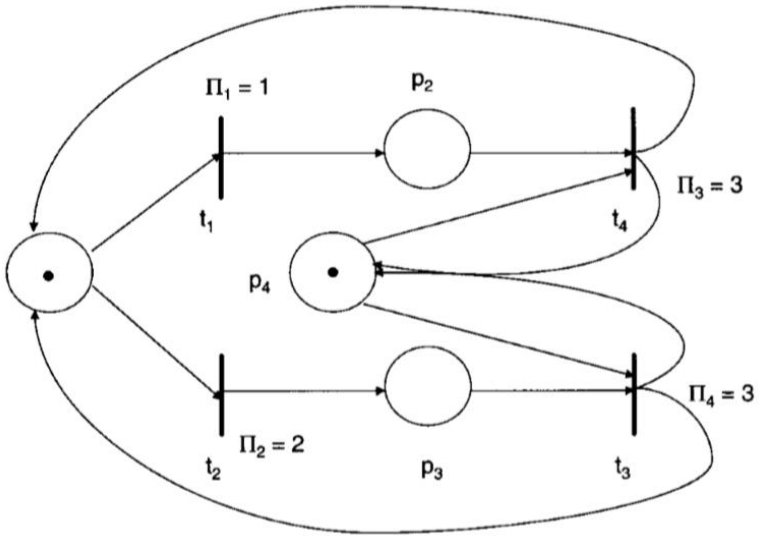
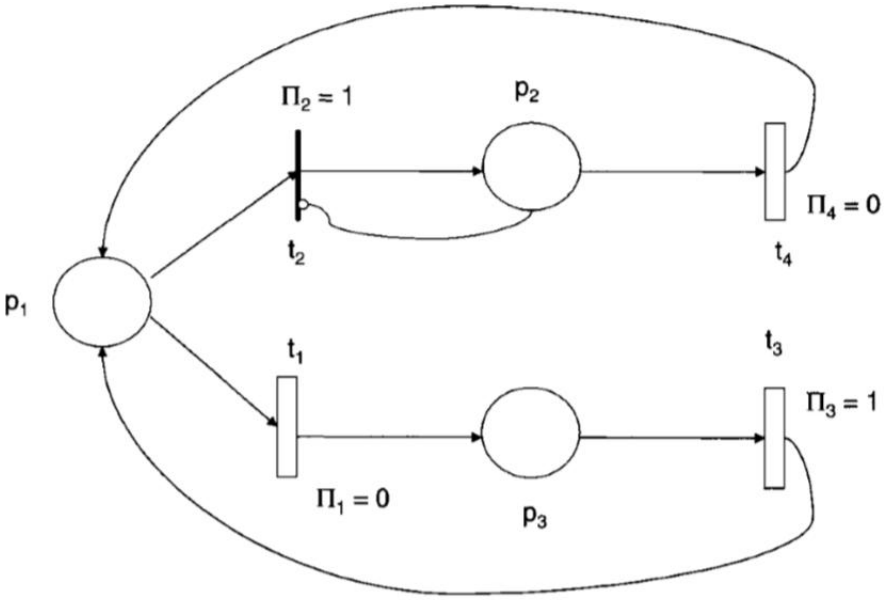


Figure 9.28
Timed and priority net.



Contention, confusion and concurrency condition can also be defined.

This picture has both time and priority features.

System can toggle between two events.

Inhibitor blocks the immediate transition at t2t_2 .

In this way it allows t1t_1 to get the service.

Summary

- We have learnt about the priority-based Petri nets.

- Credits: Paul Fortier book.

Module No.256:

Overview

- Here, we will learn about the colored Petri nets.

Colored Petri nets

- Different token types.
- Colored tokens are used to represent graphically.

  P represents the set of places.

  T is the set of transitions.

  I represents the set of input functions.

  O represents the set of output functions.

  H represents the inhibitor functions.

  Par represents the parameter set for this Petri net.

  Pred represents the predicates defining how the parameter set can be distributed.

  μ^{μ} represents the set of markings.

  Π^{Π} represents the priority function

- Cardinality of the token is represented by all colors.
- All definitions of petri nets can be redefined now.
- And firing rules.

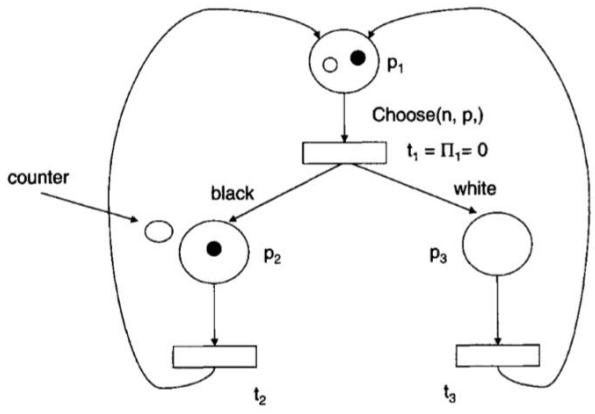


Figure 9.29
Generalized Petri net.

Two token types.

Transitions can have priority and time associated with them.

t1t_1 has priority and time.

Arc from p1p_1 to t1t_1 has condition choose.

Other arcs are used to select specific token.

- We can model about any complex condition in systems.

Summary

- We have learnt about the colored Petri nets.
- Credits: Paul Fortier book.

Module No.257:

Overview

- Here, we will learn about the Generalized Petri nets .

Generalized Petri nets

- Rate or weight transition function.
- Wtkμ$^{W(t_k\mu)}$ must be defined.
- Wtk$^{W(t_k)}$ can refer to this when function needs no definition.
- This is rate of transition.

P represents the set of places.

T is the set of transitions.

I represents the set of input functions.

O represents the set of output functions.

H represents the inhibitor functions.

Par represents the parameter set for this Petri net.

Pred represents the predicates defining how the parameter set can be distributed.

μ$^{\mu}$ represents the set of markings.

Π$^{\Pi}$ represents the priority function

Generalized Petri nets

- To compute time to fire transition an exponential function is used.
- Rate must be define.
- Computed timer value is decremented until 0.
- System state changes from one state to another.
- **Generalized Petri nets**

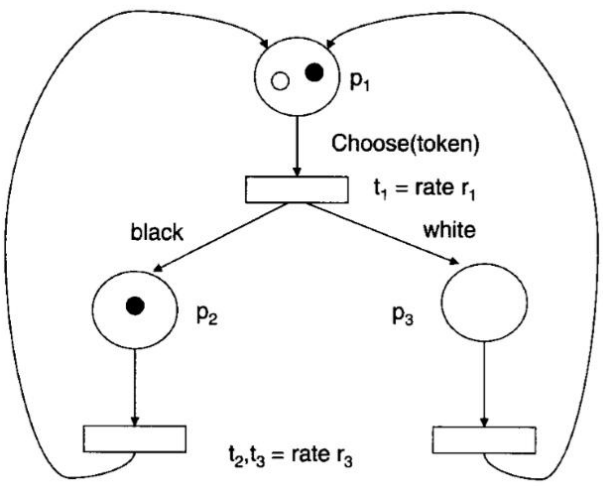


Figure 9.30 *Generalized Petri Net.*

initial marking is μoμ_o = (2,0,0).

t1 t_1 in enabled then it will compute transition rate using mean rate r1.r_1.

Once timer value reaches 0 then we will use the selected token to choose the path.

- Each state will not be computed at the same time.

Summary

- We have learnt about the generalized Petri nets.
- Credits: Paul Fortier book.

Module No.258:

(blank)

Wish you best of luck ..share with others ,

THANK YOU FOR VISTING US , REMEMBER US IN PRAYERS